



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное бюджетное образовательное учреждение
высшего образования

«УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет ИСТ

Кафедра Информационные системы

К ЗАЩИТЕ ДОПУСТИТЬ

Зав. кафедрой

_____ / А.А.Романов /
подпись инициалы, фамилия

« _____ » _____ 2026 г.

БАКАЛАВРСКАЯ РАБОТА

Вид ВКР (дипломный проект (работа) / бакалаврская работа / магистерская диссертация)

Тема _____ Разработка модуля алгоритма случайного леса для оценки
достоверности филогенетических деревьев на языке
Python

Студент _____ / Малафеев Л.С. /
подпись инициалы, фамилия

Обозначение ВКР БР - УлГТУ - 09.03.04 - 22/2533 - 2026 Группа ПИБд-41
для технических направлений подготовки (специальностей)

Направление подготовки (специальность) 09.03.04 Программная
инженерия (профиль «Программная инженерия»)
код, наименование

Руководитель ВКР _____ / Н.В. Корунова /
подпись, дата инициалы, фамилия

Нормоконтроль _____ / А.А.Романов /
подпись, дата инициалы, фамилия

Ульяновск
2026 г.



Направление подготовки (специальность) 09.03.04 Программная инженерия (профиль «Программная инженерия»)

« » 2026 г.

указать вид ВКР (дипломный проект (работа) / бакалаврская работа / магистерская диссертация)

Тема ВКР _____ Разработка модуля алгоритма случайного леса для
оценки достоверности филогенетических деревьев на языке Python _____
утверждена приказом по УлГТУ от « 11 » февраля _____ 2026 г. № 337 _____

Срок сдачи обучающимся законченной ВКР

Исходные данные к ВКР _____ Разработать модуль алгоритма случайного леса для оценки достоверности филогенетических деревьев, обеспечивающий загрузку и валидацию множественных выравниваний нуклеотидных последовательностей, загрузку и построение филогенетических деревьев, автоматическое извлечение топологических и статистических признаков, выполнение ML-предсказания уровня бутстреп-поддержки узлов с использованием предобученной модели случайного леса, интерактивную визуализацию дерева с цветовой индикацией достоверности узлов и всплывающими пояснениями о вкладе ключевых признаков

Содержание расчетно-пояснительной записки (перечень подлежащих разработке вопросов) Анализ модуля алгоритма случайного леса для оценки

достоверности филогенетических деревьев; Проектом автоматизации процесса оценки достоверности филогенетических деревьев; Реализация проекта модуля алгоритма случайного леса для оценки достоверности филогенетических деревьев; Оценка эффективности проекта модуля алгоритма случайного леса для оценки достоверности филогенетических деревьев

Перечень графического материала (с точным указанием обязательных чертежей) Диаграмма «КАК ЕСТЬ»; декомпозиция диаграммы «КАК ЕСТЬ»; диаграмма «КАК НАДО»; декомпозиция диаграммы «КАК НАДО»; диаграмма вариантов использования (Use-Case); er-модель; схема данных; блок-схемы алгоритмов; диаграммы классов; диаграмма компонентов; диаграмма развертывания; экранные формы контрольного примера; графики результатов нагрузочного тестирования.

Календарный график работы над ВКР на весь период (с указанием сроков выполнения и содержания отдельных этапов)

№ этапа	Содержание этапа	Срок выполнения
1	Анализ предметной области и планирование	13.02.2026 – 28.02.2026
2	Проектирование программного обеспечения	01.03.2026 – 14.03.2026
3	Разработка программного обеспечения	15.03.2026 – 05.04.2026
4	Тестирование и корректировка программного обеспечения	06.04.2026 – 10.04.2026
5	Оформление документации и защита проекта	11.04.2026 – 15.04.2026

Дата выдачи задания «___» _____ 2026 г.

Руководитель _____ /Н.В. Корунова /
должность, учёная степень, ученое звание подпись инициалы, фамилия

Задание принял к исполнению _____ /Л.С. Малафеев /
подпись обучающегося инициалы, фамилия

АННОТАЦИЯ

Целью работы является повышение скорости оценки достоверности филогенетического дерева и надёжности ветвей путём применения алгоритма случайного леса для многомерного анализа молекулярных последовательностей и параметров построенного дерева.

Работа состоит из четырех основных разделов. Первый раздел посвящён анализу предметной области, выявлению вычислительных ограничений существующих статистических методов и формированию комплекса функциональных и нефункциональных требований к разрабатываемому модулю. Во втором разделе описывается проектирование системы. Представлены функциональная структура, инфологическая модель данных, алгоритмическое и программное обеспечение, а также архитектура настольного приложения. Третий раздел описывает реализацию проекта, включает контрольный пример работы системы и подробные протоколы комплексного. В четвёртом разделе проводится оценка эффективности проекта, анализируется качество проектного решения и демонстрируется фактическое ускорение вычислений по сравнению с традиционными подходами.

Результатом работы является настольный программный продукт на языке python, который обеспечивает загрузку и валидацию выравниваний и филогенетических деревьев, извлекает топологические и статистические признаки и с помощью обученной модели случайного леса предсказывает уровень достоверности внутренних узлов дерева. Приложение предоставляет исследователям интерактивный графический интерфейс с цветовой индикацией уровня поддержки узлов, всплывающими пояснениями о вкладе ключевых признаков и возможностью экспорта результатов, что позволяет многократно сократить время анализа при сохранении высокой точности прогнозирования.

СОДЕРЖАНИЕ

АННОТАЦИЯ.....	4
СОДЕРЖАНИЕ.....	5
ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ, СОКРАЩЕНИЙ, ТЕРМИНОВ	8
ВВЕДЕНИЕ.....	9
РАЗДЕЛ 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ФОРМИРОВАНИЕ ТРЕБОВАНИЙ К РАЗРАБАТЫВАЕМОЙ СИСТЕМЫ.....	11
1.1 Описание предметной области	11
1.1.1 Описание филогенетических деревьев и используемых данных	11
1.1.2 Описание информационных процессов оценки достоверности филогенетических деревьев	11
1.2. Анализ проблем оценки достоверности филогенетических деревьев и существующих решений	13
1.2.1. Анализ проблем существующих методов оценки достоверности филогенетических деревьев	13
1.2.2. Анализ существующих решений в области оценки достоверности филогенетических деревьев	14
1.2.3. Формирование предложений по разработке модуля оценки достоверности филогенетических деревьев с использованием алгоритма случайного леса	16
1.3. Постановка задачи разработки модуля оценки достоверности филогенетических деревьев	16
1.3.1. Цели и задачи разработки модуля оценки достоверности филогенетических деревьев на основе алгоритма случайного леса.....	16
1.3.2. Спецификация функциональных требований к модулю оценки достоверности филогенетических деревьев.....	17

1.3.3. Спецификация и обоснование нефункциональных требований к модулю оценки достоверности филогенетических деревьев.	19
1.4. Календарно-ресурсное планирование разработки модуля оценки достоверности филогенетических деревьев.	20
РАЗДЕЛ 2. ПРОЕКТ АВТОМАТИЗАЦИИ ПРОЦЕССА ОЦЕНКИ ДОСТОВЕРНОСТИ ФИЛОГЕНЕТИЧЕСКИХ ДЕРЕВЬЕВ.....	22
2.1 Функциональная структура.....	22
2.2 Информационное обеспечение	24
2.2.1 Инфологическая модель данных	24
2.2.2 Схема данных и реквизитный состав таблиц	25
2.2.3 Выходные данные системы	26
2.3. Алгоритмическое обеспечение	26
2.3.1 Блок-схемы алгоритмов	26
2.3.2 Формулы расчётов показателей	30
2.4 Программное обеспечение	32
2.4.1 Описание используемых технологий, фреймворков и библиотек	32
2.4.2 Структура программного обеспечения	33
2.4.3 Спецификации программных модулей	40
2.4.4 Структура диалога с пользователем	43
2.5 Обеспечение информационной безопасности	44
2.5.1 Распределение прав доступа и ответственности	44
2.5.2 Методы защиты информации.....	45
2.6 Техническое обеспечение	46
2.6.1 Архитектура системы и диаграмма развертывания.....	46
2.6.2 Технические характеристики используемых средств.....	46
РАЗДЕЛ 3. РЕАЛИЗАЦИЯ ПРОЕКТА.....	48
3.1 Контрольный пример.....	48

3.2 Тестирование программного обеспечения	53
3.2.1 Протокол модульного тестирования ядра обработки данных	53
3.2.2 Протокол функционального тестирования	54
3.2.3 Протокол тестирования пользовательского интерфейса	56
3.2.4 Протокол тестирования модели машинного обучения.....	58
3.2.5 Протокол тестирования надёжности и обработки исключительных ситуаций	59
РАЗДЕЛ 4. ОЦЕНКА ЭФФЕКТИВНОСТИ ПРОЕКТА.....	61
4.1. Анализ качества проектного решения	61
4.2 Расчёт экономических результатов от внедрения	63
ЗАКЛЮЧЕНИЕ	65
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	67
ПРИЛОЖЕНИЕ А. ТЕСТ-КЕЙСЫ.....	71
ПРИЛОЖЕНИЕ Б. РЕКВИЗИТНЫЙ СОСТАВ ТАБЛИЦ	81
ПРИЛОЖЕНИЕ В. БЛОК-СХЕМЫ.....	83
ПРИЛОЖЕНИЕ Г. РИСУНКИ РЕЗУЛЬТАТОВ ТЕСТОВ	85
ПРИЛОЖЕНИЕ Д. ЛИСТИНГ КОДА.....	87
ЛИСТ С ЗАЯВЛЕНИЕМ О САМОСТОЯТЕЛЬНОСТИ РАБОТЫ.....	131

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ, СОКРАЩЕНИЙ, ТЕРМИНОВ

ВКР – выпускная квалификационная работа.

ПО – программное обеспечение.

IDE – интегрированная среда разработки.

ML – машинное обучение.

RF – случайный лес, алгоритм машинного обучения на основе ансамбля деревьев решений.

MAE – средняя абсолютная ошибка, метрика качества регрессионной модели.

RMSE – среднеквадратичная ошибка, метрика, штрафующая за большие отклонения.

R^2 – коэффициент детерминации, показатель доли объяснённой дисперсии.

FASTA – простой текстовый формат, в котором биологические последовательности записываются строками; чаще всего его используют для нуклеотидов и белков [20].

NEXUS – формат, который подходит для хранения выравниваний, деревьев и дополнительной информации, нужной в филогенетическом анализе [21].

Newick – компактная скобочная запись дерева, где структура ветвления задаётся через вложенные группы таксонов [21].

Филогенетическое дерево – схема в виде дерева, с помощью которой показывают предполагаемые эволюционные связи между таксонами [5; 18].

Бутстреп-анализ – способ проверки устойчивости узлов дерева, при котором из исходного выравнивания много раз собираются псевдовыборки и заново строятся деревья [8].

ВВЕДЕНИЕ

В современном мире филогенетический анализ широко применяется для того, чтобы по нуклеотидным последовательностям восстановить вероятные эволюционные связи между организмами. При этом в биоинформатике важно не только построить само дерево, но и понять, насколько можно доверять его ветвям и внутренним узлам [5; 8]. Традиционно для этих целей применяется бутстреп-анализ, однако его результаты часто требуют дополнительной интерпретации, а устойчивость оценок сильно зависит от объёма и качества исходных данных. Основным недостатком классического подхода является высокая вычислительная сложность: для получения достоверных результатов требуется выполнение от 100 до 1000 и более итераций построения дерева, что приводит к значительным затратам времени и ресурсов. Существующие программные средства не используют в полной мере возможности современных методов интеллектуального анализа данных. В связи с этим возникает необходимость в разработке более быстрых, устойчивых и информативных методов анализа. Практическая значимость работы заключается в многократном сокращении затрат времени исследователей по сравнению с традиционными статистическими подходами при сохранении точности прогнозирования [5; 6; 8].

Объектом исследования является процесс оценки достоверности филогенетических деревьев в задачах биоинформатики и молекулярной биологии.

Предметом исследования выступают методы и алгоритмы оценки достоверности филогенетических деревьев, включая классические статистические подходы и методы машинного обучения.

Целью работы является повышение скорости оценки достоверности филогенетического дерева и надёжности ветвей путём применения алгоритма случайного леса для многомерного анализа молекулярных последовательностей и параметров построенного дерева

Для достижения поставленной цели необходимо решить следующие задачи:

- а) провести сравнительный анализ предметной области;
- б) сформировать обучающую выборку;
- в) разработать и обучить модель машинного обучения на базе алгоритма случайного леса;
- г) спроектировать модульную архитектуру настольного приложения;
- д) реализовать графический интерфейс;
- е) разработать и выполнить комплексный план тестирования.

Технологический стек и инструментарий включают:

- а) язык программирования Python 3.x;
- б) библиотеки для обработки биологических данных BioPython, DendroPy [7; 18; 20; 21];
- в) библиотеки для машинного обучения и анализа данных scikit-learn, NumPy, Pandas [10; 13; 16; 24];
- г) фреймворк для создания графического интерфейса PyQt5 [23];
- д) библиотеку для кастомной отрисовки и визуализации Matplotlib [11];
- е) средства сериализации моделей joblib.

Результатом работы является настольный программный продукт, который обеспечивает автоматическую загрузку и валидацию множественных выравниваний и филогенетических деревьев, извлекает предметно-ориентированные признаки и с помощью обученной модели случайного леса предсказывает уровень достоверности внутренних узлов дерева. Интерактивная визуализация с цветовой индикацией и пояснительными подсказками обеспечивает исследователям наглядное принятие решений. Комплексное тестирование подтвердило полную корректность работы системы, её отказоустойчивость и строгое соответствие всем заявленным функциональным и нефункциональным требованиям.

РАЗДЕЛ 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ФОРМИРОВАНИЕ ТРЕБОВАНИЙ К РАЗРАБАТЫВАЕМОЙ СИСТЕМЫ

1.1 Описание предметной области

1.1.1 Описание филогенетических деревьев и используемых данных

Филогенетические деревья являются основным инструментом анализа эволюционных связей между организмами, генами или белками. Они представляют собой древовидные структуры, в которых вершины соответствуют видам или их предкам, а рёбра отражают степень эволюционного различия. Каждое филогенетическое дерево строится на основе молекулярных данных, таких как нуклеотидные или аминокислотные последовательности. Для анализа используется множественное выравнивание последовательностей, позволяющее выявить сходства и различия между исследуемыми объектами.

Основными элементами филогенетического дерева являются:

- а) листья, соответствующие наблюдаемым организмам или последовательностям;
- б) внутренние узлы, отражающие гипотетических предков;
- в) ветви, характеризующие эволюционное расстояние;
- г) длины ветвей, отражающие степень различий.

1.1.2 Описание информационных процессов оценки достоверности филогенетических деревьев

Процесс оценки достоверности филогенетических деревьев в существующей практике представляет собой последовательность операций, выполняемых с использованием специализированного программного обеспечения. Основным пользователем данного процесса является исследователь в области биоинформатики или молекулярной биологии. Его

задачами являются подготовка входных данных, запуск вычислений и интерпретация полученных результатов.

Существующий процесс включает следующие этапы:

- а) Подготовка данных – пользователь загружает множественное выравнивание молекулярных последовательностей;
- б) Построение филогенетического дерева – выполняется с использованием алгоритмов;
- в) Оценка достоверности – применяется бутстреп-анализ, основанный на многократной генерации псевдовыборок и повторном построении дерева;
- г) Анализ результатов – пользователь получает значения достоверности для узлов дерева и выполняет их интерпретацию.

Таким образом, существующий информационный процесс оценки достоверности филогенетических деревьев является ресурсоемким и недостаточно интерпретируемым, что определяет необходимость его совершенствования. Диаграмма «КАК ЕСТЬ» и декомпозиция диаграммы «КАК ЕСТЬ» представлены на рисунках 1 и 2.

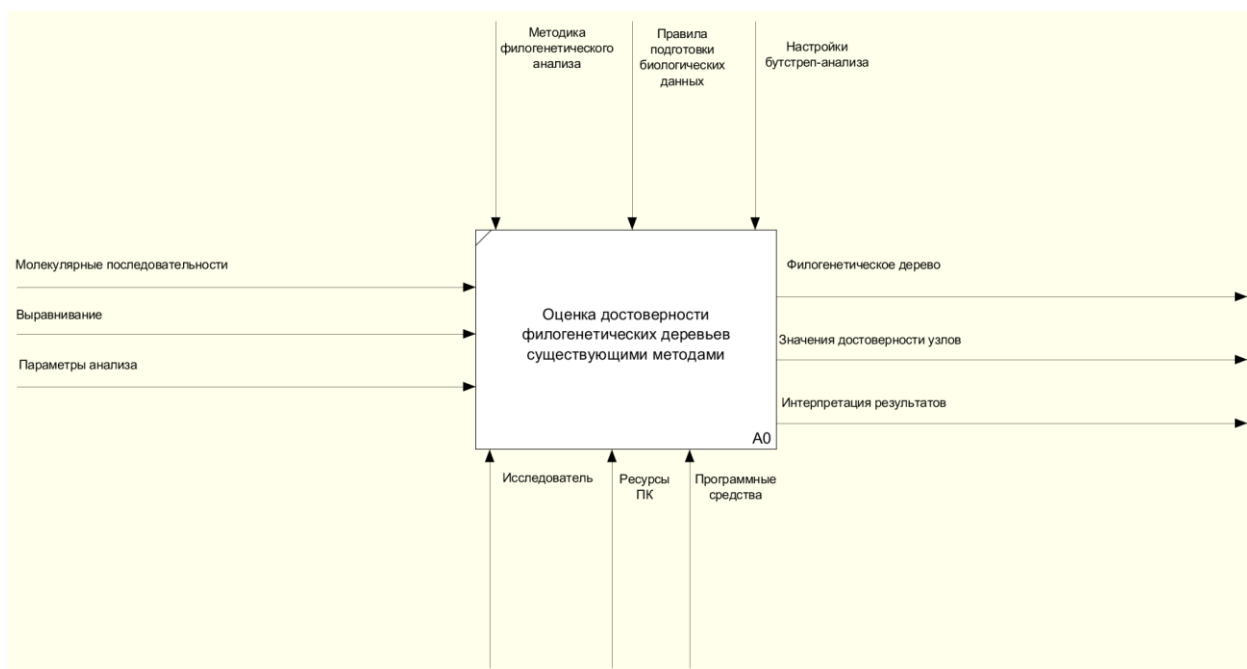


Рисунок 1 – Диаграмма "КАК ЕСТЬ"

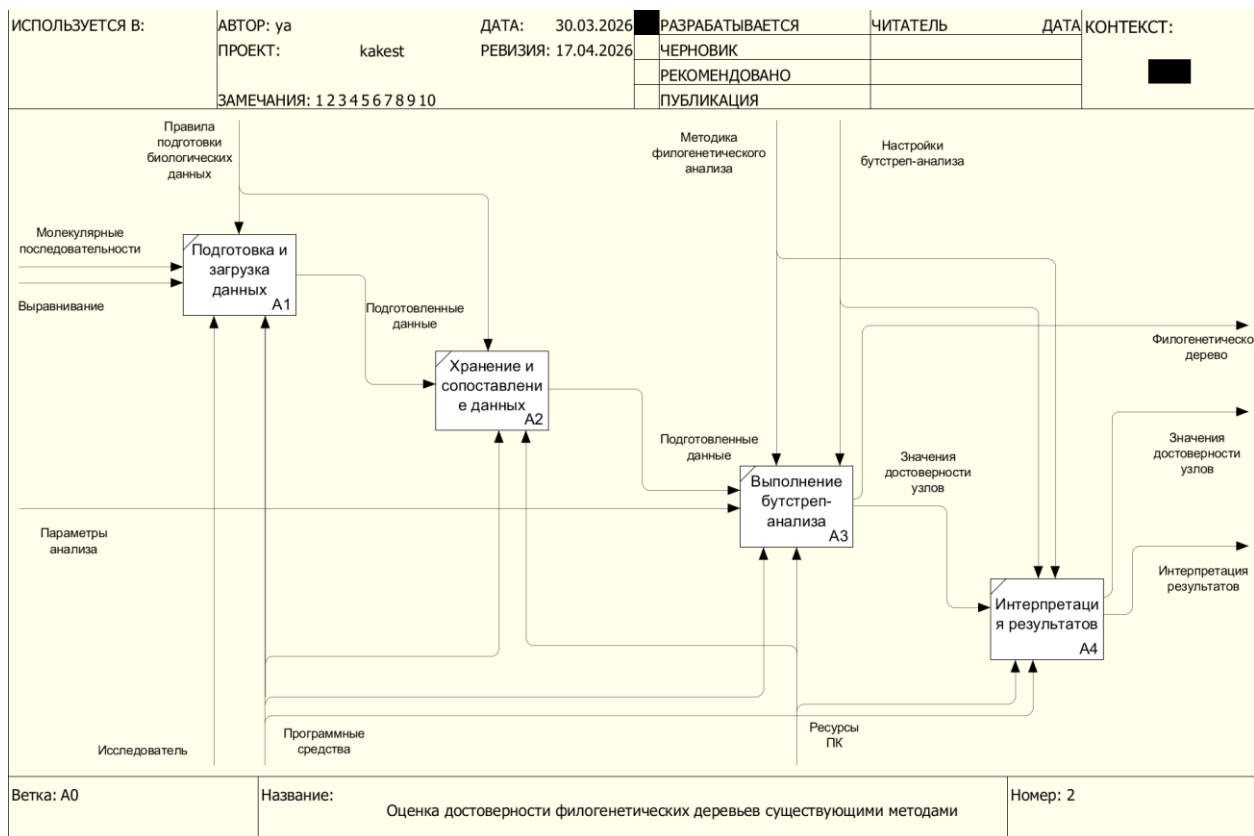


Рисунок 2 – Декомпозиция диаграммы "КАК ЕСТЬ"

1.2. Анализ проблем оценки достоверности филогенетических деревьев и существующих решений

1.2.1. Анализ проблем существующих методов оценки достоверности филогенетических деревьев

Существующие методы оценки достоверности филогенетических деревьев, несмотря на широкое применение, имеют ряд существенных недостатков, влияющих на эффективность и точность анализа. Наиболее распространённым методом является бутстреп-анализ, основанный на многократной генерации псевдовыборок исходного выравнивания и повторном построении дерева. Такой способ действительно даёт численную оценку поддержки узлов, но на практике у него есть несколько слабых мест. Прежде всего, метод получается затратным по времени: обычно приходится выполнять от 100 до 500 и более повторных построений дерева, а на больших наборах данных это быстро нагружает компьютер. Ещё один момент — не всегда понятно,

почему конкретный узел получил высокую или низкую поддержку: бутстреп показывает частоту появления клады в повторных выборках, но не раскрывает вклад отдельных признаков. Поэтому исследователю часто приходится отдельно разбирать результаты вручную. Кроме того, многие готовые программы почти не дают возможности гибко анализировать признаки, влияющие на оценку, и не используют методы машинного обучения для пояснения результата [8; 16].

Итог такой: классические подходы к оценке достоверности филогенетических деревьев остаются полезными, но их ограничения заметны. Они требуют времени, не всегда объясняют полученные значения и плохо настраиваются под конкретную задачу. Поэтому логично рассмотреть вариант, где статистическая оценка дополняется анализом признаков и моделью машинного обучения [6; 8; 16].

1.2.2. Анализ существующих решений в области оценки достоверности филогенетических деревьев

Для построения деревьев и проверки их поддержки сейчас применяются разные программные пакеты. В работе отдельно рассматривались MEGA, UGENE, RAxML и PhyML, так как эти решения часто встречаются в задачах молекулярной филогенетики и хорошо описаны в источниках [9; 12; 15; 17].

MEGA удобна тем, что имеет понятный графический интерфейс и позволяет строить деревья несколькими распространёнными методами, включая расчёт бутстреп-поддержки. Для учебных и средних по объёму задач это хороший вариант, но при больших наборах последовательностей скорость и ресурсы становятся ограничением [12].

UGENE представляет собой более общую среду для работы с биологическими данными: в ней можно выполнять операции с последовательностями, запускать отдельные инструменты анализа и строить деревья. При этом специальных средств для детального объяснения достоверности узлов в ней, по сути, недостаточно [15].

RAxML, наоборот, больше рассчитан на производительные вычисления и работу с крупными филогенетическими задачами. Он умеет быстрее оценивать поддержку, но логика остаётся статистической: программа показывает итоговые значения, а не объясняет, какие характеристики дерева и данных на них повлияли [17].

В таблицу 1 были сведены аналоги.

Таблица 1 – Сравнение аналогов

Приложение	MEGA	UGENE	PhyML	RAxML	Разрабатываемый Модуль
Построение и базовая оценка дерева	+	+	+	+	+
Различные методы построений	+	+	-	-	-
Анализ качества данных и признаков	-		-	-	+
ML-предсказание поддержки с интерпретацией	-	-	-	-	+
Работа с большими данными	-	-	-	+	-
Анализ информативных признаков	-	-	-	-	+
Объяснение предсказаний	-	-		-	+
Визуализация результатов	+	+	+	-	+

1.2.3. Формирование предложений по разработке модуля оценки достоверности филогенетических деревьев с использованием алгоритма случайного леса

После сравнения методов и готовых программ становится видно, что модуль лучше строить не как замену всему филогенетическому ПО, а как дополнительный инструмент для оценки поддержки. В сфере анализа данных всё чаще используют модели, которые работают сразу с набором признаков и могут находить неочевидные зависимости. Для данной задачи это важно, потому что достоверность узла зависит не от одного показателя, а от сочетания характеристик выравнивания и структуры дерева [6; 16].

На рынке биоинформатических программ основной упор обычно сделан на построение деревьев и расчёт классических статистических оценок. Инструментов, которые применяют машинное обучение именно для прогноза поддержки узлов и одновременно помогают понять результат, заметно меньше. Поэтому разработка отдельного модуля со случайным лесом выглядит обоснованной и закрывает практический пробел в таких системах [6; 16].

Выбор алгоритма случайного леса обусловлен его устойчивостью к шуму, способностью работать с большим количеством признаков и возможностью оценки важности признаков, что обеспечивает интерпретируемость результатов. В качестве технологии разработки предлагается использовать язык программирования Python, который широко применяется в области анализа данных и машинного обучения [6; 22].

1.3. Постановка задачи разработки модуля оценки достоверности филогенетических деревьев

1.3.1. Цели и задачи разработки модуля оценки достоверности филогенетических деревьев на основе алгоритма случайного леса

Цель работы – повышение качества оценки достоверности филогенетического дерева и надёжности ветвей путём применения алгоритма

случайного леса для многомерного анализа молекулярных последовательностей и параметров построенного дерева.

Задачи:

- а) провести анализ существующих методов и программных средств для оценки достоверности филогенетических деревьев, выделить их достоинства и недостатки, обосновать актуальность разработки альтернативного подхода;
- б) сформировать и подготовить обучающую выборку филогенетических деревьев с соответствующими молекулярными выравниваниями и известными значениями бутстреп-поддержки для каждого узла;
- в) разработать модель машинного обучения на языке Python, включающую вычисление информативных биологических признаков, нормализацию признаков, обучение алгоритма случайного леса и применение обученной модели для предсказания поддержки узлов.

1.3.2. Спецификация функциональных требований к модулю оценки достоверности филогенетических деревьев.

Основные функции системы включают:

- а) загрузка и валидация входных данных. Вход: файлы с молекулярными последовательностями, файлы деревьев. Выход: корректные структуры данных;
- б) построение филогенетического дерева. Вход: выравнивание последовательностей. Выход: построенное дерево;
- в) вычисление информативных признаков. Вход: дерево и выравнивание. Выход: таблица признаков;
- г) нормализация признаков. Вход: таблица признаков. Выход: нормализованные признаки;
- д) предсказание достоверности узлов. Вход: нормализованные признаки. Выход: значения достоверности узлов;

е) визуализация результатов. Вход: дерево и значения достоверности. Выход: визуализированное дерево с аннотациями.

Диаграмма «КАК НАДО» и декомпозиция диаграммы «КАК НАДО» представлены на рисунках 3 и 4.

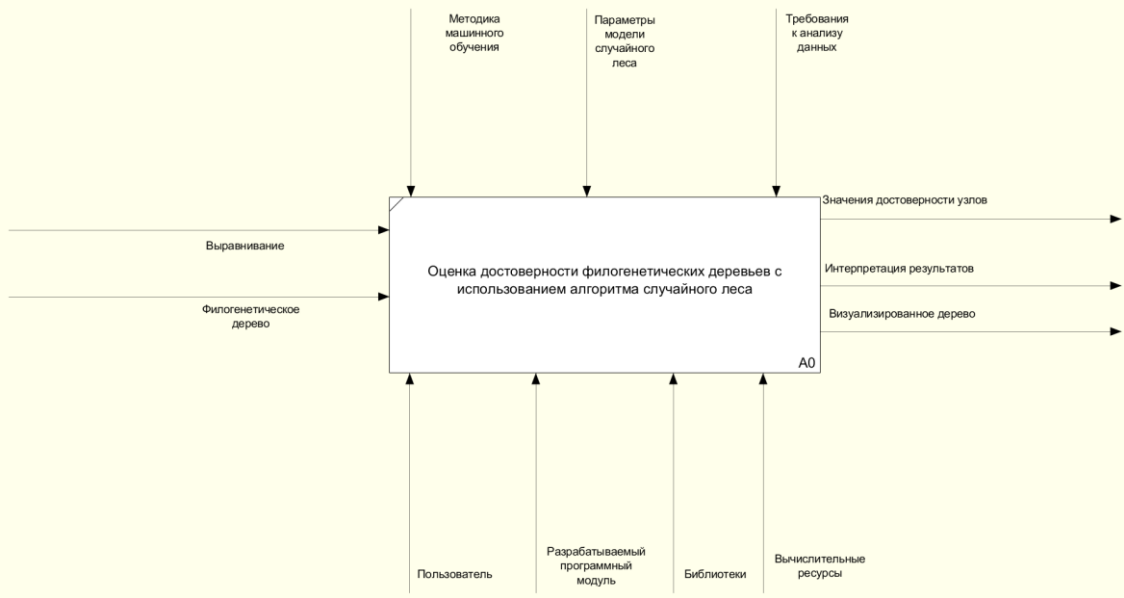


Рисунок 3 – Диаграмма "КАК НАДО"

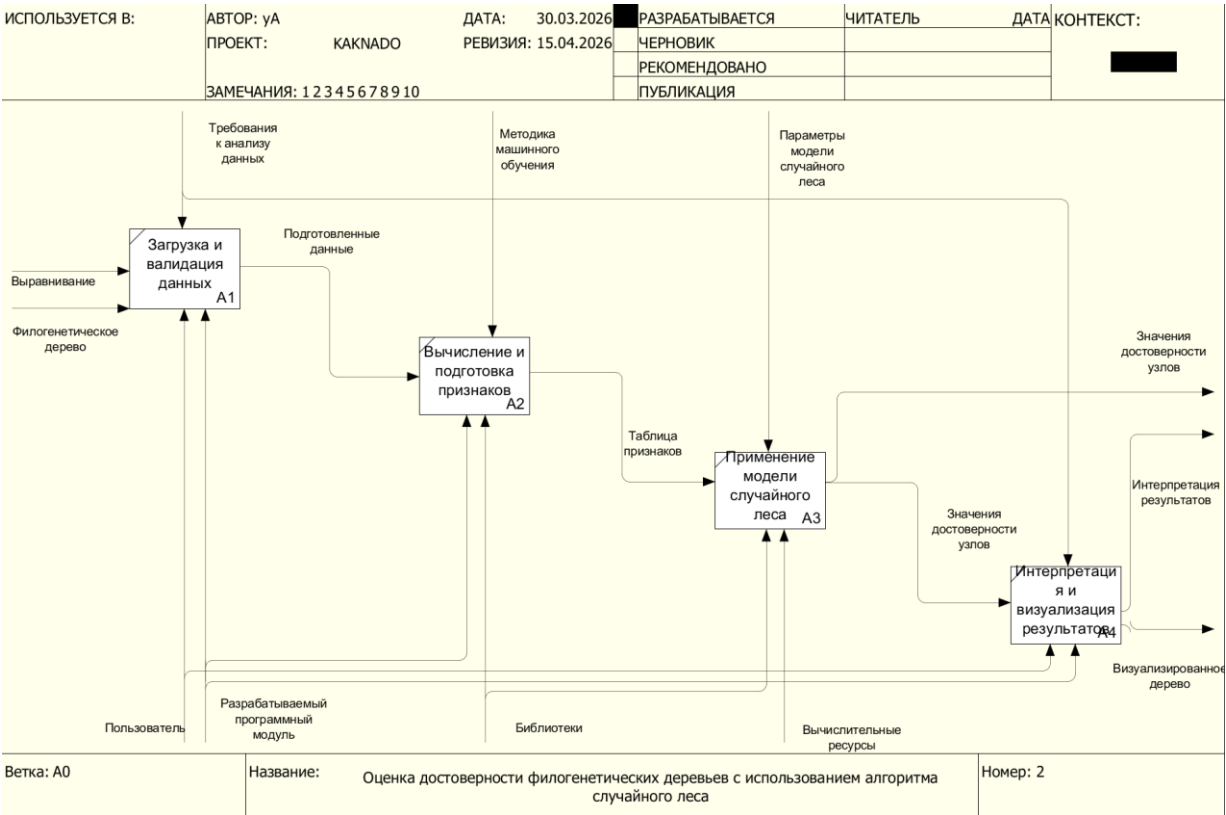


Рисунок 4 – Декомпозиция диаграммы "КАК НАДО"

1.3.3. Спецификация и обоснование нефункциональных требований к модулю оценки достоверности филогенетических деревьев.

Требования к программно-технической среде. Разрабатываемый модуль должен функционировать на персональных компьютерах пользователей и не требовать специализированного оборудования.

Минимальные требования к аппаратному обеспечению:

- а) процессор: не ниже Intel Core i5 или аналог;
- б) оперативная память: не менее 4 ГБ;
- в) свободное место на диске: не менее 2 ГБ.

Программные требования:

- а) операционная система: Windows 10 и выше, Linux или macOS;
- б) язык разработки: Python;
- в) используемые библиотеки: NumPy, Pandas, Scikit-learn, BioPython, DendroPy, PyQt5, Matplotlib;
- г) наличие установленной среды выполнения Python и необходимых зависимостей.

Сетевая архитектура:

- а) система может функционировать автономно без подключения к сети;
- б) доступ к сети требуется только для загрузки исходных биологических данных.

Пользовательские требования. Требования к быстродействию:

- а) система должна обеспечивать обработку средних по размеру наборов данных в разумное время.

Требования к надежности:

- а) система должна корректно обрабатывать некорректные входные данные и выдавать понятные сообщения об ошибках;
- б) при возникновении ошибок система не должна завершаться аварийно;
- в) результаты анализа должны сохраняться без потери данных.

Требования к информационной безопасности:

- а) обработка данных должна выполняться локально;

б) защита от некорректного ввода и ошибок пользователя.

Требования к эргономике и удобству использования:

- а) интерфейс должен быть интуитивно понятным;
- б) минимальное количество действий для запуска анализа;
- в) наличие визуализации результатов
- г) поддержка русского языка интерфейса.

1.4. Календарно-ресурсное планирование разработки модуля оценки достоверности филогенетических деревьев.

Для разработки модуля оценки достоверности филогенетических деревьев используется итерационная модель жизненного цикла, позволяющая поэтапно реализовывать функциональность системы и корректировать её на основе промежуточных результатов.

Основные этапы жизненного цикла включают:

- а) анализ требований;
- б) проектирование системы;
- в) разработку программного модуля;
- г) тестирование;
- д) внедрение и подготовку документации.

В процессе разработки системы можно выделить следующие ограничения:

- а) ограниченные вычислительные ресурсы персонального компьютера;
- б) ограниченность объема доступных обучающих данных;
- в) временные ограничения, связанные со сроками выполнения проекта.

Основные риски проекта:

- а) риск недостаточного качества обучающей выборки, что может привести к снижению точности модели;
- б) риск высокой вычислительной сложности, особенно при обработке больших деревьев;

в) риск ошибок при обработке биологических данных, связанных с некорректными форматами.

Для снижения рисков предусматриваются следующие меры:

- а) использование проверенных библиотек;
- б) проведение тестирования на различных наборах данных;
- в) реализация валидации входных данных;
- г) разработка понятного пользовательского интерфейса.

Календарный план сведён в таблицу 2.

Таблица 2 – Календарный план

№ этапа	Содержание этапа	Срок выполнения
1	Анализ предметной области и планирование	13.02.2026 – 28.02.2026
2	Проектирование программного обеспечения	01.03.2026 – 14.03.2026
3	Разработка программного обеспечения	15.03.2026 – 05.04.2026
4	Тестирование и корректировка программного обеспечения	06.04.2026 – 10.04.2026
5	Оформление документации и защита проекта	11.04.2026 – 15.04.2026

РАЗДЕЛ 2. ПРОЕКТ АВТОМАТИЗАЦИИ ПРОЦЕССА ОЦЕНКИ ДОСТОВЕРНОСТИ ФИЛОГЕНЕТИЧЕСКИХ ДЕРЕВЬЕВ

2.1 Функциональная структура

Основным актором системы является пользователь, взаимодействующий с программным модулем через интерфейс приложения. Пользователь инициирует загрузку исходных данных, запуск анализа и просмотр результатов. Функциональные возможности системы включают следующие основные группы операций:

- а) Подготовка и загрузка данных:
 - 1) загрузка файла филогенетического дерева;
 - 2) загрузка файла выравнивания;
 - 3) проверка корректности входных данных;
 - 4) предобработка данных.
- б) Построение филогенетического дерева:
 - 1) построение дерева на основе выравнивания последовательностей;
 - 2) применение метода присоединения соседей;
 - 3) визуализация построенного дерева.
- в) Анализ достоверности дерева:
 - 1) выполнение анализа дерева;
 - 2) извлечение информативных признаков;
 - 3) применение обученной модели машинного обучения;
 - 4) формирование оценок достоверности узлов.
- г) Представление результатов:
 - 1) просмотр результатов анализа;
 - 2) отображение значений достоверности на дереве;
 - 3) отображение значений достоверности в табличном виде;
 - 4) сохранение результатов анализа.

Вся структура представлена на диаграмме вариантов использования на рисунке 5.



Рисунок 5 – Диаграмма вариантов использования

2.2 Информационное обеспечение

2.2.1 Инфологическая модель данных

В основе модели лежит процесс анализа филогенетического дерева, включающий хранение исходных данных, параметров анализа и результатов оценки достоверности.

В модели выделены следующие основные сущности:

- а) ModelRF – модель случайного леса. Сущность предназначена для хранения информации об используемой модели машинного обучения.
- б) Analysis – анализ. Сущность отражает факт выполнения анализа филогенетического дерева.
- в) Tree – филогенетическое дерево. Сущность описывает характеристики построенного дерева.
- г) Alignment – выравнивание последовательностей. Сущность хранит данные множественного выравнивания.
- д) Sequence – последовательность. Сущность содержит информацию о биологических последовательностях.
- е) Branch – ветвь дерева. Сущность описывает отдельные ветви филогенетического дерева.
- ж) BranchAssessment – оценка ветви. Сущность хранит результаты оценки достоверности ветвей.

Связи между сущностями:

- а) одна модель может использоваться в нескольких анализах;
- б) один анализ связан с одним деревом и одной моделью;
- в) одно дерево содержит множество ветвей;
- г) каждая ветвь имеет одну оценку;
- д) одно выравнивание содержит множество последовательностей;
- е) результаты анализа связывают дерево, ветви и вычисленные оценки достоверности.

Связи и сущности обозначены на рисунке 6.

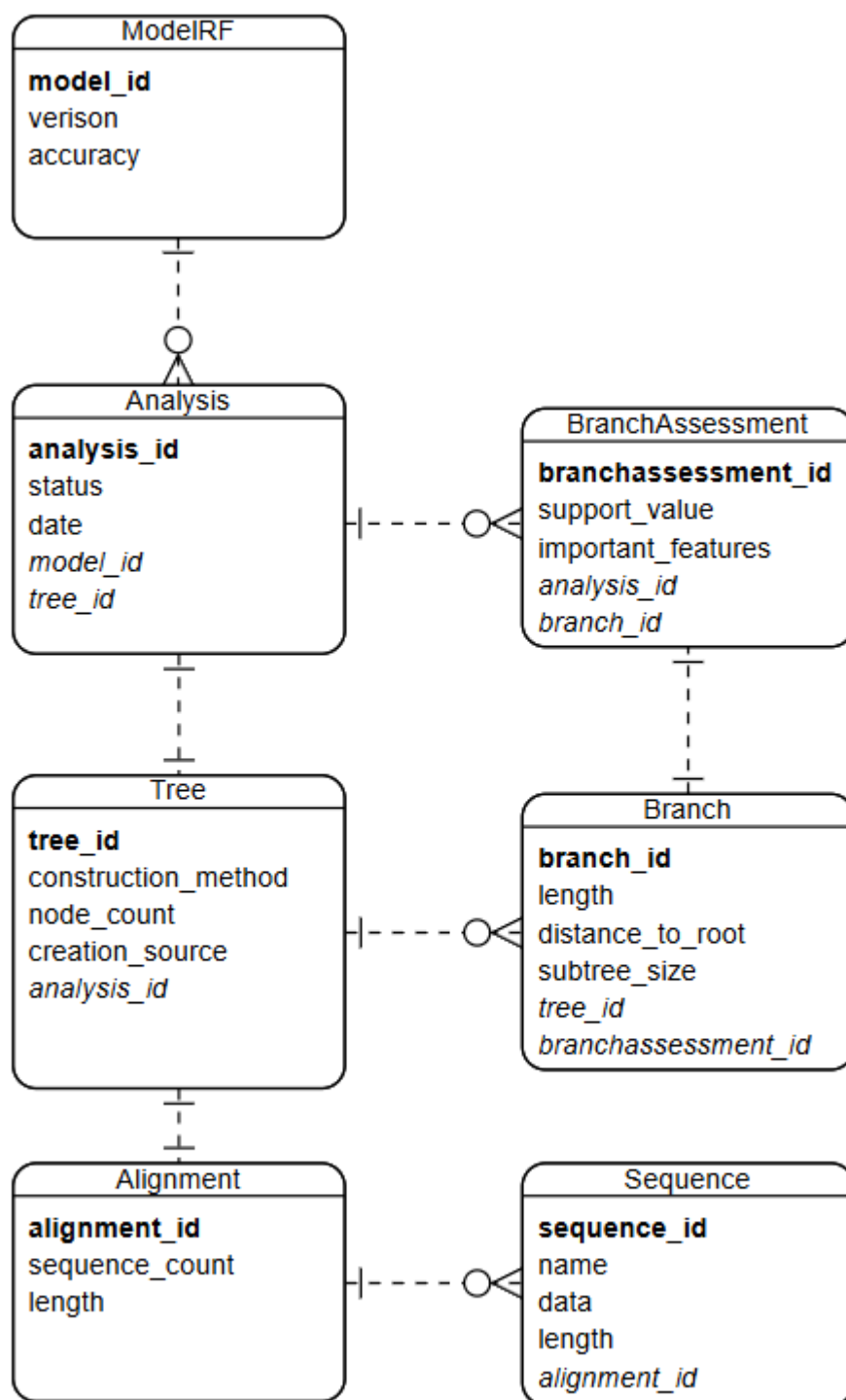


Рисунок 6 – ER-диаграмма

2.2.2 Схема данных и реквизитный состав таблиц

Схема данных и реквизитный состав таблиц находятся в приложении Б. Там описаны все сущности и атрибуты.

2.2.3 Выходные данные системы

Разрабатываемая система формирует результаты анализа в виде экранных форм, отображаемых пользователю через интерфейс приложения. Выходные данные представлены в удобной для пользователя форме и обеспечивают наглядную интерпретацию результатов анализа.

К основным выходным данным относятся:

- а) визуализированное филогенетическое дерево с отображением значений достоверности ветвей;
- б) числовые значения достоверности узлов;
- в) информация о значимости признаков, используемых моделью машинного обучения;
- г) возможность сохранения результатов анализа.

Реквизитный состав таблиц находится в приложении Б.

2.3. Алгоритмическое обеспечение

2.3.1 Блок-схемы алгоритмов

Для построения дерева используется метод присоединения соседей, представляющий собой итерационный алгоритм кластеризации, основанный на анализе матрицы расстояний между объектами. На первом этапе осуществляется получение входных данных в виде множественного выравнивания последовательностей. На основе этих данных вычисляется матрица попарных расстояний, отражающая степень различия между последовательностями. Далее производится инициализация кластеров, при которой каждая последовательность рассматривается как отдельный кластер. Основным циклом алгоритма выполняется до тех пор, пока количество кластеров не станет равным двум [9]. Блок-схема алгоритма представлена на рисунке 7.

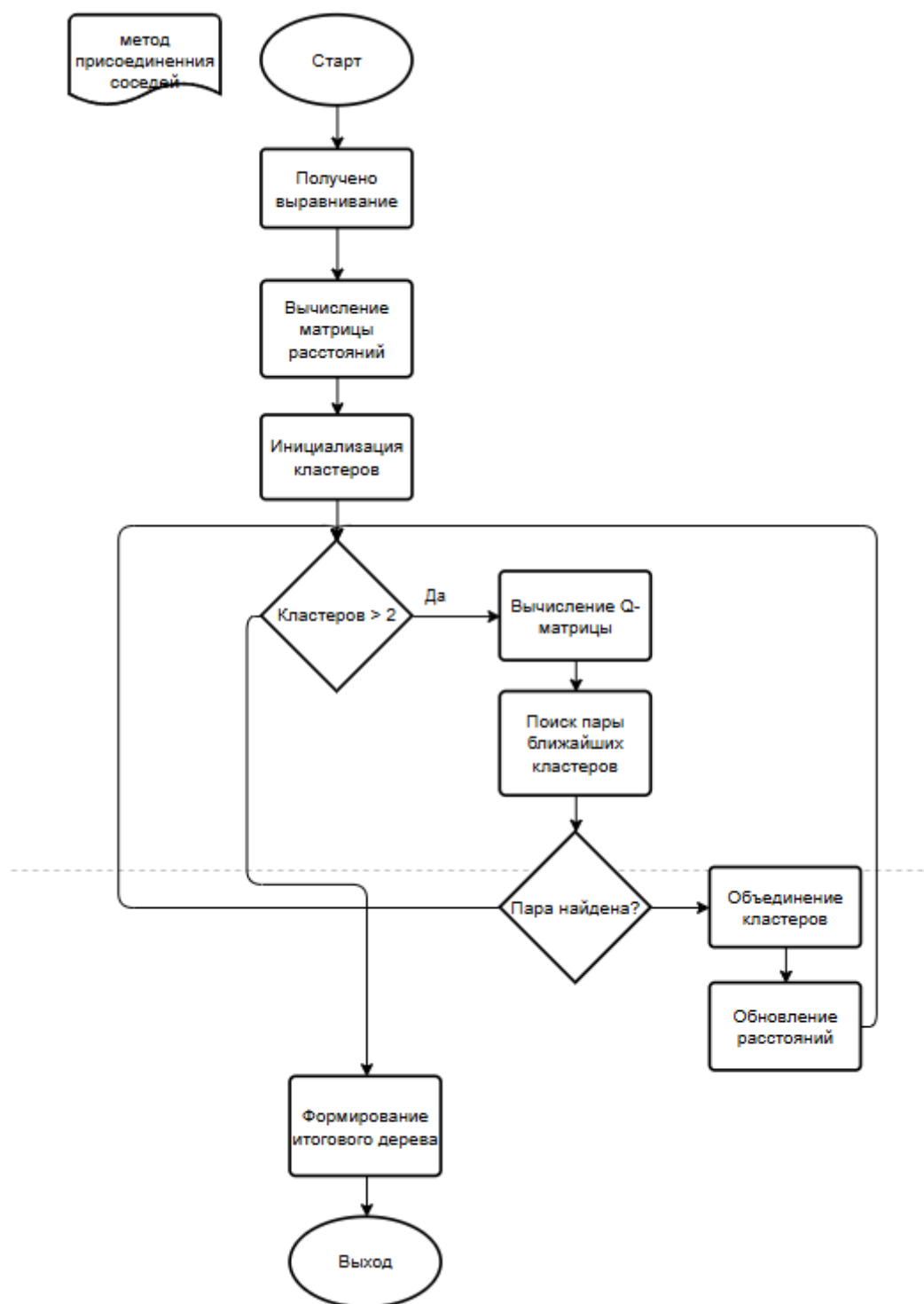


Рисунок 7 – Блок-схема "метод присоединения соседей"

Алгоритм формирования признаков предназначен для извлечения количественных характеристик из структуры филогенетического дерева, которые впоследствии используются в качестве входных данных для модели случайного

леса. На первом этапе осуществляется последовательный обход узлов дерева. Для каждого узла выполняется набор вычислений, направленных на получение информативных характеристик. Блок-схема представлена на рисунке 8.

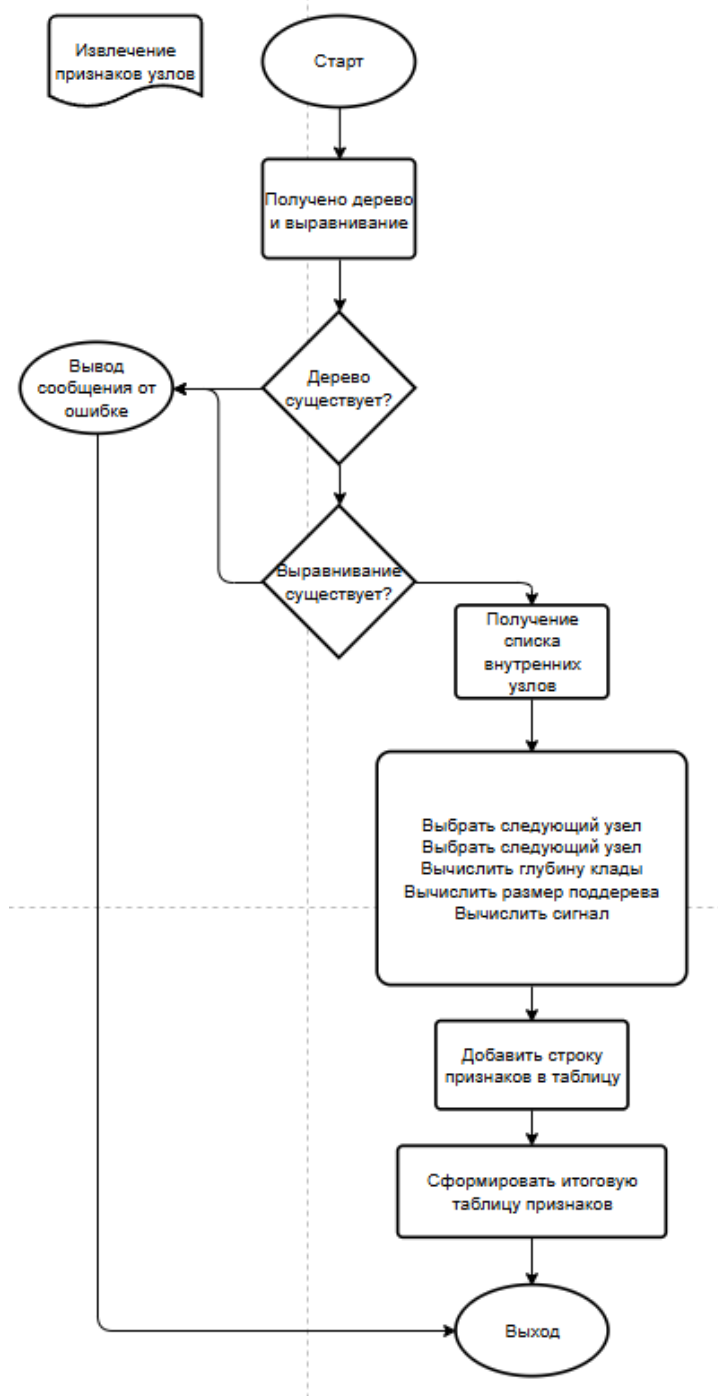


Рисунок 8 – Алгоритм формирования признаков

Алгоритм предсказания достоверности узлов начинается с получения входных данных, включающих филогенетическое дерево и соответствующее множественное выравнивание последовательностей. На их основе выполняется извлечение признаков узлов дерева. Далее осуществляется проверка

корректности формирования таблицы признаков. В случае возникновения ошибки выполнение алгоритма прерывается с выводом соответствующего сообщения пользователю. Блок-схема алгоритма представлена на рисунке 9.

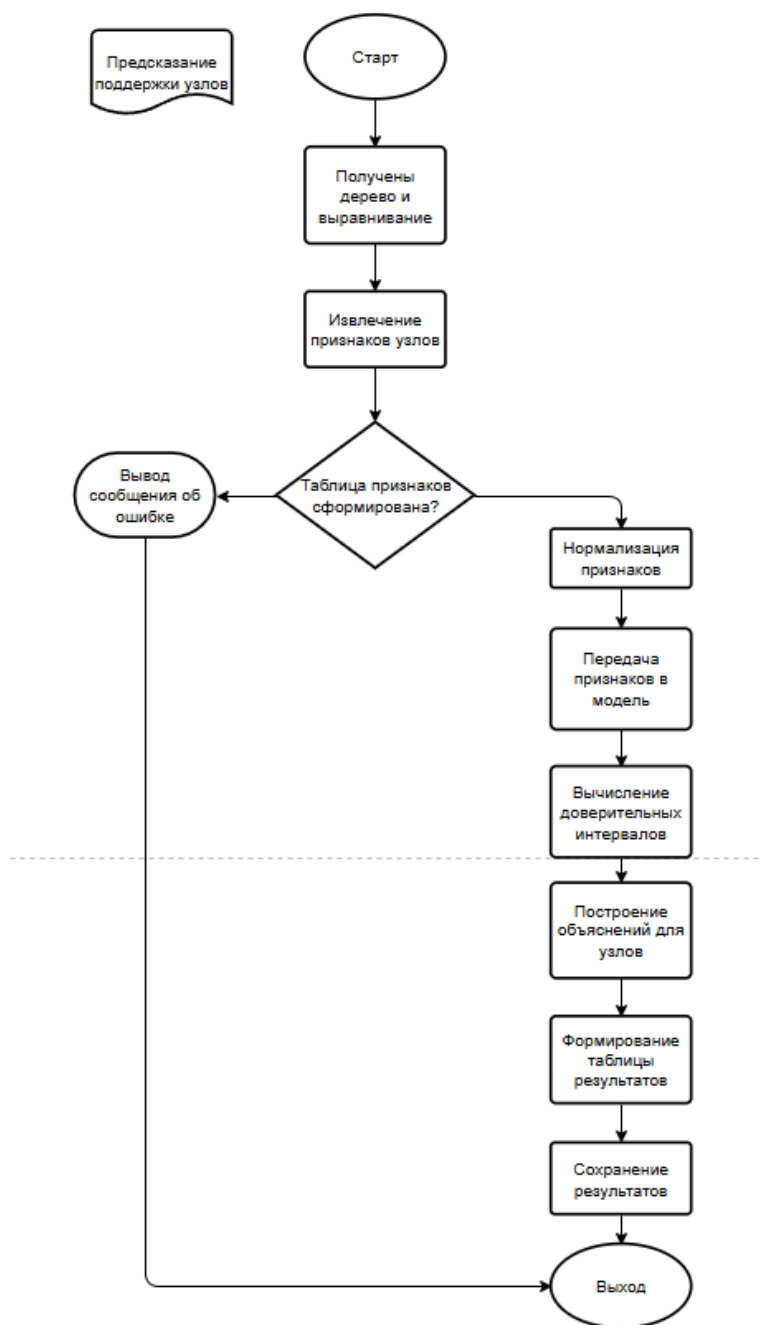


Рисунок 9 – Алгоритм предсказания узлов

Алгоритм получения списка доступных моделей начинается с получения списка доступных моделей, сохранённых в системе. Данный список формируется на основе ранее обученных и сохранённых моделей, доступных для использования. На следующем этапе выполняется проверка наличия доступных

моделей. В случае, если список моделей пуст, пользователю выводится сообщение об отсутствии доступных моделей, и выполнение алгоритма завершается. Если модели доступны, пользователю предоставляется возможность выбора модели из выпадающего списка интерфейса. После выбора модели выполняется проверка корректности выбора. При успешном выборе модели запускается процесс её загрузки. После завершения загрузки выполняется проверка успешности операции. В случае ошибки загрузки пользователю выводится соответствующее сообщение о невозможности загрузки модели. При успешной загрузке выполняется сохранение состояния приложения, включая информацию о выбранной модели. Рисунок с блок-схемой алгоритма списка доступных моделей находится в приложении В.

Алгоритм загрузки входных данных начинается с инициализации процесса загрузки данных пользователем через интерфейс приложения. На первом этапе выполняется выбор файла выравнивания. После выбора осуществляется попытка загрузки файла в систему. Далее выполняется проверка успешности загрузки. В случае возникновения ошибки пользователю выводится сообщение о невозможности загрузки данных, и выполнение алгоритма завершается. При успешной загрузке выполняется проверка корректности структуры данных. Проверка включает анализ формата файла, наличие необходимых данных и соответствие требованиям системы. Если данные некорректны, пользователю выводится сообщение об ошибке. В случае успешной проверки данные сохраняются в системе и становятся доступными для последующей обработки.

2.3.2 Формулы расчётов показателей

В основе алгоритма присоединения соседей лежит минимизация Q -критерия. На каждой итерации выбирается пара таксонов i и j для которой значение $Q(i,j)$ минимально:

$$Q(i, j) = (N - 2) \cdot d(i, j) - \sum_{k=1}^N d(i, k) - \sum_{k=1}^N d(j, k), \quad (1)$$

где N – текущее количество активных кластеров, $d(i, j)$ – расстояние между таксонами i и j .

В качестве функции потерь и критериев остановки обучения ансамбля деревьев решений, а также для итоговой оценки качества регрессии, применяются следующие метрики:

Средняя абсолютная ошибка (MAE):

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|, \quad (2)$$

где, где n – количество внутренних узлов, y_i – истинное значение бутстреп-поддержки, $\hat{y}_i$ – предсказанное моделью значение, $\bar{y}$ – среднее значение.

Коэффициент детерминации (R^2), показывающий долю объясненной дисперсии:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}, \quad (3)$$

где, где n – количество внутренних узлов, y_i – истинное значение бутстреп-поддержки, $\hat{y}_i$ – предсказанное моделью значение, $\bar{y}$ – среднее значение.

2.4 Программное обеспечение

2.4.1 Описание используемых технологий, фреймворков и библиотек

Программное обеспечение разрабатываемой системы реализовано с использованием языка программирования Python и набора специализированных библиотек, обеспечивающих обработку биологических данных, построение филогенетических деревьев, применение методов машинного обучения и организацию пользовательского интерфейса. В процессе разработки используются следующие технологии и программные средства:

- а) Python – основной язык разработки программного модуля. Используется для реализации логики обработки данных, алгоритмов анализа, интеграции библиотек машинного обучения и создания пользовательского интерфейса [22].
- б) NumPy – библиотека для выполнения численных вычислений и работы с многомерными массивами данных. Применяется при вычислении числовых признаков, подготовке входных данных и выполнении математических операций [10].
- в) Pandas – библиотека для обработки и анализа табличных данных. Используется для формирования таблиц признаков, хранения промежуточных результатов анализа и подготовки данных для передачи в модель машинного обучения [13].
- г) Scikit-learn – библиотека машинного обучения, применяемая для загрузки, использования и, при необходимости, повторного обучения модели случайного леса. Данная библиотека обеспечивает средства предсказания значений достоверности узлов, нормализации признаков и оценки качества модели [16; 24].
- д) BioPython – специализированная библиотека для работы с биологическими последовательностями и форматами хранения биоинформатических данных. Используется при чтении и обработке файлов последовательностей и выравниваний [7; 20].

- е) DendroPy – библиотека для работы с филогенетическими деревьями. Применяется для загрузки, анализа структуры дерева, обхода узлов, вычисления характеристик ветвей и извлечения признаков [18; 21].
- ж) PyQt5 – библиотека для разработки графического пользовательского интерфейса. Используется для реализации окон приложения, загрузки файлов, выбора модели, запуска анализа и отображения сообщений пользователю [23].
- з) Matplotlib – библиотека визуализации данных. Применяется для графического отображения филогенетического дерева, результатов анализа и дополнительных визуальных элементов интерфейса [11].

2.4.2 Структура программного обеспечения

В первом фрагменте диаграммы классов представлены элементы пользовательского интерфейса системы, обеспечивающие взаимодействие пользователя с программным модулем и отображение результатов анализа.

Ключевым классом является MainWindow, реализующий основное окно приложения. Данный класс выполняет функции координации пользовательских действий и взаимодействия с сервисным слоем системы. В его обязанности входят загрузка входных данных, запуск построения филогенетического дерева, выполнение анализа и отображение результатов. Класс содержит ссылку на объект состояния приложения AppState, а также использует сервисный слой через объект AnalysisService.

Для визуализации филогенетического дерева используется класс PhyloTreeCanvas, отвечающий за графическое отображение структуры дерева, масштабирование, выделение узлов и отображение значений достоверности. Данный класс инкапсулирует параметры отображения, такие как уровень масштабирования, расстояние между листьями и кэш расположения элементов.

Выбор модели машинного обучения в интерфейсе вынесен в отдельный класс ModelSelectionPresenter. Он хранит доступные модели, отслеживает выбор пользователя и возвращает текущую модель, когда анализ уже нужно запускать.

Также предусмотрена очистка выбора, чтобы при смене данных состояние приложения не оставалось старым [16; 23].

За вывод информации по выбранному узлу отвечает NodeDetailsPresenter. В нём хранится текущий узел и основные сведения по нему, после чего эти данные передаются в элементы интерфейса в понятном для пользователя виде [23].

Журнал сообщений реализован через LogPresenter. По сути, этот класс собирает текстовые записи о действиях системы: загрузке файлов, ошибках, запуске анализа и завершении операций. Пользователь может посмотреть эти сообщения, а приложение при необходимости очищает журнал [23].

Первый фрагмент диаграммы классов представлена на рисунке 10.

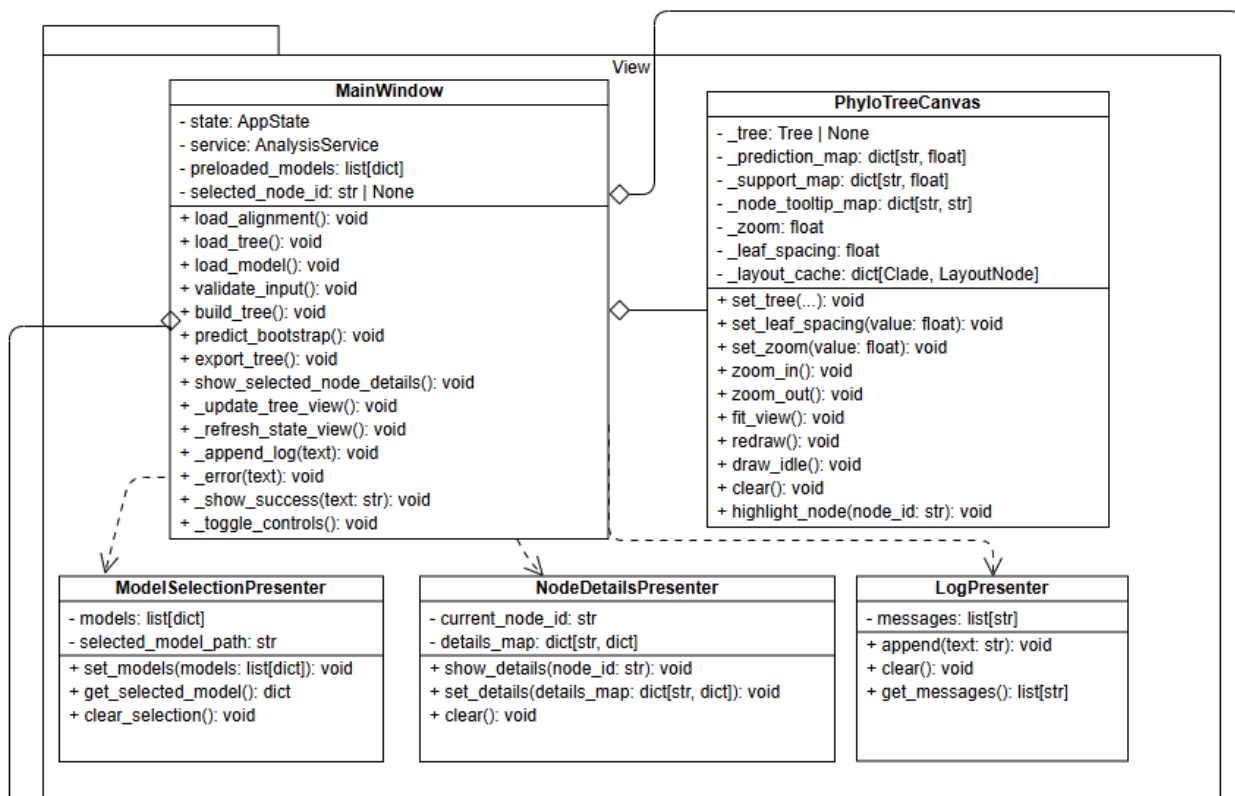


Рисунок 10 – Диаграмма классов. Пользовательский интерфейс

Во второй части диаграммы показаны вспомогательные структуры данных. Они нужны, чтобы хранить результаты промежуточных вычислений и передавать их между интерфейсом, сервисным слоем и моделью без смешивания логики компонентов [22].

Ключевым элементом является класс AppState, реализующий хранение текущего состояния приложения. Он содержит информацию о загруженных файлах, состоянии выполнения операций, параметрах построения дерева, а также результатах анализа. Данный класс используется пользовательским интерфейсом для отслеживания состояния системы и управления доступностью функций. Наличие методов сброса состояния и обработки ошибок позволяет централизованно управлять жизненным циклом данных в приложении.

Класс AnalysisArtifacts предназначен для хранения промежуточных результатов обработки данных. Он инкапсулирует загруженное выравнивание, построенное дерево, сводную информацию о данных, а также вычисленные признаки узлов. Данный класс используется сервисным слоем для передачи результатов между этапами обработки без необходимости повторных вычислений.

Класс NodeFeatureRow предназначен для представления набора признаков, вычисленных для отдельного узла филогенетического дерева. Он содержит идентификатор узла и словарь признаков, отражающих численные характеристики структуры дерева. Данный класс используется на этапе формирования входных данных для модели машинного обучения и предоставляет метод преобразования данных в словарное представление.

Класс PredictionRow используется для хранения результатов предсказания достоверности узлов. Он содержит идентификатор узла, значение достоверности, границы доверительного интервала, а также объяснение, представленное в виде вклада отдельных признаков. Наличие метода преобразования в словарь позволяет удобно передавать данные между компонентами системы и использовать их при визуализации.

Класс TreeBuildResult предназначен для хранения результата построения филогенетического дерева. Он включает в себя само дерево, информацию о методе построения, а также параметры, характеризующие тип оценки достоверности. Данный класс используется при передаче результата из модуля построения дерева в другие компоненты системы.

Второй фрагмент диаграммы классов представлен на рисунке 11.

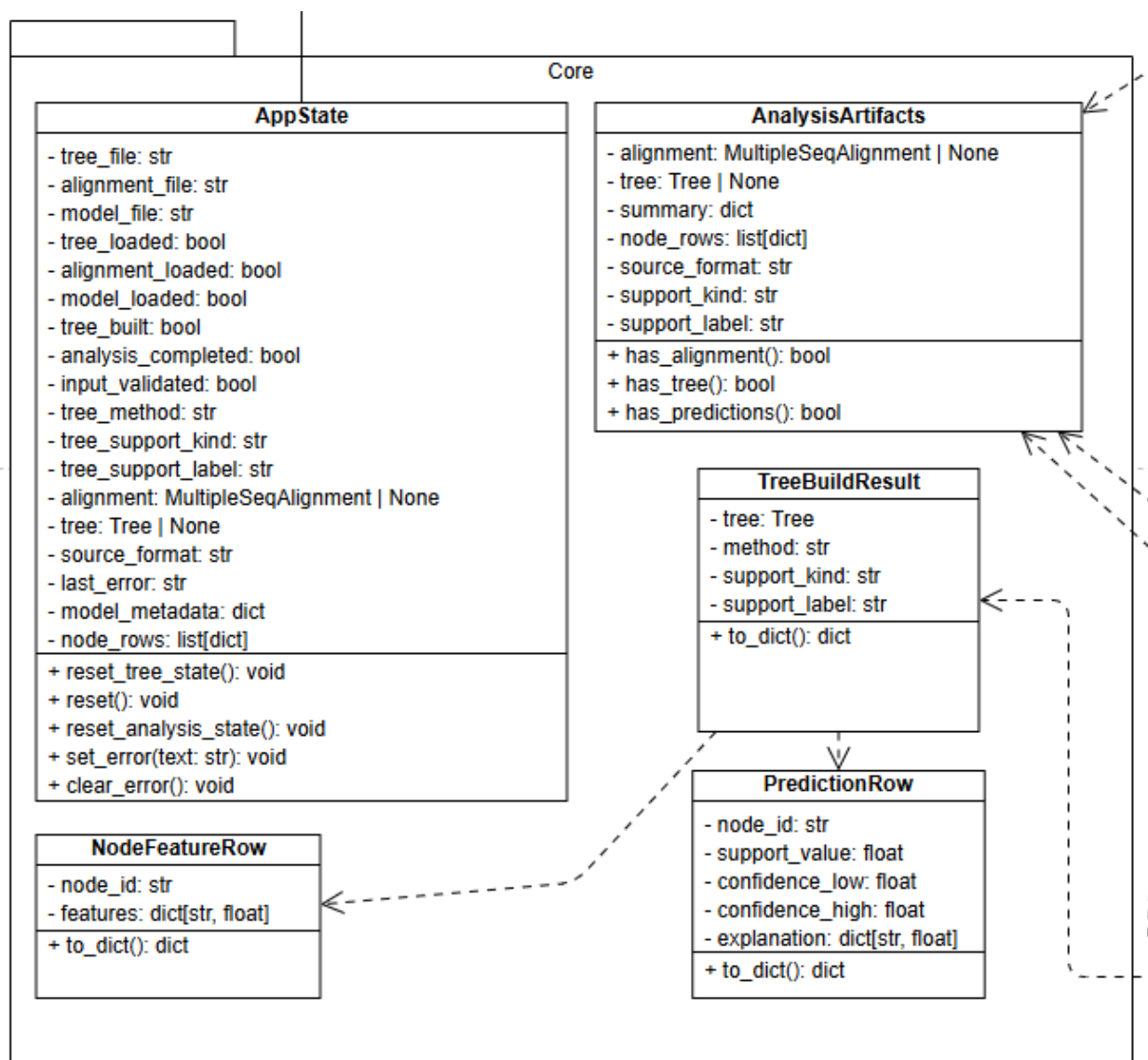


Рисунок 11 – Диаграмма классов. Состояние системы

В третьем фрагменте диаграммы классов представлен сервисный слой системы, обеспечивающий реализацию основной бизнес-логики и координацию выполнения операций обработки данных. Центральным элементом является класс *AnalysisService*, выполняющий роль фасада системы. Данный класс агрегирует специализированные сервисы и предоставляет единый интерфейс для выполнения основных операций.

Класс `AlignmentInspectionService` предназначен для предварительной проверки файлов множественного выравнивания. Он выполняет анализ формата данных и выявление возможных ошибок до этапа загрузки.

`TreeInspectionService` выполняет похожую проверку, но уже для файлов филогенетических деревьев. Он помогает заранее выявить ошибки структуры и проверить, подходит ли формат для дальнейшей обработки в приложении [18; 21].

Загрузка данных разделена между `AlignmentLoadingService` и `TreeLoadingService`. Первый компонент работает с выравниваниями, второй — с деревьями; после чтения и проверки они приводят данные к внутренним объектам `AnalysisArtifacts`, которые дальше используются в анализе [7; 18; 20; 21].

`TreeBuildingService` отвечает за построение филогенетического дерева по выравниванию. В рамках модуля он запускает нужный алгоритм построения и возвращает результат в объекте `TreeBuildResult`, чтобы остальные части программы работали уже с готовой структурой дерева [18; 21].

`ModelLoadingService` нужен для загрузки обученной модели и связанных с ней служебных сведений. Это позволяет отделить работу с файлом модели от интерфейса и самого алгоритма предсказания [16; 24].

Оценка достоверности узлов выполняется через `BootstrapPredictionService`. Этот сервис связывает подготовленные признаки с ML-подсистемой и отвечает за получение прогнозных значений поддержки [6; 16].

Таким образом, сервисный слой получился модульным: каждая крупная операция вынесена в свой класс, а общий запуск проходит через фасад `AnalysisService`. [22].

Третий фрагмент диаграммы классов представлен на рисунке 12.

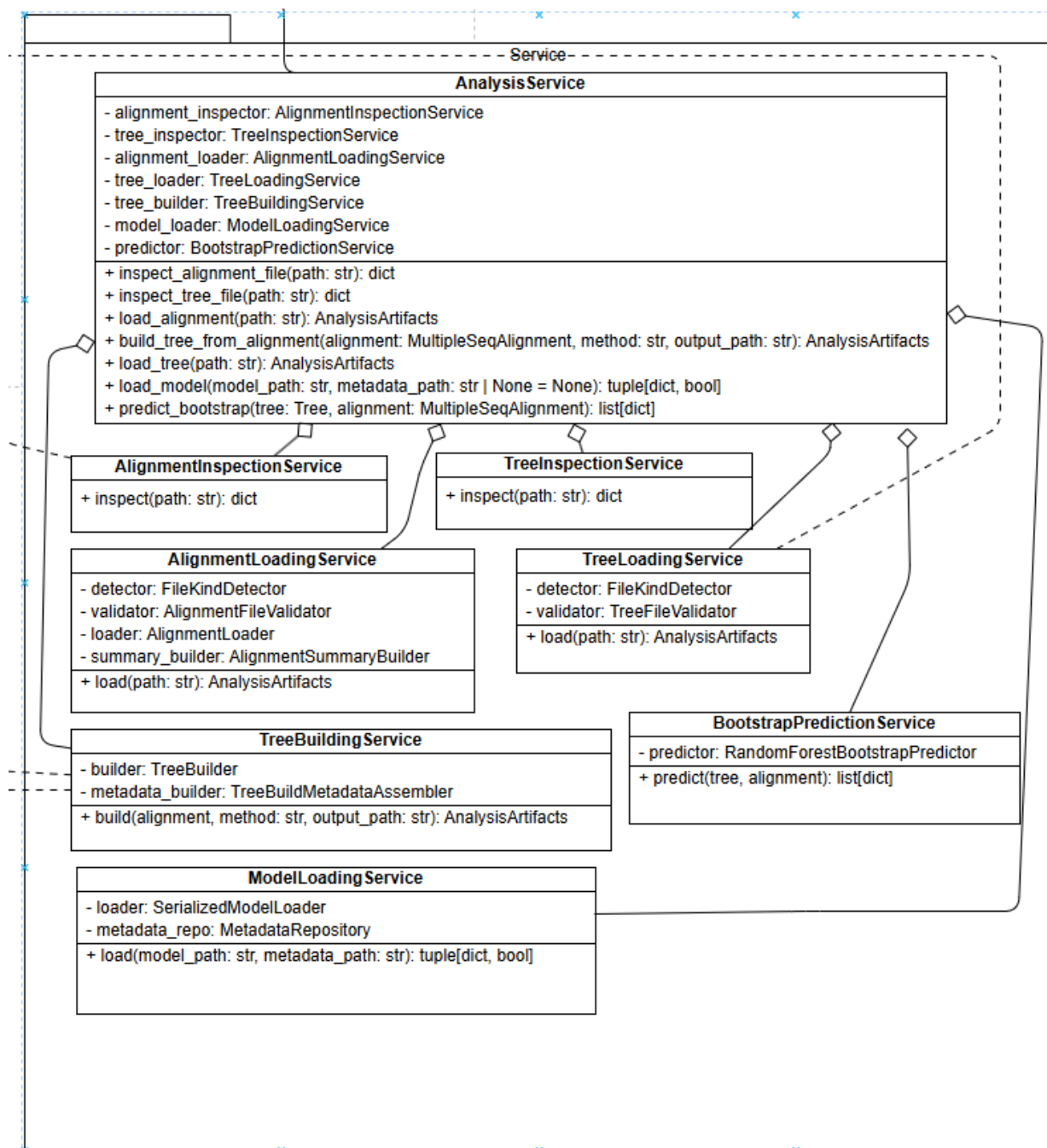


Рисунок 12 – Диаграмма классов. Фрагмент сервис

В четвертом фрагменте диаграммы классов представлены компоненты подсистемы машинного обучения, обеспечивающие выполнение предсказания достоверности узлов филогенетического дерева на основе алгоритма случайного леса.

Ключевым элементом является класс `RandomForestModel`, представляющий обученную модель машинного обучения. Данный класс

инкапсулирует параметры модели, а также предоставляет методы для выполнения предсказаний на основе входных данных. Он используется в процессе анализа для вычисления значений достоверности узлов дерева.

Для подготовки входных данных модели используется класс `FeatureExtractor`, отвечающий за преобразование структуры филогенетического дерева и связанных данных в набор числовых признаков. Класс реализует методы извлечения характеристик узлов, таких как длина ветвей, глубина узла, размер поддерева и другие структурные параметры.

Класс `FeaturePreprocessor` предназначен для предварительной обработки признаков перед подачей в модель машинного обучения. Он выполняет нормализацию, масштабирование и приведение данных к формату, соответствующему требованиям обученной модели.

Выполнение предсказаний реализуется с помощью класса `PredictionEngine`, который обеспечивает взаимодействие между подготовленными признаками и моделью случайного леса. На вход он получает таблицу признаков, а на выходе формирует предсказанные значения поддержки узлов и дополнительные показатели, которые затем используются при отображении результата [16; 24].

`PredictionInterpreter` используется для пояснения результата модели. Он показывает, какие признаки сильнее участвовали в итоговом прогнозе, поэтому пользователь видит не просто число поддержки, а примерное объяснение, за счёт чего оно получилось [6; 16].

`ModelMetadata` хранит служебную информацию о модели: версию, параметры обучения, показатели качества и другие данные, которые нужны при загрузке и показе сведений в интерфейсе. Благодаря этому пользователь может понять, какая именно модель используется в текущем анализе [16; 24].

Четвертый фрагмент диаграммы классов представлен на рисунке 13.

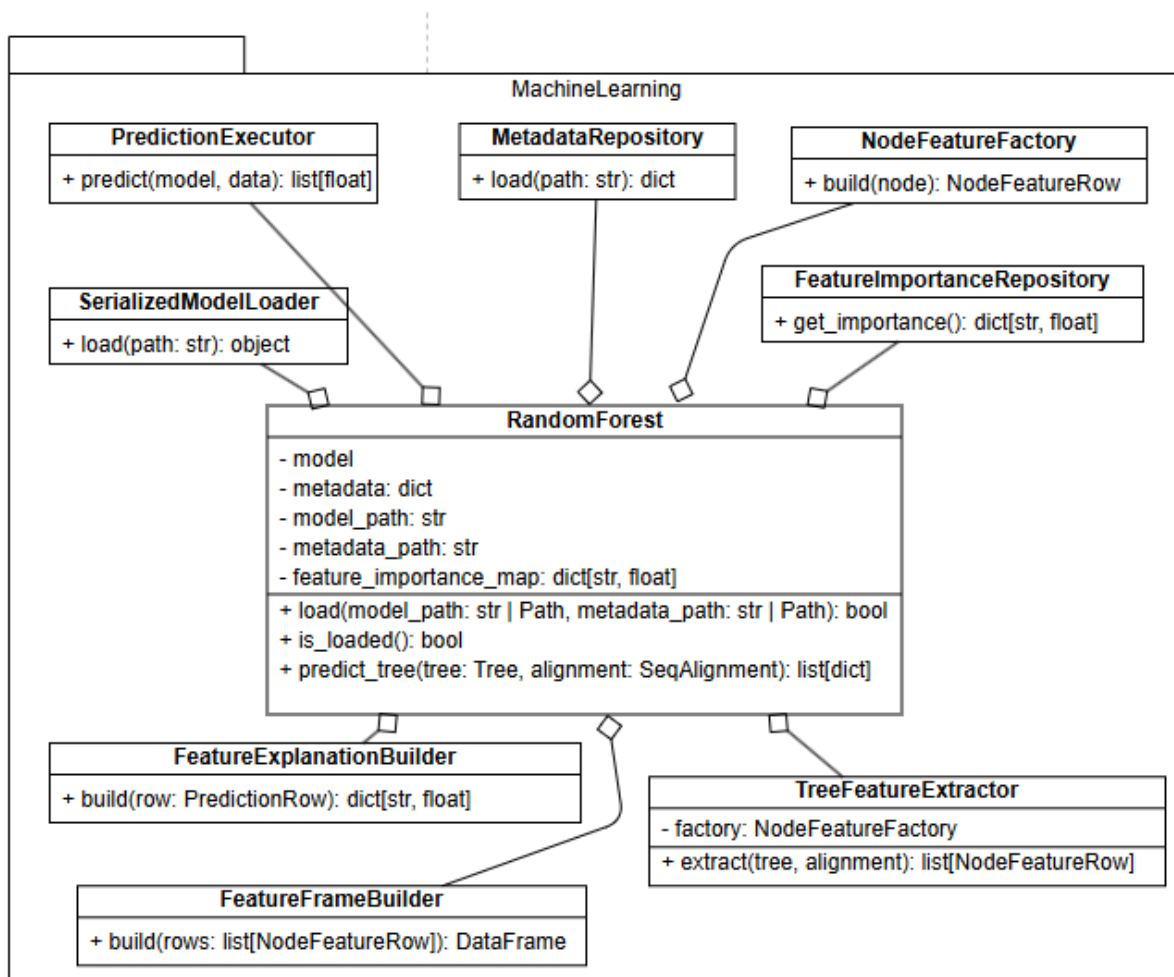


Рисунок 13 – Диаграмма классов. Фрагмент ML

2.4.3 Спецификации программных модулей

Система состоит из модулей. Основным элементом выступает приложение, которое объединяет интерфейс, вычислительное ядро и отдельные функциональные модули [22]. Такой подход обеспечивает разделение ответственности между компонентами, упрощает сопровождение программного продукта и позволяет независимо развивать отдельные части системы.

Внешними по отношению к приложению являются:

- Model RandomForest – сохранённая обученная модель случайного леса;
- InputFiles external – входные файлы, содержащие выравнивания последовательностей и данные филогенетического дерева;
- OutputResults – результирующие данные, формируемые системой;
- Libraries – набор используемых библиотек и внешних программных зависимостей.

Модуль пользовательского интерфейса. Модуль User Interface обеспечивает взаимодействие пользователя с системой. Через данный модуль выполняется загрузка исходных данных, запуск анализа, выбор модели машинного обучения и просмотр результатов работы программы. Интерфейс формирует команды для вычислительного ядра и получает от него данные для отображения. Для реализации данного модуля используется библиотека PyQt5, обеспечивающая создание окон, элементов управления, диалогов выбора файлов и сообщений пользователю.

Ядро анализа. Компонент Phylogenetic Analysis Core является центральным координирующим модулем системы. Он управляет последовательностью выполнения операций, организует обмен данными между остальными компонентами и реализует основную бизнес-логику приложения. Именно через данный модуль объединяются процессы загрузки данных, построения дерева, извлечения признаков, предсказания достоверности и формирования итоговых результатов.

Модуль ввода данных. Компонент Data Input Module предназначен для загрузки и предварительной обработки входных данных. Он получает внешние файлы, содержащие множественное выравнивание последовательностей и, при необходимости, исходное филогенетическое дерево. В рамках данного модуля выполняется чтение файлов, проверка корректности формата и преобразование данных во внутренние структуры программы, пригодные для дальнейшего анализа.

Модуль построения дерева. Компонент Tree Construction Module реализует алгоритмы построения филогенетического дерева на основе входного выравнивания. Этот модуль используется в тех случаях, когда дерево отсутствует во входных данных и должно быть построено автоматически. В качестве вычислительной основы могут применяться специализированные средства биоинформатической обработки, включая библиотеки BioPython, SciPy и другие вспомогательные средства численного анализа [7; 19].

Модуль представления дерева. Компонент Tree Representation Module отвечает за внутреннее представление структуры филогенетического дерева. Он обеспечивает хранение узлов, ветвей и связей между ними в форме, удобной для вычислений, визуализации и извлечения признаков. Данный модуль является промежуточным связующим звеном между построением дерева, анализом его структуры и графическим отображением результатов.

Модуль извлечения признаков. Компонент Feature Extraction Module предназначен для вычисления признаков, характеризующих узлы и ветви филогенетического дерева. В рамках данного модуля из дерева и выравнивания извлекаются количественные параметры, используемые далее в качестве входных данных для модели машинного обучения. К таким признакам могут относиться длины ветвей, глубина узла, размер поддерева и другие структурные характеристики. Для выполнения вычислений применяются библиотеки NumPy, Pandas и DendroPy [10; 13; 18; 21].

Модуль предсказания на основе случайного леса. Компонент Random Forest Prediction Module реализует применение обученной модели случайного леса к подготовленным данным. Модуль получает сформированный набор признаков, загружает модель RandomForest и вычисляет значения достоверности для анализируемых узлов филогенетического дерева. Для реализации данного компонента используется библиотека scikit-learn, обеспечивающая работу с моделью машинного обучения и выполнение предсказаний [16; 24].

Модуль визуализации. Компонент Visualization Module предназначен для графического отображения результатов анализа. Он взаимодействует с модулем представления дерева и формирует визуальную интерпретацию структуры дерева с нанесёнными значениями достоверности. Данный модуль обеспечивает наглядное представление результатов пользователю и используется как часть пользовательского интерфейса.

Модуль формирования результатов. Компонент Result Export Module отвечает за подготовку и вывод итоговых результатов анализа. Он получает данные из вычислительного ядра и модуля предсказания, после чего формирует

результатирующие структуры, пригодные для сохранения, отображения или передачи во внешний файл. Через этот модуль обеспечивается экспорт итоговых оценок достоверности и сопутствующей информации.

Структура представлена на рисунке 14.

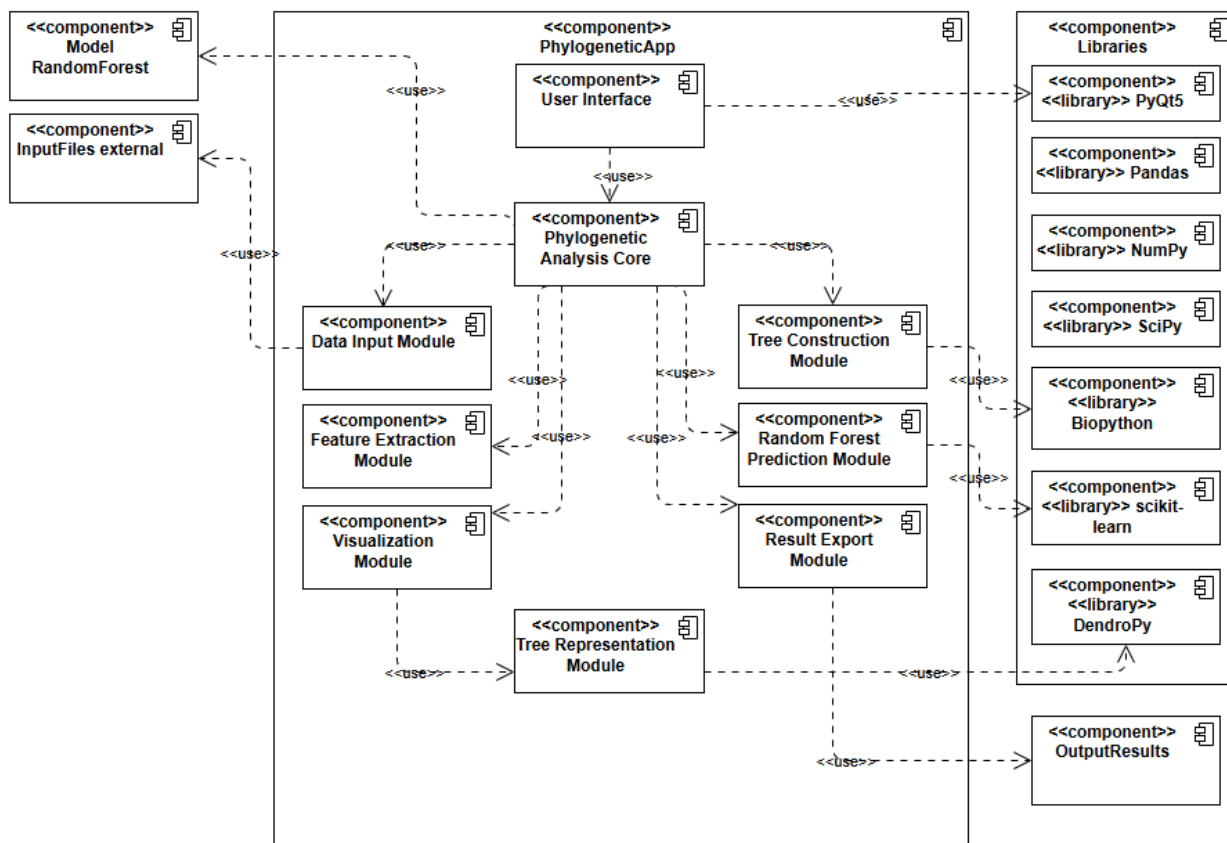


Рисунок 14 – Диаграмма компонентов

2.4.4 Структура диалога с пользователем

Структура диалога с пользователем определяет последовательность взаимодействия пользователя с программным модулем и описывает основные сценарии работы с системой. Диалог реализован в виде графического пользовательского интерфейса, разработанного с использованием библиотеки PyQt5. Взаимодействие пользователя с системой организовано по событийному принципу и включает следующие основные этапы [23]:

- а) Запуск приложения. После запуска программы пользователю отображается главное окно приложения, содержащее элементы управления для загрузки данных, выбора модели и запуска анализа.
- б) Загрузка входных данных. Пользователь выполняет выбор файла множественного выравнивания последовательностей через диалог выбора файла. После загрузки система выполняет проверку корректности данных и уведомляет пользователя о результате операции.
- в) Выбор модели машинного обучения. Пользователь выбирает доступную модель случайного леса из списка. После выбора выполняется загрузка модели и её подготовка к использованию.
- г) Построение филогенетического дерева. При необходимости пользователь инициирует построение дерева на основе загруженных данных. Результат отображается в интерфейсе приложения.
- д) Запуск анализа достоверности. Пользователь запускает процесс анализа, в ходе которого выполняется извлечение признаков и предсказание достоверности узлов дерева.
- е) Просмотр результатов. После завершения анализа пользователю отображается филогенетическое дерево с нанесёнными значениями достоверности, а также дополнительная информация в табличной форме.

2.5 Обеспечение информационной безопасности

2.5.1 Распределение прав доступа и ответственности

В рамках разрабатываемой системы выделяется одна основная роль – пользователь приложения. Пользователь взаимодействует с системой через графический интерфейс и имеет доступ ко всем функциям программы, включая загрузку данных, выбор модели, запуск анализа и сохранение результатов. При этом разграничение ответственности реализуется на логическом уровне и включает следующие аспекты:

- а) пользователь несёт ответственность за корректность загружаемых входных данных;
- б) система выполняет проверку входных данных и предотвращает выполнение анализа при обнаружении ошибок;
- в) операции, изменяющие состояние системы, выполняются только по явному действию пользователя;
- г) результаты анализа формируются только после успешного завершения всех этапов обработки.

2.5.2 Методы защиты информации

В системе реализован ряд методов защиты информации, направленных на обеспечение корректной и безопасной обработки данных.

Проверка входных данных. Одним из ключевых механизмов защиты является валидация входных файлов. Перед началом обработки выполняется:

- а) проверка формата файлов;
- б) контроль наличия необходимых данных;
- в) проверка структуры входных данных на соответствие требованиям системы.

Обработка исключительных ситуаций. Для предотвращения сбоев и потери данных в системе реализована обработка исключений. Все основные операции сопровождаются проверками и обработкой возможных ошибок. Это позволяет избежать аварийного завершения программы и сохранить стабильность её работы.

Защита целостности данных. Целостность данных обеспечивается за счёт:

- а) использования внутренних структур данных, исключающих некорректные изменения;
- б) последовательного выполнения этапов обработки;
- в) запрета на использование неинициализированных или некорректных данных.

Ограничение доступа к ресурсам. Поскольку система функционирует в локальной среде, доступ к данным регулируется средствами операционной

системы. Программа не выполняет операций, требующих повышенных прав доступа, и не изменяет системные файлы. Все операции ограничены пользовательской областью файловой системы.

Безопасность модели машинного обучения. Используемая модель случайного леса загружается из локального хранилища.

2.6 Техническое обеспечение

2.6.1 Архитектура системы и диаграмма развертывания

Разработанная система функционирует на персональном компьютере пользователя, который выступает в роли единственного вычислительного узла. Пользователь взаимодействует с системой через графический интерфейс, реализованный с использованием библиотеки PyQt. В рамках данного приложения выделены следующие основные программные компоненты [22; 23]:

- а) GUI – модуль пользовательского интерфейса, обеспечивающий взаимодействие пользователя с системой;
- б) Parsers – модуль загрузки и обработки входных данных, отвечающий за чтение файлов выравнивания и деревьев;
- в) Tree Builder – модуль построения филогенетического дерева на основе входных данных;
- г) Analysis – модуль анализа, реализующий извлечение признаков и применение модели машинного обучения для оценки достоверности узлов.

2.6.2 Технические характеристики используемых средств

Для функционирования программного модуля требуется персональный компьютер со следующими минимальными характеристиками:

- а) Процессор: не ниже Intel Core i3 или аналогичный;
- б) Оперативная память: не менее 4 ГБ;

- в) Свободное место на диске: не менее 500 МБ для установки программного обеспечения и хранения данных;
- г) Операционная система: Windows, Linux или macOS;
- д) Программное обеспечение: установленный интерпретатор Python версии 3.x и необходимые библиотеки.

Архитектура представлена на рисунке 15.

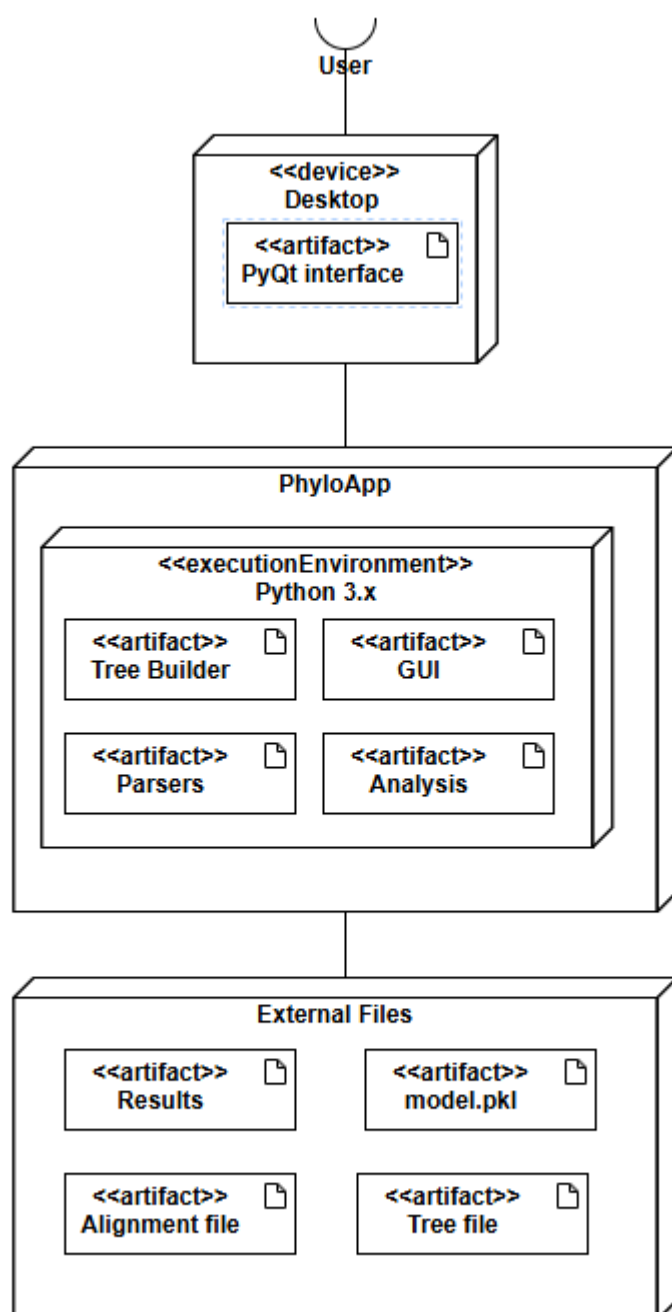


Рисунок 15 – Диаграмма развертывания

РАЗДЕЛ 3. РЕАЛИЗАЦИЯ ПРОЕКТА

3.1 Контрольный пример

Для работы необходимо установить интерпретатор Python 3.10+ и любую удобную среду разработки, например, Visual Studio Code. Откройте папку с исходным кодом проекта в VS Code. Приложение опирается на набор открытых библиотек. Их нужно установить. Выполните в терминале: *pip install biopython dendropy numpy pandas scikit-learn joblib PyQt5 matplotlib scipy*. Далее программу можно запускать при помощи *python main.py*.

При запуске приложения отображается главное окно, содержащее панель навигации по этапам анализа, рабочую область с вкладками, строку состояния и журнал операций. Интерфейс спроектирован таким образом, чтобы направлять пользователя через последовательность шагов: кнопки запуска анализа и предсказания изначально заблокированы до загрузки обязательных входных данных, что предотвращает возникновение исключительных ситуаций. Главный экран представлен на рисунке 16.

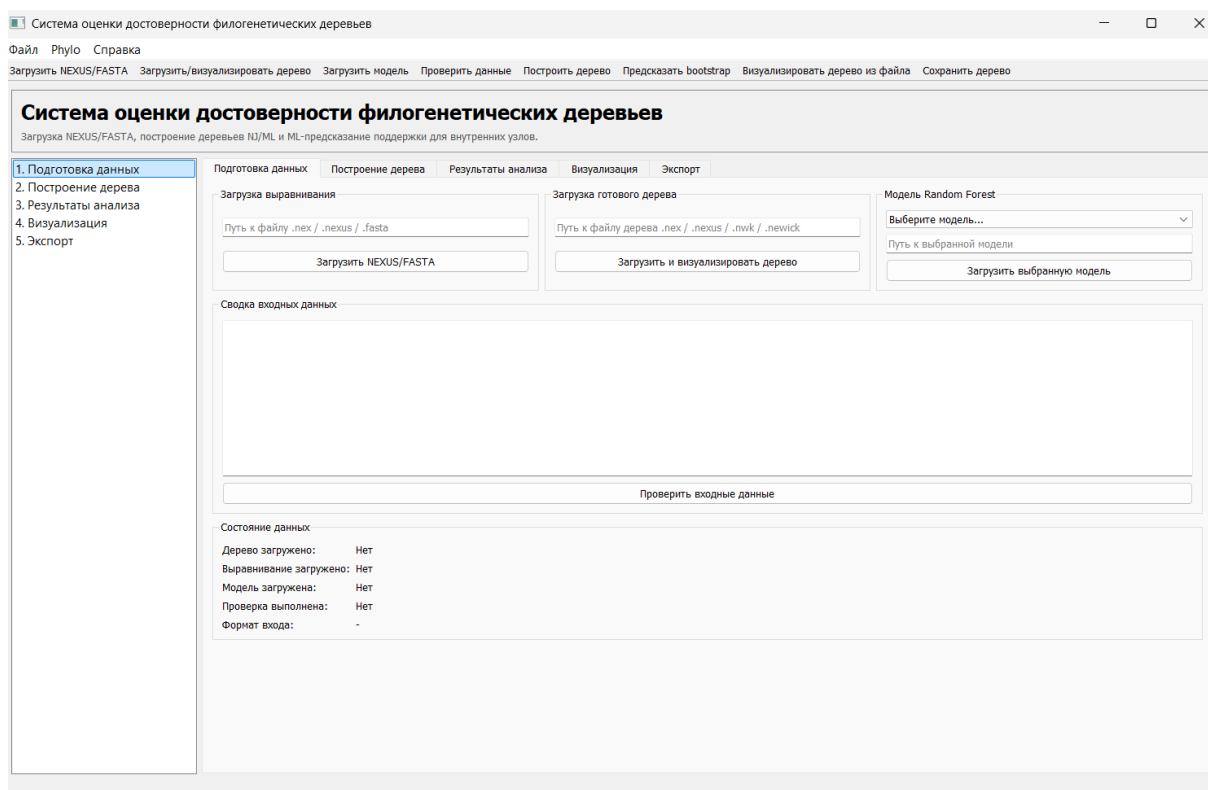


Рисунок 16 – Главный экран приложения

На первом этапе пользователь загружает файл выравнивания и выбирает модель из выпадающего списка предобученных алгоритмов. Успешная загрузка и отсутствие критических ошибок отражаются в статус-баре и журнале, разблокируя доступ к вычислительным функциям. Процесс загрузки модели представлен на рисунке 17.

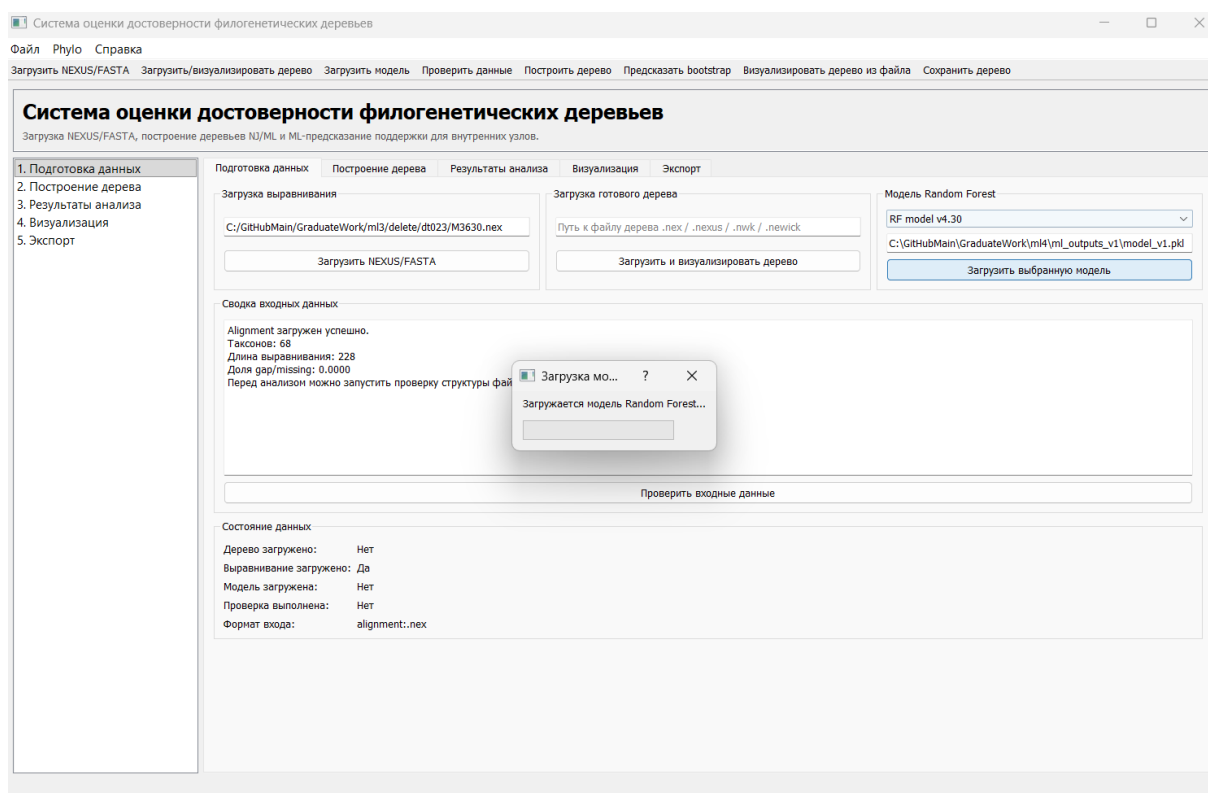


Рисунок 17 – Главный экран в процессе загрузки модели

Далее пользователь переходит на вкладку «Построение дерева», выбирает метод и инициирует вычисление топологии. Результат построения выводится в виде текстовой сводки, содержащей количество таксонов, число внутренних узлов, суммарную длину ветвей и информацию о корне. Окно построения дерева представлено на рисунке 18.

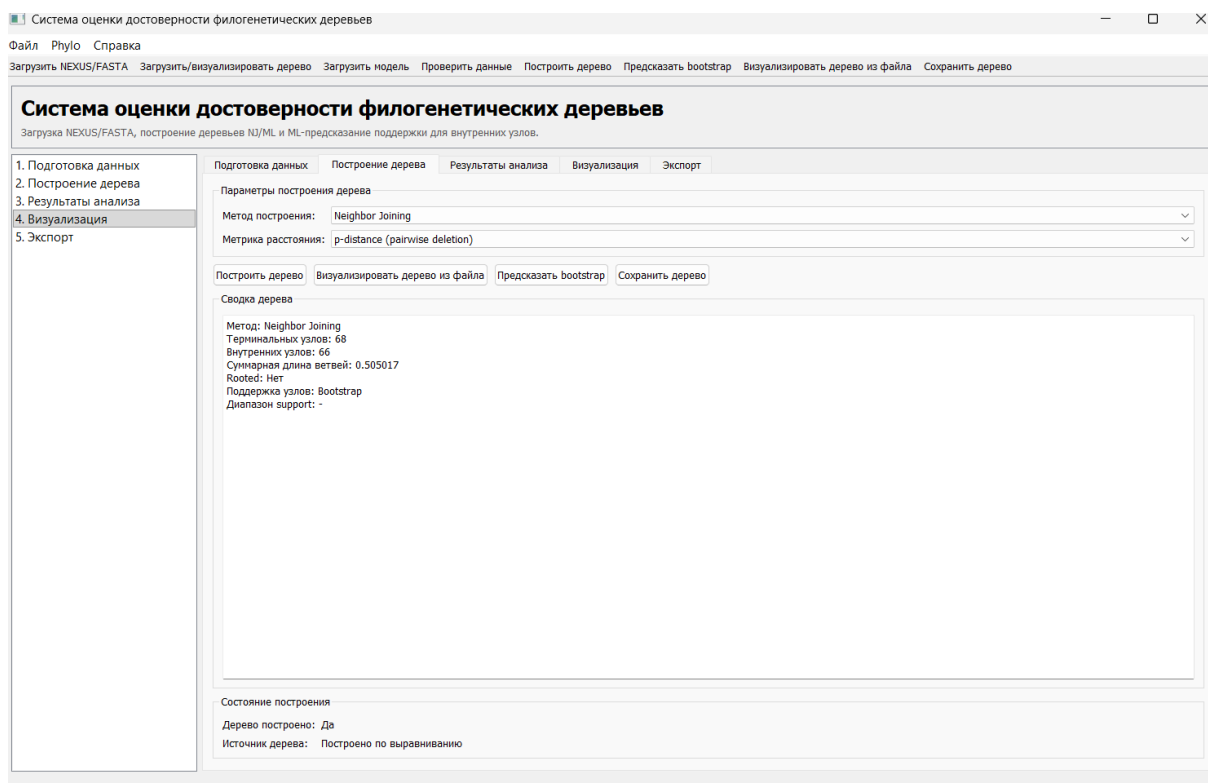


Рисунок 18 – Окно построения дерева

Когда дерево построено, пользователь может запустить ML-предсказание поддержки. Подсистема машинного обучения берёт признаки внутренних узлов и передаёт их в обученную модель случайного леса. Результаты выводятся в интерактивную таблицу: там указаны идентификаторы узлов, топологические параметры, исходная бутстреп-поддержка при её наличии и прогноз модели [6; 16; 24]. Цветовая индикация сделана автоматически: значения ниже 50% помечаются как проблемные, диапазон 50–75% – как средний уровень поддержки, а значения выше 75% – как достаточно надёжные. Окно результатов анализа представлено на рисунке 19.

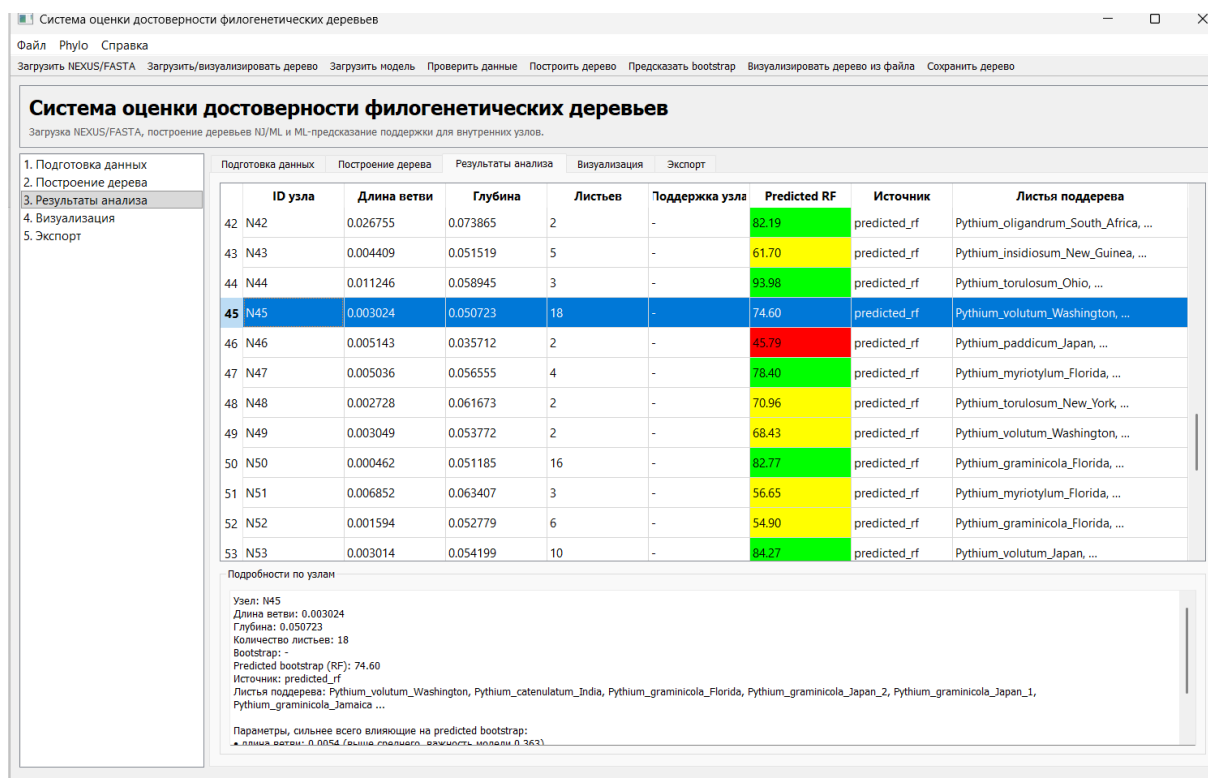


Рисунок 19 – Окно результатов анализа

Для визуального анализа пользователь переходит на вкладку «Визуализация». Дерево отображается на пользовательском холсте Matplotlib с учётом рассчитанных длин ветвей, а внутренние узлы окрашиваются по шкале предсказанной достоверности. При наведении курсора появляется подсказка с пояснением по ключевым признакам, поэтому результат воспринимается не только как цвет на схеме, но и как краткое объяснение прогноза [11; 16].

На завершающем этапе пользователь может сохранить аннотированное дерево во внешнюю среду для публикации или дальнейшего анализа в сторонних биоинформатических пакетах. В журнале экспорта фиксируется временная метка и путь к файлу, что обеспечивает воспроизводимость результатов исследования. Окно визуализации и окно экспорта представлены на рисунке 20 и 21 соответственно.

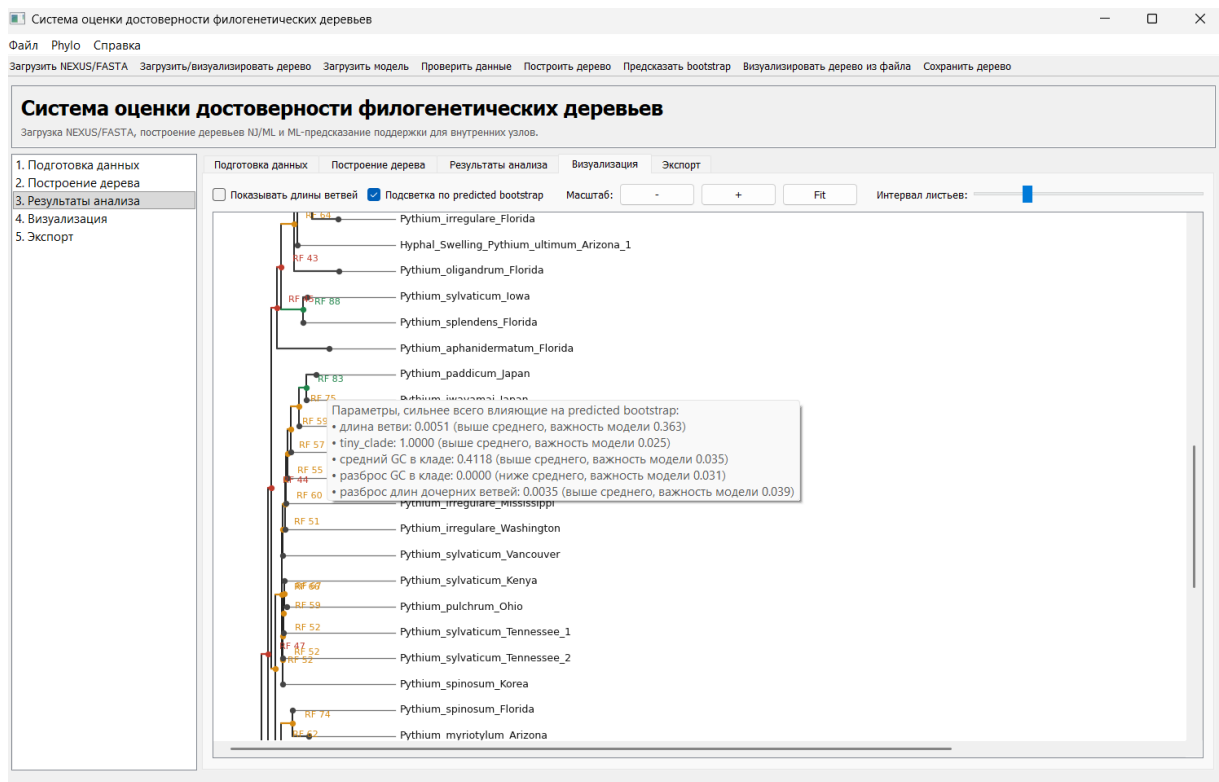


Рисунок 20 – Окно визуализации

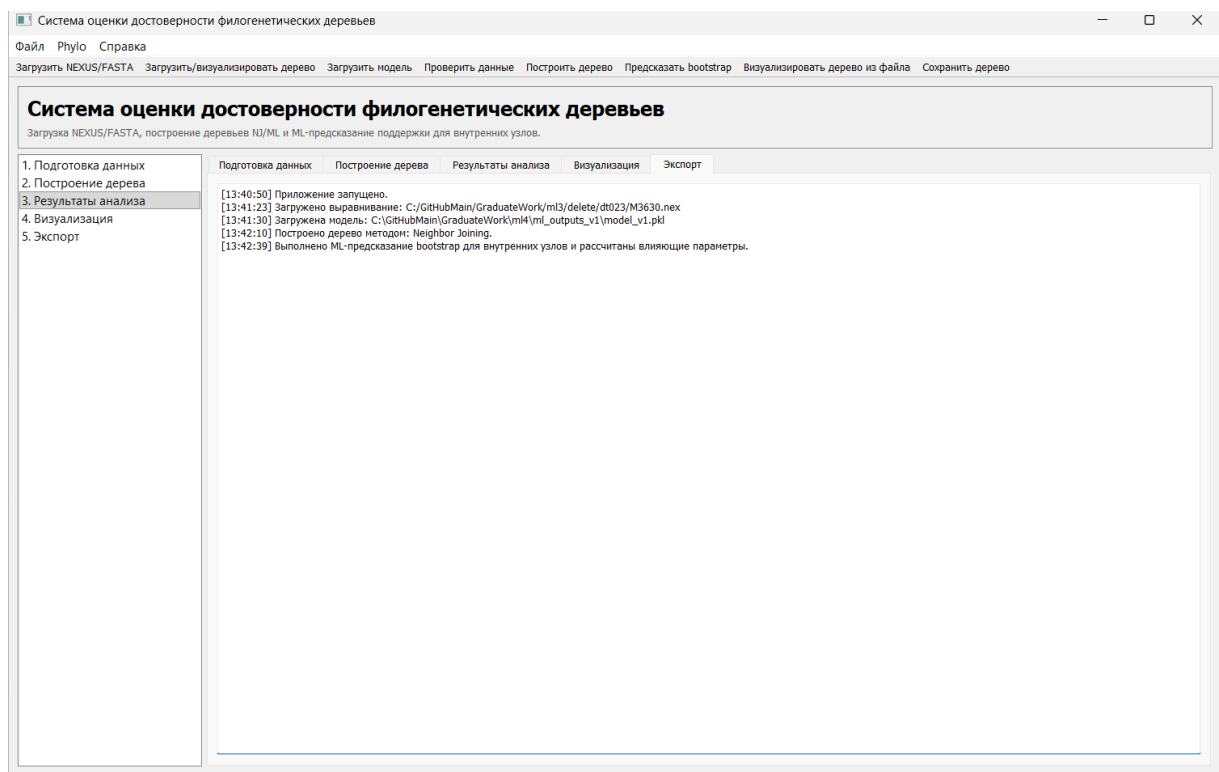


Рисунок 21 – Окно экспорта

3.2 Тестирование программного обеспечения

3.2.1 *Протокол модульного тестирования ядра обработки данных*

Цель тестирования: проверка корректности работы минимальных вычислительных модулей ядра системы, выявление ошибок в логике обработки биологических последовательностей, расчёта эволюционных расстояний и топологических характеристик дерева до передачи данных в конвейер машинного обучения.

Объект тестирования: программные компоненты очистки данных, расчёта попарных расстояний, вычисления баланса поддеревьев.

Метод: автоматизированное тестирование системы как «белый ящик».

Техника: специализированное тестирование, эквивалентное разбиение, анализ граничных значений.

Инструменты: фреймворк `pytest`, язык Python 3.12, виртуальное окружение проекта, библиотеки `BioPython`, `NumPy`, `pytest-parametrize`.

Развёрнутое описание проведения тестирования. Модульное тестирование применяется для изолированной верификации низкоуровневых вычислительных алгоритмов, поскольку ошибки на уровне расчёта долей пропусков, содержания нуклеотидов или эволюционных расстояний напрямую искажают входные признаки модели машинного обучения. В ходе данного этапа проверяется:

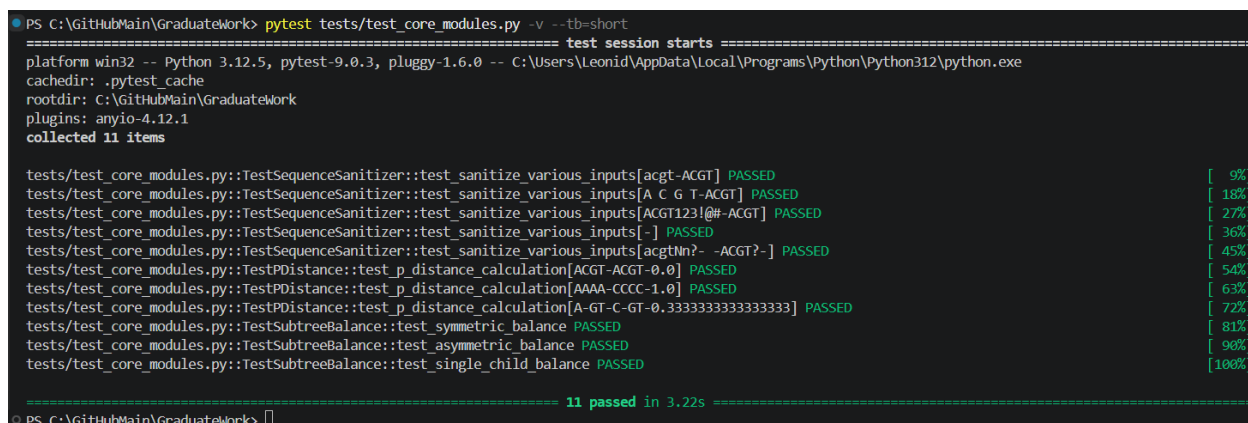
Зачем: гарантировать математическую и биологическую корректность преобразования сырых данных перед подачей в ML-конвейер, исключить накопление погрешностей и предотвратить аварийные падения приложения при встрече с «грязными» биологическими файлами.

Что проверяется: корректность удаления недопустимых символов и приведения к верхнему регистру, точность расчёта доли несовпадений с учётом стратегии игнорирования пропусков, правильность вычисления структурной асимметрии поддеревьев на синтетических топологиях.

Каким образом: тестирование выполняется автоматически через параметризованные сценарии `pytest`. Для каждой функции формируются

эквивалентные классы данных (валидные строки, строки с техническим мусором, пустые последовательности, деревья с разным количеством потомков) и граничные случаи.

Тест-кейсы с подробным описанием находятся в приложении А. Результаты выполнения тестов представлены на рисунке 22. Другие результаты тестов представлены в приложении Г.



```
PS C:\GithubMain\GraduateWork> pytest tests/test_core_modules.py -v --tb=short
===== test session starts =====
platform win32 -- Python 3.12.5, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\Leonid\AppData\Local\Programs\Python\Python312\python.exe
cachedir: .pytest_cache
rootdir: C:\GithubMain\GraduateWork
plugins: anyio-4.12.1
collected 11 items

tests/test_core_modules.py::TestSequenceSanitizer::test_sanitize_various_inputs[acgt-ACGT] PASSED [ 9%]
tests/test_core_modules.py::TestSequenceSanitizer::test_sanitize_various_inputs[A C G T-ACGT] PASSED [ 18%]
tests/test_core_modules.py::TestSequenceSanitizer::test_sanitize_various_inputs[ACGT123!@#-ACGT] PASSED [ 27%]
tests/test_core_modules.py::TestSequenceSanitizer::test_sanitize_various_inputs[-] PASSED [ 36%]
tests/test_core_modules.py::TestSequenceSanitizer::test_sanitize_various_inputs[acgtWn?- -ACGT?-] PASSED [ 45%]
tests/test_core_modules.py::TestPDistance::test_p_distance_calculation[ACGT-ACGT-0.0] PASSED [ 54%]
tests/test_core_modules.py::TestPDistance::test_p_distance_calculation[AAAA-CCCC-1.0] PASSED [ 63%]
tests/test_core_modules.py::TestPDistance::test_p_distance_calculation[A-GT-C-GT-0.3333333333333333] PASSED [ 72%]
tests/test_core_modules.py::TestSubtreeBalance::test_symmetric_balance PASSED [ 81%]
tests/test_core_modules.py::TestSubtreeBalance::test_asymmetric_balance PASSED [ 90%]
tests/test_core_modules.py::TestSubtreeBalance::test_single_child_balance PASSED [100%]

===== 11 passed in 3.22s =====
PS C:\GithubMain\GraduateWork>
```

Рисунок 22 – Результаты выполнения тестов

3.2.2 Протокол функционального тестирования

Цель тестирования: проверка соответствия программного модуля заявленным функциональным требованиям и основным сценариям использования, включая загрузку биологических данных, построение дерева, машинное предсказание достоверности узлов и экспорт результатов.

Объект тестирования: сквозные пользовательские сценарии работы приложения, интерфейс управления состоянием анализа и сервисный слой координации операций.

Метод: ручное тестирование системы как чёрный ящик.

Техника: тестирование на основе формальной спецификации и сценариев использования.

Инструменты: рабочее настольное приложение, эталонные биологические файлы выравнивания и деревьев, сериализованная модель машинного обучения, встроенный журнал операций и системные средства просмотра файлов.

Зачем: убедиться, что последовательность операций, выполняемая исследователем, приводит к корректному результату без потери данных, с правильной блокировкой недоступных функций и соблюдением требований к автономной работе на локальном компьютере.

Что проверяется: корректность загрузки и валидации входных файлов форматов FASTA и NEXUS, зависимость этапов анализа друг от друга, успешность запуска предсказания достоверности узлов, формирование таблицы результатов, визуализация дерева с цветовой подсветкой и сохранение выходных файлов в формате Newick.

Каким образом: тестировщик последовательно выполняет утверждённые сценарии на чистой сессии приложения, фиксирует время отклика, состояние кнопок и полей, корректность заполнения таблиц и формирование файлов. Для каждого шага сравнивается фактическое поведение интерфейса с ожидаемым согласно технической спецификации. Сетевой адаптер тестовой машины временно отключается для проверки требования автономности.

Что используется: набор валидных файлов множественного выравнивания, готовые филогенетические деревья, обученная модель случайного леса, встроенный журнал сообщений приложения и стандартные средства операционной системы для проверки выходных файлов.

Тест-кейсы с подробным описанием находятся в приложении А.

Тестирование выполнялось вручную согласно утверждённой спецификации. Каждый тестовый сценарий запускался на чистой сессии приложения после перезапуска исполняемого файла. Для проверки автономности работы сетевой адаптер тестовой машины был временно отключён.

Результаты выполнения тестов представлены на рисунке 23, а также в приложении Г.

Подготовка данных		Построение дерева	Результаты анализа	Визуализация	Экспорт			
	ID узла	Длина ветви	Глубина	Листьев	Поддержка узла	Predicted RF	Источник	Листья поддерева
11	N11	0.002998	0.022602	12	0.025	78.17	predicted_rf	Bursaphelenchus_abietinus, ...
12	N12	0.059365	0.082827	3	1.000	100.00	predicted_rf	Bursaphelenchus_seani_175_California_U...
13	N13	0.005079	0.041214	10	0.682	47.32	predicted_rf	Bursaphelenchus_mucronatus_163_JP, ...
14	N14	0.046307	0.082443	4	1.000	100.00	predicted_rf	Bursaphelenchus_fraudulentus_147_Helm...
15	N15	0.006286	0.028889	4	0.144	22.81	predicted_rf	Bursaphelenchus_abietinus, ...
16	N16	0.002896	0.025499	8	0.051	37.09	predicted_rf	Bursaphelenchus_sexdentati_179_GR, ...
17	N17	0.001101	0.083929	2	0.516	55.05	predicted_rf	Bursaphelenchus_seani_174_California_U...
18	N18	0.014526	0.055740	6	0.941	87.84	predicted_rf	Bursaphelenchus_mucronatus_163_JP, ...
19	N19	0.048648	0.089862	4	1.000	100.00	predicted_rf	Bursaphelenchus_xylophilus_185_CAN, ...
20	N20	0.005753	0.088196	2	0.952	86.35	predicted_rf	Bursaphelenchus_fraudulentus_147_Helm...
21	N21	0.005594	0.088037	2	0.998	98.97	predicted_rf	Bursaphelenchus_fraudulentus_150_Krasn...
22	N22	0.008346	0.037234	3	0.439	34.34	predicted_rf	Bursaphelenchus_abietinus, ...

Подробности по узлам

Узел: N25
Длина ветви: 0.019535
Глубина: 0.075275
Количество листьев: 3
Bootstrap / support: 1.000
Predicted bootstrap (RF): 99.14
Источник: predicted_rf
Листья поддерева: Bursaphelenchus_mucronatus_163_JP, Bursaphelenchus_mucronatus_168_Zusmarshausen_DE, Bursaphelenchus_mucronatus_167_Gruenberg_DE

Параметры, сильнее всего влияющие на predicted bootstrap:
• относительная длина ветви: 0.0519 (выше среднего, важность модели 0.335)
• длина ветви: 0.0195 (выше среднего, важность модели 0.206)

Рисунок 23 – Таблица предсказаний

3.2.3 Протокол тестирования пользовательского интерфейса

Цель тестирования: проверка корректности отображения филогенетического дерева, интерактивности элементов управления и соответствия графического интерфейса эргономическим требованиям.

Объект тестирования: графический интерфейс приложения, кастомный холст отрисовки деревьев, таблицы результатов и элементы навигации.

Метод: ручное тестирование системы как серый ящик.

Инструменты: рабочее настольное приложение, эталонные наборы биологических данных разного объёма, встроенные средства масштабирования и навигации, системные инструменты захвата экрана и мониторинга отзывчивости.

Зачем: убедиться, что визуализация корректно передаёт биологическую информацию, не искажает топологию дерева, а интерактивные элементы работают предсказуемо, обеспечивая исследователю удобный и быстрый анализ результатов без риска визуальных артефактов.

Что проверяется: масштабирование и перемещение графического холста, цветовая подсветка узлов по уровню предсказанной поддержки, отображение всплывающих подсказок с объяснением влияния признаков, корректность табличного представления результатов, адаптивность интерфейса при различных размерах окна и количестве таксонов.

Каким образом: тестировщик последовательно выполняет утверждённые сценарии взаимодействия с интерфейсом, фиксирует поведение элементов управления при изменении параметров отображения, проверяет читаемость подписей при большом количестве листьев, корректность работы всплывающих окон и соответствие цветовой схемы установленным пороговым значениям. Каждый этап сопровождается замером времени отклика и фиксацией визуальных состояний.

Что используется: наборы данных малого, среднего и большого объёма, встроенные ползунки и кнопки управления масштабом, средства мониторинга отзывчивости интерфейса, встроенный журнал сообщений приложения, эталонные скриншоты для сравнения.

Тест-кейсы с подробным описанием находятся в приложении А. Результаты выполнения тестов представлены на рисунке 24 и в приложении Г.



Рисунок 24 – Цвет узлов и описание

3.2.4 Протокол тестирования модели машинного обучения

Цель тестирования: Проверка качества предсказаний модели, соответствия метрик заявленным требованиям – минимальный $R^2 \geq 0.70$, корректности интерпретации признаков, устойчивости к пропущенным данным и времени выполнения предсказания на реальных биологических выборках.

Объект тестирования: Обученные модели машинного обучения

Метод: Полуавтоматизированное тестирование с использованием скриптов оценки и ручного анализа отчётов

Техника: Тестирование на выборке данных, кросс-валидация по группам для оценки обобщающей способности, сравнительный анализ метрик между версиями моделей

Инструменты: Библиотеки, обучающая выборка – узлы филогенетических деревьев с известными значениями bootstrap, обученные модели и файлы метаданных

Зачем: убедиться, что последовательное совершенствование признакового пространства и гиперпараметров привело к статистически значимому росту точности, а модель устойчива к шумным биологическим данным и соответствует требованиям интерактивной работы.

Что проверяется: корректность загрузки модели и карты признаков, соответствие метрик R^2 , MAE, RMSE пороговым значениям, калибровка предсказаний в диапазон $[0; 100]$, время выполнения предсказания на деревьях разного размера, устойчивость к пропущенным значениям и экстремумам.

Каким образом: тестирующий последовательно запускает оценку на кросс-валидации, фиксирует метрики каждой версии, сравнивает время работы с классическим бутстрепом, проводит стресс-тесты с искусственными пропусками и выбросами.

Тест-кейсы с подробным описанием находятся в приложении А. Результаты выполнения тестов представлены в приложении Г.

3.2.5 Протокол тестирования надёжности и обработки исключительных ситуаций

Цели: поиск дефектов, не относящихся к основному поведению системы, проверка устойчивости к некорректным данным, пользовательским ошибкам и пограничным условиям.

Метод: ручное тестирование с элементами анализа логики работы как белый ящик.

Техника: предположение ошибок, анализ граничных значений и негативные сценарии.

Инструменты: рабочее настольное приложение, набор искусственно повреждённых файлов, документы с несовпадающими таксонами, пустые структуры и данные с отсутствующими значениями, встроенный журнал операций.

Зачем: гарантировать, что приложение не завершится аварийно при загрузке нестандартных или повреждённых биологических файлов, а пользователь получит понятные инструкции по устранению проблемы без потери промежуточных результатов.

Что проверяется: реакция системы на битые форматы NEXUS/FASTA, обработку несовпадения таксонов между выравниванием и деревом, работу с минимальными деревьями, корректность блокировки кнопок и отображения диалоговых окон, сохранение рабочего состояния после исключения.

Каким образом: тестировщик последовательно подаёт на вход специально подготовленные негативные сценарии, фиксирует появление модальных окон с ошибками, проверяет логи приложения и убеждается в отсутствии крашей или невалидных состояний интерфейса.

Что используется: повреждённые файлы выравнивания, деревья с изменёнными именами таксонов, пустые директории, отладочный журнал приложения, системные мониторы процессов.

Тест-кейсы с подробным описанием находятся в приложении А. Рисунки с результатами находятся в приложении Г.

РАЗДЕЛ 4. ОЦЕНКА ЭФФЕКТИВНОСТИ ПРОЕКТА

4.1. Анализ качества проектного решения

Оценка качества разработанного программного модуля проводилась на основе его соответствия заявленным функциональным и нефункциональным требованиям, а также по качеству интегрированной подсистемы машинного обучения. По итогам комплексного тестирования из раздела 3 можно выделить несколько важных характеристик качества системы:

- а) функциональная пригодность: приложение закрывает основной сценарий работы – от загрузки исходных биологических файлов до получения аннотированного дерева и экспорта результата [7; 18; 20; 21];
- б) надёжность и отказоустойчивость: в модуле предусмотрены проверки входных данных, обработка ошибок и защита от некорректных действий пользователя, поэтому приложение не должно завершаться аварийно при типовых сбоях [22];
- в) удобство использования: графический интерфейс с подсветкой узлов, таблицей результатов и всплывающими пояснениями помогает быстрее разобраться в итоговой оценке, особенно при работе с несколькими деревьями [11; 23];

Отдельно оценивалась точность модели случайного леса, потому что именно от качества прогноза зависит практическая ценность полученных оценок поддержки [6; 16; 24]. В ходе работы была проведена последовательная итеративная доработка модели случайного леса, что позволило существенно повысить качество предсказаний по сравнению с базовой версией.

Сравнение моделей приведено в таблице 3.

Таблица 3 – Сравнительная таблица моделей

Версия	Датасеты	Признаков	MAE	RMSE	R ²
V1	18	17	12.92	17.21	0.678
V2	33	17	20.84	27.63	0.392
V3	39	21	16.45	22.61	0.523
V4	30	21	10.68	13.89	0.807
V5	30	21	7.96	10.41	0.869

Базовая версия модели v1 опиралась на 17 первичных статистик, описывающих топологию дерева и глобальные/локальные параметры выравнивания. В версии v5 к этому набору добавлены 4 производных признаков, рассчитываемых на этапе предобработки. Их введение позволило алгоритму выявлять нелинейные зависимости и биологические артефакты, которые не фиксировались линейными метриками. Разработанные признаки отмечены в таблице 4.

Таблица 4 – Дополнительные признаки для улучшения

Признак	Формула расчёта	Биологическая интерпретация
bl_cv	$\text{std_child_branch_length} / (\text{mean_child_branch_length} + 1e-6)$	Коэффициент вариации длин дочерних ветвей. Высокие значения указывают на гетерогенность скоростей эволюции, что коррелирует с нестабильностью топологии.
gap_var_clade	$\text{gap_var_clade} = f_gap_clade - f_gap_global $	Симметричная реакция модели на избыток или дефицит пропусков в локальной области. Указывает на локальное ухудшение качества выравнивания.

Признак	Формула расчёта	Биологическая интерпретация
tiny_clade	1, если $n_leaves_subtree \leq 3$ иначе 0	Флаг малых клад. Небольшие подгруппы содержат ограниченный статистический материал, из-за чего их поддержка менее устойчива к шуму.
deep_node	$depth / (max_depth + 1e-6)$	Нормализованная глубина узла. Узлы ближе к корню аккумулируют больше мутаций и демонстрируют более высокую статистическую поддержку.

4.2 Расчёт экономических результатов от внедрения

Основной экономический эффект от внедрения разработанного программного модуля в научно-исследовательскую деятельность заключается в сокращении трудозатрат исследователей-биоинформатиков за счёт ускорения процесса оценки достоверности филогенетических деревьев по времени. Замер времени выполнения операций производился с помощью встроенных средств профилирования Python для разработанного модуля и секундомера для классического бутстреп-анализа в программном комплексе MEGA. Для чистоты эксперимента каждый замер усреднялся по 5 повторениям [12]. Тестирование времени представлено в таблице 5.

Таблица 5 – Тестирование времени

Количество узлов	Классический бутстреп (MEGA), с	Предсказание моделью v4, с	Ускорение, в n раз
14	4.11	0.350	11.7
18	4.56	0.846	5.4

Количество узлов	Классический бутстреп (MEGA), с	Предсказание моделью v4, с	Ускорение, в п раз
34	4.94	1.514	3.3
67	6.74	3.952	1.7

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы была успешно решена комплексная научно-практическая задача по проектированию, реализации и всесторонней верификации настольного программного модуля для оценки достоверности филогенетических деревьев с применением методов машинного обучения. Поставленная цель работы, заключающаяся в повышении скорости оценки достоверности филогенетических деревьев путём создания программного инструмента на базе алгоритма случайного леса для многомерного анализа молекулярных последовательностей и топологических параметров дерева, достигнута в полном объёме. Разработанный продукт представляет собой законченное технологическое решение, сочетающее в себе современные подходы биоинформатики, методы интеллектуального анализа данных и эргономичный пользовательский интерфейс, готовое к интеграции в реальную научно-исследовательскую практику.

Для достижения заявленной цели последовательно были решены все сформулированные во введении задачи. Был проведён глубокий сравнительный анализ предметной области, в рамках которого изучены классические статистические методы оценки надёжности ветвей, в частности традиционный бутстреп-анализ. Был выполнен сбор набора данных, включающего множественные выравнивания и филогенетические деревья с известными значениями статистической поддержки. На базе алгоритма случайного леса был реализован конвейер предсказания, включающий этапы нормализации признаков и обработки пропусков. В процессе итеративной доработки была проделана значительная работа по повышению качества модели: базовый набор из семнадцати первичных статистик был расширен четырьмя специализированными производными признаками, отражающими биологические артефакты, такие как гетерогенность скоростей эволюции, локальное ухудшение качества выравнивания, малый размер клад и нормализованная глубина узла. Параллельно была проведена целенаправленная

регуляризация гиперпараметров ансамбля деревьев решений для предотвращения переобучения на шумовых биологических данных. Результатом этой работы стала финальная версия модели, продемонстрировавшая коэффициент детерминации R^2 свыше 0,80 и среднюю абсолютную ошибку MAE менее 11, что подтверждает высокую статистическую устойчивость и предсказательную силу алгоритма [6; 8; 16].

Таким образом, результаты выпускной квалификационной работы вносят ощутимый вклад в развитие прикладной биоинформатики и демонстрируют высокую эффективность применения методов машинного обучения для решения классических статистических задач. Разработанный программный продукт открывает перспективы для ускорения научных открытий в области эволюционной генетики и молекулярной биологии, обеспечивая исследователей быстрым, точным и интерпретируемым инструментом для анализа эволюционных связей.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 2.105–2019. Единая система конструкторской документации. Общие требования к текстовым документам = Unified system for design documentation. General requirements for textual documents : национальный стандарт Российской Федерации : издание официальное. – Москва : Стандартинформ, 2021. – 44 с. – Текст : непосредственный.
2. ГОСТ 7.32–2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления = System of standards on information, librarianship and publishing. The research report. Structure and rules of presentation : межгосударственный стандарт : издание официальное. – Москва : Стандартинформ, 2018. – 35 с. – Текст : непосредственный.
3. ГОСТ Р 7.0.97–2016. Система стандартов по информации, библиотечному и издательскому делу. Организационно-распорядительная документация. Требования к оформлению документов = System of standards on information, librarianship and publishing. Organizational and administrative documentation. Requirements for presentation of records : национальный стандарт Российской Федерации : издание официальное. – Москва : Стандартинформ, 2019. – 32 с. – Текст : непосредственный.
4. ГОСТ Р 7.0.100–2018. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Библиографическое описание. Общие требования и правила составления = System of standards on information, librarianship and publishing. Bibliographic record. Bibliographic description. General requirements and rules : национальный стандарт Российской Федерации : издание официальное. – Москва : Стандартинформ, 2018. – 124 с. – Текст : непосредственный.
5. Стефанов В. Е. Биоинформатика : учебник для вузов / В. Е. Стефанов, А. А. Тулуб, Г. Р. Мавропуло-Столяренко. – Москва : Издательство Юрайт, 2022. – 252 с. – ISBN 978-5-534-00860-9. – Текст : электронный.

6. Breiman L. Random forests / L. Breiman. – DOI 10.1023/A:1010933404324. – Текст : непосредственный // Machine Learning. – 2001. – Vol. 45, no. 1. – P. 5–32.
7. Cock P. J. A. Biopython: freely available Python tools for computational molecular biology and bioinformatics / P. J. A. Cock, T. Antao, J. T. Chang [et al.]. – DOI 10.1093/bioinformatics/btp163. – Текст : электронный // Bioinformatics. – 2009. – Vol. 25, no. 11. – P. 1422–1423.
8. Felsenstein J. Confidence limits on phylogenies: an approach using the bootstrap / J. Felsenstein. – DOI 10.1111/j.1558-5646.1985.tb00420.x. – Текст : непосредственный // Evolution. – 1985. – Vol. 39, no. 4. – P. 783–791.
9. Guindon S. New algorithms and methods to estimate maximum-likelihood phylogenies: assessing the performance of PhyML 3.0 / S. Guindon, J.-F. Dufayard, V. Lefort [et al.]. – DOI 10.1093/sysbio/syq010. – Текст : непосредственный // Systematic Biology. – 2010. – Vol. 59, no. 3. – P. 307–321.
10. Harris C. R. Array programming with NumPy / C. R. Harris, K. J. Millman, S. J. van der Walt [et al.]. – DOI 10.1038/s41586-020-2649-2. – Текст : электронный // Nature. – 2020. – Vol. 585. – P. 357–362.
11. Hunter J. D. Matplotlib: a 2D graphics environment / J. D. Hunter. – DOI 10.1109/MCSE.2007.55. – Текст : непосредственный // Computing in Science & Engineering. – 2007. – Vol. 9, no. 3. – P. 90–95.
12. Kumar S. MEGA X: Molecular Evolutionary Genetics Analysis across computing platforms / S. Kumar, G. Stecher, M. Li, C. Knyaz, K. Tamura. – DOI 10.1093/molbev/msy096. – Текст : электронный // Molecular Biology and Evolution. – 2018. – Vol. 35, no. 6. – P. 1547–1549.
13. McKinney W. Data structures for statistical computing in Python / W. McKinney. – DOI 10.25080/Majora-92bf1922-00a. – Текст : электронный // Proceedings of the 9th Python in Science Conference. – 2010. – P. 56–61.

14. Minh B. Q. IQ-TREE 2: new models and efficient methods for phylogenetic inference in the genomic era / B. Q. Minh, H. A. Schmidt, O. Chernomor [et al.]. – DOI 10.1093/molbev/msaa015. – Текст : электронный // Molecular Biology and Evolution. – 2020. – Vol. 37, no. 5. – P. 1530–1534.
15. Okonechnikov K. Unipro UGENE: a unified bioinformatics toolkit / K. Okonechnikov, O. Golosova, M. Fursov, the UGENE team. – DOI 10.1093/bioinformatics/bts091. – Текст : электронный // Bioinformatics. – 2012. – Vol. 28, no. 8. – P. 1166–1167.
16. Pedregosa F. Scikit-learn: machine learning in Python / F. Pedregosa, G. Varoquaux, A. Gramfort [et al.]. – Текст : электронный // Journal of Machine Learning Research. – 2011. – Vol. 12. – P. 2825–2830.
17. Stamatakis A. RAxML version 8: a tool for phylogenetic analysis and post-analysis of large phylogenies / A. Stamatakis. – DOI 10.1093/bioinformatics/btu033. – Текст : электронный // Bioinformatics. – 2014. – Vol. 30, no. 9. – P. 1312–1313.
18. Sukumaran J. DendroPy: a Python library for phylogenetic computing / J. Sukumaran, M. T. Holder. – DOI 10.1093/bioinformatics/btq228. – Текст : электронный // Bioinformatics. – 2010. – Vol. 26, no. 12. – P. 1569–1571.
19. Virtanen P. SciPy 1.0: fundamental algorithms for scientific computing in Python / P. Virtanen, R. Gommers, T. E. Oliphant [et al.]. – DOI 10.1038/s41592-019-0686-2. – Текст : электронный // Nature Methods. – 2020. – Vol. 17, no. 3. – P. 261–272.
20. Biopython Tutorial and Cookbook : документация [Электронный ресурс]. – URL: <https://biopython.org/docs/latest/Tutorial/> (дата обращения: 31.05.2026). – Текст : электронный.
21. DendroPy Phylogenetic Computing Library : документация [Электронный ресурс]. – URL: <https://dendropy.org/> (дата обращения: 31.05.2026). – Текст : электронный.

22. Python 3.14.5 documentation : официальная документация [Электронный ресурс]. – URL: <https://docs.python.org/3/> (дата обращения: 31.05.2026). – Текст : электронный.
23. PyQt5 Reference Guide : документация [Электронный ресурс]. – URL: <https://www.riverbankcomputing.com/static/Docs/PyQt5/> (дата обращения: 31.05.2026). – Текст : электронный.
24. RandomForestRegressor : документация scikit-learn [Электронный ресурс]. – URL: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html> (дата обращения: 31.05.2026). – Текст : электронный.

ПРИЛОЖЕНИЕ А. ТЕСТ-КЕЙСЫ

Таблица 6 – Тест-кейс пункта 3.2.1 - №1

Функция	Очистка и нормализация нуклеотидных последовательностей
Действие	Проверка удаления недопустимых символов, приведения к верхнему регистру, обработки пустых строк и граничных символов
Ожидаемый результат	На выходе содержится только последовательность из допустимых символов (А, С, G, Т, -, ?), приведённая к верхнему регистру; все пробелы, цифры, спецсимволы и недопустимые буквы удалены
Предусловие	Тестовое окружение pytest инициализировано, модуль обработки последовательностей доступен
Шаги теста	Запустить параметризованный тест с набором входных строк: строчные буквы, строка с пробелами, строка с цифрами и знаками пунктуации, пустая строка, смешанная строка с пропусками и невалидными символами. Вызвать функцию очистки для каждого набора. Сравнить фактический результат с заранее рассчитанным эталонным значением.
Постусловие	Все ассерты выполнены успешно, временные объекты не создавались
Результат	Тест пройден
Примечания	Используется техника эквивалентного разбиения. Проверка покрывает 5 граничных классов входных данных

Таблица 7 – Тест-кейс пункта 3.2.1 - №2

Функция	Расчёт эволюционного расстояния между двумя последовательностями
Действие	Проверка корректности вычисления доли несовпадений при полном совпадении, полном различии и наличии пропусков

Ожидаемый результат	Для идентичных строк возвращается 0.0, для полностью различных без пропусков возвращается 1.0, позиции с пропусками игнорируются, результат для строк с пропусками равен одной трети
Предусловие	Модуль расчёта расстояний импортирован, тестовые данные подготовлены
Шаги теста	Передать в функцию две идентичные последовательности, проверить равенство результата нулю. Передать две последовательности без совпадающих позиций, проверить равенство результата единице. Передать последовательности с пропусками, проверить, что позиции с символами '-' исключаются из расчёта знаменателя, и результат соответствует формуле $1/3 \approx 0.3333$.
Постусловие	Все ассерты выполнены успешно, погрешность сравнения не превышает $1e-6$
Результат	Тест пройден
Примечания	Проверка логики и устойчивости к нулевому знаменателю

Таблица 8 – Тест-кейс пункта 3.2.1 - №3

Функция	Вычисление структурного баланса поддерева филогенетического дерева
Действие	Проверка расчёта асимметрии для симметричных, асимметричных ветвей и узлов с единственным потомком
Ожидаемый результат	Для симметричного поддерева с равным количеством листьев возвращается 0.0, для асимметричного возвращается значение, пропорциональное разнице размеров поддеревьев, для единственного потомка возвращается 0.0
Предусловие	Созданы тестовые объекты с заданной топологией и количеством листьев
Шаги теста	Сформировать узел с двумя дочерними ветвями, содержащими по 4 листа, проверить возврат нулевого баланса. Сформировать узел с ветвями размером 3 и 1 лист, проверить корректность расчёта по формуле нормализованной разницы

Постусловие	Все ассерты выполнены успешно, объекты дерева освобождены сборщиком мусора
Результат	Тест пройден
Примечания	Используется техника граничных значений и проверки логики ветвления

Таблица 9 – Тест-кейс пункта 3.2.2 - №1

Функция	Сквозной сценарий анализа биологических данных
Действие	Выполнение полного цикла работы приложения от загрузки выравнивания до получения предсказанных значений достоверности
Ожидаемый результат	Пользователь последовательно загружает выравнивание, строит дерево, применяет модель машинного обучения, рассчитывает значения предсказанной поддержки для внутренних узлов и отображает их в таблице и на графическом холсте. Все этапы завершаются без ошибок, статус-бар информирует о завершении операций.
Предусловие	Приложение запущено. В наличии корректный файл выравнивания, выбран метод построения дерева, доступна сохранённая модель. Сетевое подключение отключено.
Шаги теста	Загрузить файл выравнивания через диалог выбора. Выбрать метод построения дерева и запустить расчёт Выбрать модель из списка и нажать кнопку загрузки. Запустить предсказание достоверности узлов. Дождаться завершения расчёта и проверить появление значений в колонке предсказанной поддержки.
Постусловие	Внутреннее состояние приложения переведено в режим завершённого анализа. Таблица результатов заполнена, холст обновлён.
Результат	Тест пройден
Примечания	Покрывает требования ФТ-1, ФТ-2, ФТ-5 и требование автономной работы

Таблица 10 – Тест-кейс пункта 3.2.2 - №2

Функция	Проверка зависимостей состояний и блокировки недоступных операций
Действие	Попытка запуска предсказания или построения дерева без загрузки обязательных данных
Ожидаемый результат	Кнопки запуска анализа заблокированы или при нажатии выводится информационное сообщение с указанием недостающего этапа. Система не переходит в невалидное состояние, промежуточные данные не теряются.
Предусловие	Приложение запущено. Данные не загружены, модель не выбрана.
Шаги теста	Нажать кнопку запуска предсказания без загрузки выравнивания. Попробовать построить дерево, не указав метод или файл. Выбрать модель без загрузки данных и нажать кнопку предсказания.
Постусловие	Интерфейс остаётся в исходном состоянии. В журнале операций появляются записи с описанием ошибки.
Результат	Тест пройден
Примечания	Покрывает требования ФТ-1, ФТ-2 и требование отказоустойчивости интерфейса

Таблица 11 – Тест-кейс пункта 3.2.2 - №3

Функция	Экспорт и сохранение результатов анализа
Действие	Сохранение построенного дерева и таблицы предсказаний в локальную файловую систему
Ожидаемый результат	Файл дерева сохраняется в формате Newick по указанному пути. В журнале операций фиксируется временная метка и путь сохранения. Сторонние инструменты биологического анализа успешно открывают сохранённый файл.
Предусловие	Анализ завершён успешно, данные предсказания загружены в интерфейс приложения.

Шаги теста	Перейти на вкладку экспорта результатов. Выбрать команду сохранения дерева. Указать путь и имя выходного файла. Подтвердить сохранение и проверить наличие файла в целевой директории.
Постусловие	Файл создан, доступен для открытия в независимых программах просмотра филогенетических деревьев.
Результат	Тест пройден
Примечания	Покрывает требование ФТ-6 и требование локальной обработки данных

Таблица 12 – Тест-кейс пункта 3.2.3 - №1

Функция	Цветовая подсветка узлов по уровню предсказанной достоверности
Действие	Проверка корректности применения цветовой схемы к внутренним узлам дерева после запуска предсказания
Ожидаемый результат	Узлы со значением менее пятидесяти процентов окрашены в красный цвет, от пятидесяти до семидесяти пяти процентов в жёлтый, свыше семидесяти пяти процентов в зелёный. Цвета соответствуют ячейкам в таблице результатов.
Предусловие	Приложение запущено. Загружены выравнивание, дерево и модель. Предсказание выполнено успешно.
Шаги теста	Перейти на вкладку визуализации. Включить опцию подсветки по предсказанной достоверности. Сравнить цвет каждого внутреннего узла с его значением в таблице результатов. Убедиться в отсутствии узлов с некорректной окраской или отсутствием цвета.
Постусловие	Визуальная схема соответствует пороговым значениям. Цветовое кодирование синхронизировано с табличными данными.
Результат	Тест пройден

Примечания	Покрывает требование ФТ-5 и нефункциональное требование к эргономике визуального восприятия
------------	---

Таблица 13 – Тест-кейс пункта 3.2.3 - №2

Функция	Масштабирование, панорамирование и всплывающие подсказки
Действие	Проверка интерактивности холста, корректности навигации и отображения пояснений о влияющих признаках
Ожидаемый результат	Холст масштабируется без искажения пропорций, перетаскивание работает плавно, при наведении курсора на узел появляется всплывающая подсказка с текстовым описанием топ-5 параметров, влияющих на прогноз.
Предусловие	Приложение запущено. Дерево загружено и визуализировано. Предсказание выполнено.
Шаги теста	Зажать левую кнопку мыши и переместить дерево в разные области холста. Навести курсор на внутренний узел и удерживать курсор неподвижно. Проверить появление и читаемость всплывающей подсказки с перечнем признаков и их вкладом.
Постусловие	Навигация работает корректно, подсказки отображаются без задержек, текст полностью читаем.
Результат	Тест пройден
Примечания	Покрывает требование ФТ-5 и нефункциональное требование к удобству использования интерфейса

Таблица 14 – Тест-кейс пункта 3.2.4 - №1

Функция	Загрузка модели и валидация метаданных
Действие	Проверка корректной загрузки сериализованной модели и файла метаданных
Ожидаемый результат	Модель загружается без ошибок; ключ со списком признаков в метаданных содержит ожидаемое количество признаков для данной версии

Предусловие	В рабочей директории присутствуют файлы обученной модели и соответствующий файл метаданных
Шаги теста	Инициализировать компонент предсказания. Вызвать метод загрузки с путями к модели и метаданным.
Постусловие	Модель готова к предсказанию
Результат	Тест пройден
Примечания	Покрывает требования ФТ-4 и нефункциональное требование к надёжности

Таблица 15 – Тест-кейс пункта 3.2.4 - №2

Функция	Предсказание на эталонных данных и проверка метрик
Действие	Запуск предсказания на отложенной выборке и расчёт метрик качества
Ожидаемый результат	Значения предсказаний находятся в диапазоне [0; 100] $R^2 \geq 0.70$ $MAE \leq 15.0$
Предусловие	Загружена модель, подготовлен тестовый набор данных с признаками и целевой переменной
Шаги теста	Сформировать матрицу признаков и вектор целевых значений из тестовой выборки. Выполнить предсказание. Рассчитать метрики качества. Сравнить с пороговыми значениями.
Постусловие	Метрики зафиксированы в отчёте; при отклонении от порога сгенерировано предупреждение
Результат	Тест пройден
Примечания	Покрывает требования ФТ-4, нефункциональное требование к надёжности и производительности

Таблица 15 – Тест-кейс пункта 3.2.5 - №1

Функция	Сравнительный анализ версий моделей
Действие	Запуск предсказания одной и той же тестовой выборки на моделях разных версий и сравнение метрик

Ожидаемый результат	Таблица сравнения показывает тенденцию улучшения/стабилизации метрик; финальная версия демонстрирует наилучший R^2 за счёт расширенного набора признаков
Предусловие	Доступны все версии моделей и единая тестовая выборка
Шаги теста	Для каждой модели: загрузить, выполнить предсказание, рассчитать R^2 , MAE, RMSE. Свести результаты в сравнительную таблицу. Построить график зависимости качества от версии модели.
Постусловие	Оформлен отчёт с обоснованием выбора финальной версии модели
Результат	Тест пройден
Примечания	Покрывает требования ФТ-4, нефункциональное требование производительности и надежности

Таблица 16 – Тест-кейс пункта 3.2.5 - №1

Функция	Обработка повреждённых входных файлов
Действие	Попытка загрузки файлов с нарушенной структурой: битый NEXUS, FASTA без заголовков, дерево без закрывающей скобки
Ожидаемый результат	Приложение не завершается аварийно, выводит диалоговое окно с понятным сообщением об ошибке, сохраняет работоспособность интерфейса
Предусловие	Приложение запущено, подготовлены тестовые файлы с искусственными повреждениями
Шаги теста	Нажать кнопку загрузки выравнивания. Выбрать файл с отсутствующим блоком MATRIX. Зафиксировать реакцию интерфейса. Повторить для файла без заголовков и дерева с незавершённой скобочной записью.
Постусловие	Интерфейс остаётся в исходном состоянии, кнопки загрузки активны, в журнале операций записаны сообщения об ошибках валидации
Результат	Тест пройден

Примечания	Покрывает требования: ФТ-1, НФТ-Надёжность, НФТ-Безопасность
------------	--

Таблица 17 – Тест-кейс пункта 3.2.5 - №2

Функция	Обработка несовпадения таксонов между выравниванием и деревом
Действие	Загрузка пары файлов, в которых множества имён таксонов частично или полностью не пересекаются
Ожидаемый результат	При запуске анализа система выявляет расхождение, выводит предупреждение с указанием количества несовпадающих таксонов, блокирует запуск предсказания
Предусловие	Загружено валидное выравнивание, подготовлен файл дерева с изменёнными именами таксонов
Шаги теста	Загрузить корректный файл выравнивания. Загрузить файл дерева, в котором 30% имён таксонов изменены. Нажать кнопку проверки входных данных. Попытаться запустить предсказание.
Постусловие	Система не переходит в невалидное состояние, пользователь получает информативную обратную связь, интерфейс предлагает загрузить согласованные файлы
Результат	Тест пройден
Примечания	Покрывает требования: ФТ-1, ФТ-4, НФТ-Надёжность, НФТ-Удобство использования

Таблица 18 – Тест-кейс пункта 3.2.5 - №3

Функция	Обработка пустых и минимальных входных структур
Действие	Подача на вход структур с экстремально малым объёмом: выравнивание из 1–2 таксонов, дерево без внутренних узлов
Ожидаемый результат	Система корректно обрабатывает пограничные случаи: для недостаточных данных выводится сообщение о невозможности анализа, приложение не генерирует исключения IndexError или KeyError

Предусловие	Подготовлены минимальные тестовые файлы: FASTA с двумя последовательностями, Newick с одним листовым узлом
Шаги теста	Загрузить выравнивание из 2 таксонов. Попытаться построить дерево. Загрузить дерево без внутренних узлов и запустить предсказание. Убедиться, что возвращается пустой результат, а не ошибка.
Постусловие	Приложение демонстрирует отказоустойчивость к пограничным условиям, пользователь получает понятные сообщения
Результат	Тест пройден
Примечания	Покрывает требования: ФТ-4, НФТ-Надёжность, НФТ-Безопасность

ПРИЛОЖЕНИЕ Б. РЕКВИЗИТНЫЙ СОСТАВ ТАБЛИЦ

Таблица 19 – Реквизитный состав таблиц

Таблица	Реквизит	Тип данных	Описание
ModelRF	model_id	INT (PK)	Уникальный идентификатор модели
ModelRF	version	VARCHAR	Версия модели
ModelRF	accuracy	FLOAT	Точность модели
Analysis	analysis_id	INT (PK)	Уникальный идентификатор анализа
Analysis	status	VARCHAR	Статус выполнения анализа
Analysis	date	DATETIME	Дата выполнения анализа
Analysis	model_id	INT (FK)	Ссылка на модель
Analysis	tree_id	INT (FK)	Ссылка на дерево
Tree	tree_id	INT (PK)	Уникальный идентификатор дерева
Tree	construction_method	VARCHAR	Метод построения
Tree	node_count	INT	Количество узлов
Tree	creation_source	VARCHAR	Источник создания
Tree	analysis_id	INT (FK)	Ссылка на анализ
Alignment	alignment_id	INT (PK)	Уникальный идентификатор
Alignment	sequence_count	INT	Количество последовательностей
Alignment	length	INT	Длина выравнивания
Sequence	sequence_id	INT (PK)	Уникальный идентификатор
Sequence	name	VARCHAR	Название последовательности
Sequence	data	TEXT	Данные последовательности
Sequence	length	INT	Длина последовательности
Sequence	alignment_id	INT (FK)	Ссылка на выравнивание

Таблица	Реквизит	Тип данных	Описание
Branch	branch_id	INT (PK)	Уникальный идентификатор ветви
Branch	length	FLOAT	Длина ветви
Branch	distance_to_root	FLOAT	Расстояние до корня
Branch	subtree_size	INT	Размер поддерева
Branch	tree_id	INT (FK)	Ссылка на дерево
Branch	branchassessment_id	INT (FK)	Ссылка на оценку
BranchAssessment	branchassessment_id	INT (PK)	Уникальный идентификатор
BranchAssessment	support_value	FLOAT	Значение достоверности
BranchAssessment	important_features	TEXT	Значимые признаки
BranchAssessment	analysis_id	INT (FK)	Ссылка на анализ
BranchAssessment	branch_id	INT (FK)	Ссылка на ветвь

ПРИЛОЖЕНИЕ В. БЛОК-СХЕМЫ

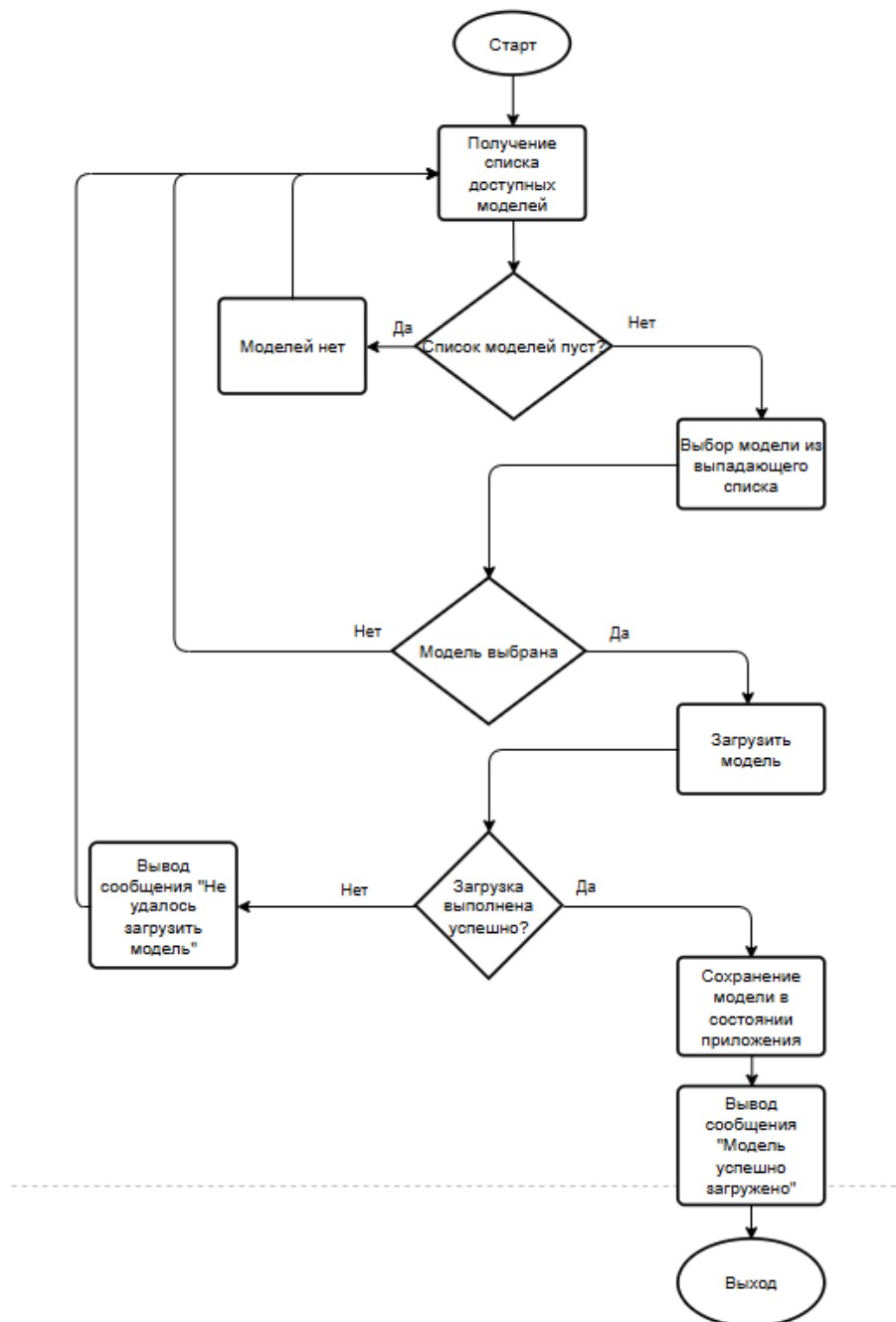


Рисунок 25 – Получение списка доступных моделей

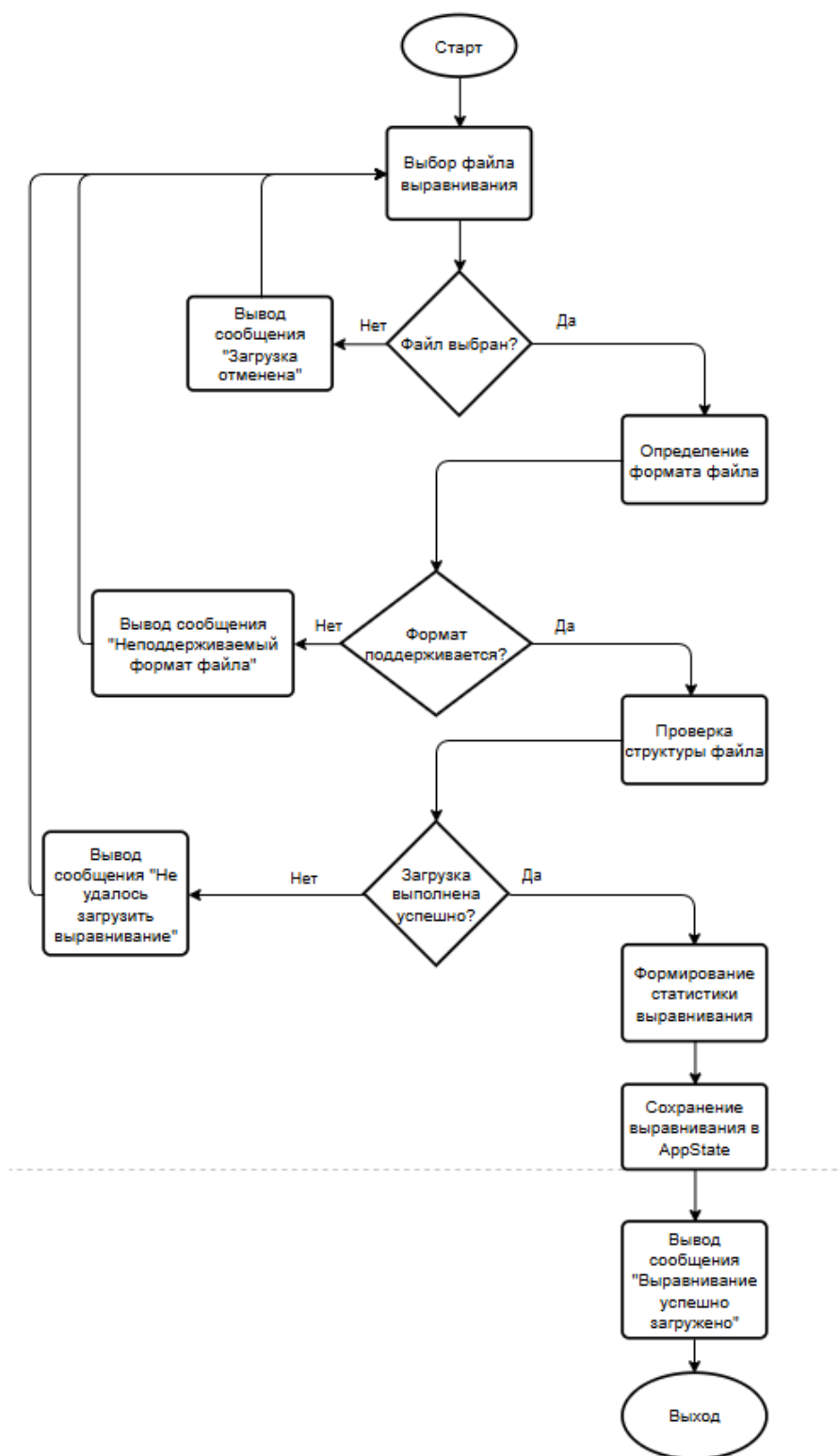


Рисунок 26 – Алгоритм загрузки входных данных

ПРИЛОЖЕНИЕ Г. РИСУНКИ РЕЗУЛЬТАТОВ ТЕСТОВ

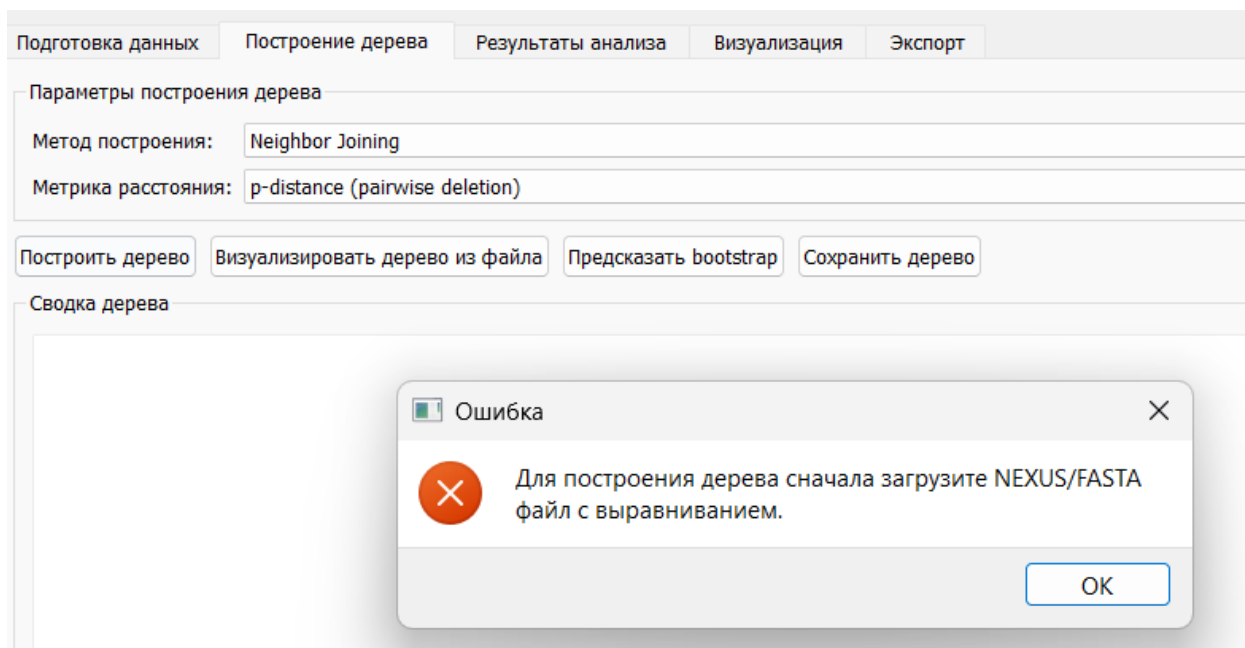


Рисунок 27 – Вывод сообщения об ошибке при незагруженном файле выравнивания

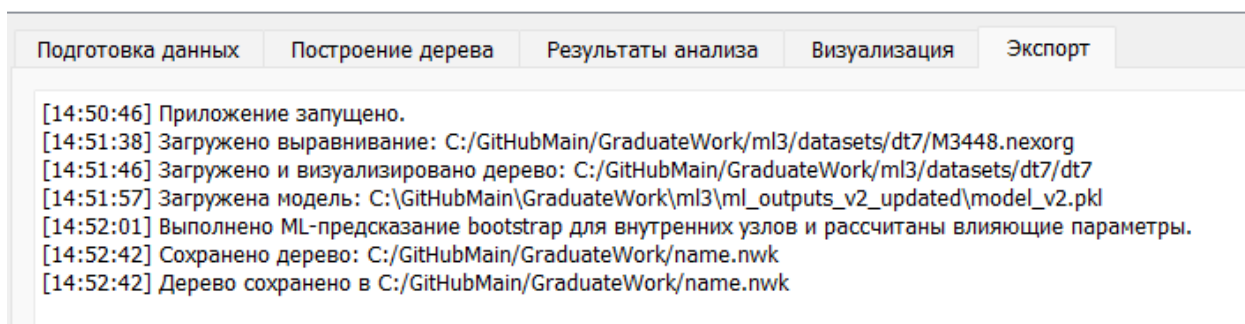


Рисунок 28 – Сохранение дерева в файл

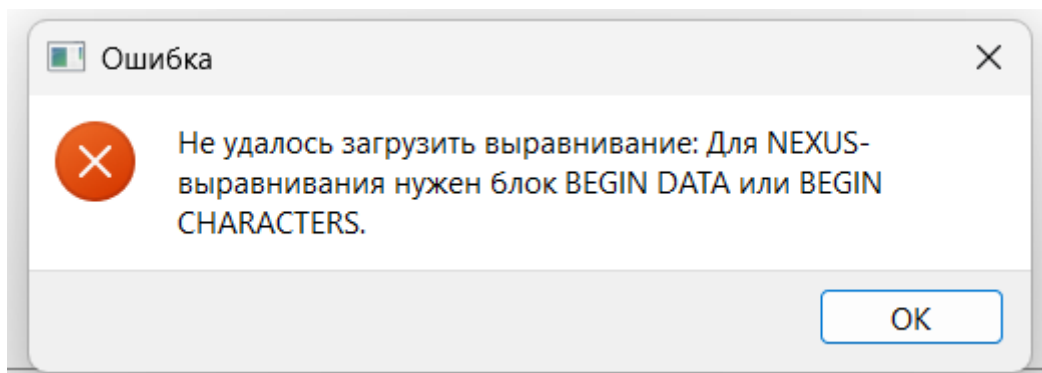


Рисунок 29 – Обработка ошибки

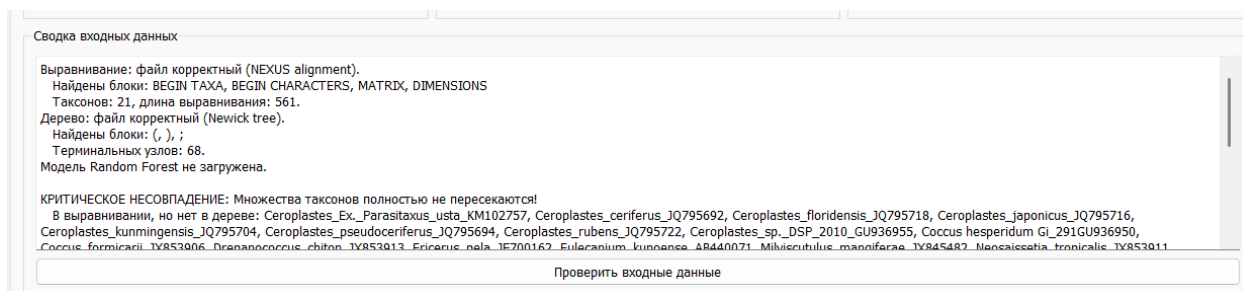


Рисунок 30 – Ошибка при проверке входных данных

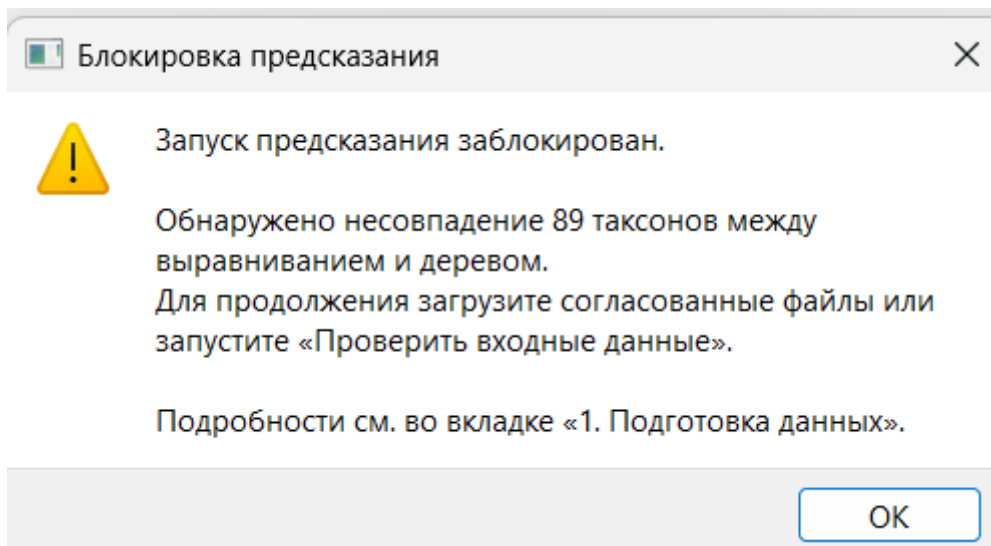


Рисунок 31 – Ошибка при запуске предсказания

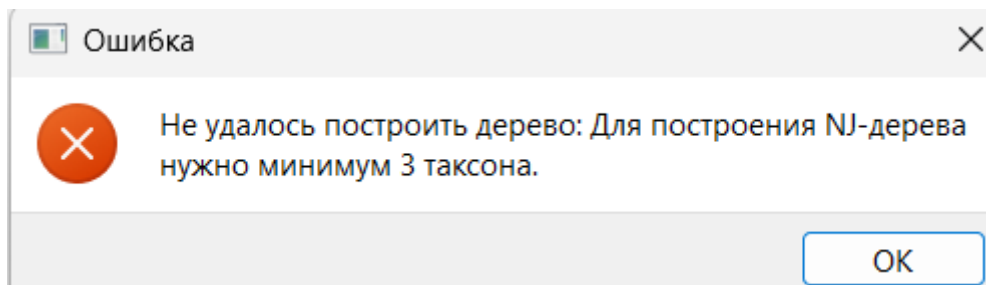


Рисунок 32 – Демонстрация ошибки малого количества

ПРИЛОЖЕНИЕ Д. ЛИСТИНГ КОДА

```

from domain.app_state_model import AppState
__all__ = ['AppState']
from ui.main_window import run
if __name__ == "__main__":
    run()
import os
import sys
from datetime import datetime
def extract_code_lines(root_dir,
output_file="project_code_lines.txt",
skip_empty=True):
    ALLOWED_EXTS = {".py"}
    ignore_dirs = {
        "__pycache__",
        ".pytest_cache",
        "iqtree-3.1.1.-Windows",
        "MrBayes-3.2.7.-WIN"
    }
    file_count = 0
    line_count = 0
    collected_lines = []
    print(" Сканирование директории...")
    for dirpath, dirnames, filenames in
os.walk(root_dir):
        dirnames[:] = [d for d in dirnames if d not
in ignore_dirs]
        for filename in filenames:
            _, ext = os.path.splitext(filename)
            if ext.lower() not in ALLOWED_EXTS:
                continue
            filepath = os.path.join(dirpath,
filename)
            try:
                with open(filepath, "r",
encoding="utf-8", errors="ignore") as f:
                    for line in f:
                        if skip_empty:
                            stripped = line.strip()
                            if not stripped:
                                continue
                            if ext.lower() == ".py"
and stripped.startswith(("#", "REM")):
                                continue
                        collected_lines.append(line)
                        line_count += 1
                        file_count += 1
            except Exception as e:
                print(f Ошибка чтения {filepath}:
{e}", file=sys.stderr)
                print(f Найдено файлов: {file_count}, строк:
{line_count}")
                print("Запись результата...")
                with open(output_file, "w", encoding="utf-8")
as out:
                    out.write("=" * 60 + "\n")
                    out.write("СТАТИСТИКА ИЗВЛЕЧЕНИЯ КОДА\n")
                    out.write("=" * 60 + "\n")
                    out.write(f"Путь к проекту:
{os.path.abspath(root_dir)}\n")
                    out.write(f"Дата генерации:
{datetime.now().strftime('%Y-%m-%d %H:%M:%S')}\n")
                    out.write(f"Обработано файлов:
{file_count}\n")
                    out.write(f"Извлечено строк:
{line_count}\n")
                    out.write(f"Фильтр расширений: {'',
'.join(sorted(ALLOWED_EXTS))}\n")
                    out.write("=" * 60 + "\n\n")
                    for line in collected_lines:
                        out.write(line)
                        print(f" Готово! Результат сохранён в:
{os.path.abspath(output_file)}")
if __name__ == "__main__":
    PROJECT_DIR = "." # "." = текущая папка, где
лежит скрипт
    OUTPUT_FILE = "project_code_lines.txt"
    extract_code_lines(PROJECT_DIR, OUTPUT_FILE,
skip_empty=True)
from dataclasses import dataclass, field
from Bio.Align import MultipleSeqAlignment
from Bio.Phylo.BaseTree import Tree
@dataclass
class AnalysisArtifacts:
    alignment: MultipleSeqAlignment | None
    tree: Tree | None
    alignment_info: dict =
field(default_factory=dict)
    tree_info: dict = field(default_factory=dict)
    node_rows: list[dict] =
field(default_factory=list)
@dataclass
class TreeBuildResultView:
    tree: Tree
    method: str
    log_likelihood: float | None = None
    support_label: str | None = None
    support_kind: str | None = None
    metadata: dict = field(default_factory=dict)
@dataclass
class ModelLoadResult:
    metadata: dict = field(default_factory=dict)
    loaded_now: bool = False
from dataclasses import dataclass, field
from typing import Optional
from Bio.Align import MultipleSeqAlignment
from Bio.Phylo.BaseTree import Tree
@dataclass
class AppState:
    tree_file: str = ""
    alignment_file: str = ""
    model_file: str = ""
    tree_loaded: bool = False
    alignment_loaded: bool = False
    model_loaded: bool = False
    tree_built: bool = False
    analysis_completed: bool = False
    input_validated: bool = False
    tree_method: str = "Neighbor Joining"
    tree_support_kind: str = "observed"
    tree_support_label: str = "Bootstrap"
    alignment: Optional[MultipleSeqAlignment] =
None
    tree: Optional[Tree] = None
    source_format: str = ""
    last_error: str = ""
    model_metadata: dict =
field(default_factory=dict)
    node_rows: list[dict] =
field(default_factory=list)
from .app_state_model import AppState
from .analysis_models import AnalysisArtifacts,
TreeBuildResultView, ModelLoadResult
__all__ = [
    'AppState',
    'AnalysisArtifacts',
    'TreeBuildResultView',
    'ModelLoadResult',
]
from __future__ import annotations
import numpy as np

```

```

from .sequence_processing import
GCFractionCalculator, GapFractionCalculator,
SequenceSanitizer
from .feature_constants import VALID_DNA
class AlignmentStatisticsCalculator:
    def __init__(self) -> None:
        self.sanitizer = SequenceSanitizer()
        self.gap_calculator =
GapFractionCalculator()
        self.gc_calculator = GCFractionCalculator()
    def calculate(self, seqs: dict[str, str]) ->
dict[str, float]:
        items = [self.sanitizer.sanitize(s) for s
in seqs.values()]
        if not items:
            return {
                'taxa_count': np.nan,
                'alignment_length': np.nan,
                'gap_fraction_global': np.nan,
                'gc_mean_global': np.nan,
                'gc_std_global': np.nan,
                'variable_site_fraction_global':
np.nan,
            }
            n = len(items)
            length = max(len(s) for s in items)
            items = [s.ljust(length, '-') for s in
items]
            gap_fraction_global =
np.mean([self.gap_calculator.calculate(s) for s in
items])
            gc_values =
[self.gc_calculator.calculate(s) for s in items]
            variable_site_fraction =
self._calculate_variable_site_fraction(items,
length)
            return {
                'taxa_count': float(n),
                'alignment_length': float(length),
                'gap_fraction_global':
float(gap_fraction_global),
                'gc_mean_global':
float(np.nanmean(gc_values)),
                'gc_std_global':
float(np.nanstd(gc_values)),
                'variable_site_fraction_global':
float(variable_site_fraction),
            }
        def _calculate_variable_site_fraction(self,
items: list[str], length: int) -> float:
            variable_sites = 0
            informative_positions = 0
            for i in range(length):
                col = [s[i] for s in items if s[i] in
VALID_DNA]
                if len(col) < 2:
                    continue
                informative_positions += 1
                if len(set(col)) > 1:
                    variable_sites += 1
            return variable_sites /
informative_positions if informative_positions else
np.nan
from __future__ import annotations
import numpy as np
from .sequence_processing import
GapFractionCalculator, GCFractionCalculator,
PairwiseDistanceCalculator, SequenceSanitizer
class CladeLeafCollector:
    def get_leaf_names(self, clade) -> list[str]:
        return [leaf.name for leaf in
clade.get_terminals() if leaf.name]
class MeanPairwiseDistanceCalculator:
    def __init__(self) -> None:
        self.sanitizer = SequenceSanitizer()

        self.distance_calculator =
PairwiseDistanceCalculator()
    def calculate(self, names: list[str], seqs:
dict[str, str]) -> float:
        if len(names) < 2:
            return 0.0
        vals = []
        for i in range(len(names)):
            for j in range(i + 1, len(names)):
                s1 =
self.sanitizer.sanitize(seqs.get(names[i], ''))
                s2 =
self.sanitizer.sanitize(seqs.get(names[j], ''))
                if not s1 or not s2:
                    continue
                d =
self.distance_calculator.calculate(s1, s2)
                if not np.isnan(d):
                    vals.append(d)
            return np.nan if not vals else
float(np.mean(vals))
class CladeStatisticsCalculator:
    def __init__(self) -> None:
        self.gap_calculator =
GapFractionCalculator()
        self.gc_calculator = GCFractionCalculator()
        self.sanitizer = SequenceSanitizer()
        self.distance_calculator =
MeanPairwiseDistanceCalculator()
    def calculate(self, names: list[str], seqs:
dict[str, str]) -> dict[str, float]:
        sub = {k: seqs[k] for k in names if k in
seqs}
        if not sub:
            return {
                'gap_fraction_clade': np.nan,
                'gc_mean_clade': np.nan,
                'gc_std_clade': np.nan,
                'mean_pairwise_pdist_clade':
np.nan,
            }
            gap_fraction_clade =
float(np.nanmean([self.gap_calculator.calculate(se
lf.sanitizer.sanitize(v)) for v in sub.values()])))
            gc_vals =
[self.gc_calculator.calculate(self.sanitizer.sanit
ize(v)) for v in sub.values()]
            return {
                'gap_fraction_clade':
gap_fraction_clade,
                'gc_mean_clade':
float(np.nanmean(gc_vals)),
                'gc_std_clade':
float(np.nanstd(gc_vals)),
                'mean_pairwise_pdist_clade':
self.distance_calculator.calculate(list(sub.keys()
), sub),
            }
class SubtreeBalanceCalculator:
    def calculate(self, clade) -> float:
        children = clade.clades
        if len(children) < 2:
            return 0.0
        sizes = [len(ch.get_terminals()) for ch in
children]
        total = sum(sizes)
        return 0.0 if total == 0 else abs(sizes[0]
- sizes[1]) / total
from __future__ import annotations
import pandas as pd
FEATURE_LABELS = {
    'branch_length': 'длина ветви',
    'depth': 'глубина узла',
    'subtree_fraction': 'доля поддеревы',
    'subtree_balance': 'баланс поддеревы',

```



```

        'mean_child_branch_length': 'средняя длина
дочерних ветвей',
        'std_child_branch_length': 'разброс длин
дочерних ветвей',
        'n_children': 'число дочерних узлов',
        'clade_size_log': 'логарифм размера клады',
        'relative_branch_length': 'относительная длина
ветви',
        'depth_ratio': 'относительная глубина',
        'taxa_count': 'число таксонов',
        'alignment_length': 'длина выравнивания',
        'gap_fraction_global': 'глобальная доля
пропусков',
        'gc_mean_global': 'средний GC по всему
выравниванию',
        'gc_std_global': 'разброс GC по всему
выравниванию',
        'variable_site_fraction_global': 'доля
вариабельных сайтов',
        'gap_fraction_clade': 'доля пропусков в кладе',
        'gc_mean_clade': 'средний GC в кладе',
        'gc_std_clade': 'разброс GC в кладе',
        'mean_pairwise_pdist_clade': 'средняя p-
distance внутри клады',
    }
}
class FeatureExplanationBuilder:
    def build(self, X: pd.DataFrame,
feature_importance_map: dict[str, float]) ->
list[dict]:
    if X.empty:
        return []
    numeric = X.astype(float)
    medians = numeric.median(axis=0)
    stds = numeric.std(axis=0).replace(0.0,
1.0).fillna(1.0)
    importances = {col:
float(feature_importance_map.get(col, 0.0)) for col
in numeric.columns}
    if not any(importances.values()):
        importances = {col: 1.0 / max(1,
len(numeric.columns)) for col in numeric.columns}
    results = []
    for _, row in numeric.iterrows():
        items = []
        for col in numeric.columns:
            val = row[col]
            if pd.isna(val):
                value = 0.0
                direction = 'отсутствует'
            else:
                value = float(val)
                direction = 'выше среднего' if
value >= float(medians[col]) else 'ниже среднего'
            score = abs((value -
float(medians[col])) / float(stds[col])) *
importances.get(col, 0.0)
            items.append({
                'feature': col,
                'label':
FEATURE_LABELS.get(col, col),
                'value': value,
                'importance':
float(importances.get(col, 0.0)),
                'score': float(score),
                'direction': direction,
            })
        top = sorted(items, key=lambda x:
x['score'], reverse=True)[:5]
        text_lines = ['Параметры, сильнее всего
влияющие на predicted bootstrap:']
        for item in top:
            text_lines.append(
                f"• {item['label']}:
{item['value']:.4f} ({item['direction']}), важность
модели {item['importance']:.3f}")
    )

```

```

        results.append({'items': top, 'text':
'\n'.join(text_lines)})
    return results
FEATURE_COLUMNS = [
    'branch_length',
    'depth',
    'n_leaves_subtree',
    'subtree_fraction',
    'subtree_balance',
    'mean_child_branch_length',
    'std_child_branch_length',
    'n_children',
    'clade_size_log',
    'relative_branch_length',
    'depth_ratio',
    'taxa_count',
    'alignment_length',
    'gap_fraction_global',
    'gc_mean_global',
    'gc_std_global',
    'variable_site_fraction_global',
    'gap_fraction_clade',
    'gc_mean_clade',
    'gc_std_clade',
    'mean_pairwise_pdist_clade',
]
VALID_DNA = set('ACGT')
DNA_PLUS_GAP = set('ACGT-?')
from __future__ import
annotations
from Bio.Align import MultipleSeqAlignment
from Bio.Phylo.BaseTree import Tree
from .alignment_statistics import
AlignmentStatisticsCalculator
from .clade_statistics import CladeLeafCollector,
CladeStatisticsCalculator,
MeanPairwiseDistanceCalculator,
SubtreeBalanceCalculator
from .feature_constants import DNA_PLUS_GAP,
FEATURE_COLUMNS, VALID_DNA
from .feature_extractor import NodeFeatureRow,
TreeFeatureExtractor
from .sequence_processing import
AlignmentSequenceMapper, GapFractionCalculator,
GCFractionCalculator, PairwiseDistanceCalculator,
SequenceSanitizer
_sanitizer = SequenceSanitizer()
_mapper = AlignmentSequenceMapper()
_distance = PairwiseDistanceCalculator()
_gap = GapFractionCalculator()
_gc = GCFractionCalculator()
_alignment_stats = AlignmentStatisticsCalculator()
_leaf_collector = CladeLeafCollector()
_mpd = MeanPairwiseDistanceCalculator()
_clade_stats = CladeStatisticsCalculator()
_balance = SubtreeBalanceCalculator()
_extractor = TreeFeatureExtractor()
def sanitize_seq(seq: str) -> str:
    return _sanitizer.sanitize(seq)
def alignment_to_dict(alignment:
MultipleSeqAlignment) -> dict[str, str]:
    return _mapper.to_dict(alignment)
def p_distance(seq1: str, seq2: str) -> float:
    return _distance.calculate(seq1, seq2)
def seq_gap_fraction(seq: str) -> float:
    return _gap.calculate(seq)
def seq_gc_fraction(seq: str) -> float:
    return _gc.calculate(seq)
def alignment_stats(seqs: dict[str, str]) ->
dict[str, float]:
    return _alignment_stats.calculate(seqs)
def get_leaf_names(clade) -> list[str]:
    return _leaf_collector.get_leaf_names(clade)
def mean_pairwise_distance(names: list[str], seqs:
dict[str, str]) -> float:
    return _mpd.calculate(names, seqs)
def clade_alignment_stats(names: list[str], seqs:
dict[str, str]) -> dict[str, float]:

```

```

    return _clade_stats.calculate(names, seqs)
def subtree_balance(clade) -> float:
    return _balance.calculate(clade)
def extract_feature_rows(tree: Tree, alignment:
MultipleSeqAlignment) -> list[NodeFeatureRow]:
    return _extractor.extract(tree, alignment)
__all__ = [
    'FEATURE_COLUMNS', 'VALID_DNA', 'DNA_PLUS_GAP',
    'NodeFeatureRow',
    'sanitize_seq', 'alignment_to_dict',
    'p_distance', 'seq_gap_fraction',
    'seq_gc_fraction',
    'alignment_stats', 'get_leaf_names',
    'mean_pairwise_distance', 'clade_alignment_stats',
    'subtree_balance', 'extract_feature_rows'
]
from __future__ import annotations
from dataclasses import dataclass
import numpy as np
from Bio.Align import MultipleSeqAlignment
from Bio.Phylo.BaseTree import Tree
from .alignment_statistics import
AlignmentStatisticsCalculator
from .clade_statistics import CladeLeafCollector,
CladeStatisticsCalculator, SubtreeBalanceCalculator
from .sequence_processing import
AlignmentSequenceMapper
@dataclass
class NodeFeatureRow:
    node_id: str
    features: dict[str, float]
class NodeFeatureFactory:
    def create(self, node_id: str, features:
dict[str, float]) -> NodeFeatureRow:
        return NodeFeatureRow(node_id=node_id,
features=features)
class TreeFeatureExtractor:
    def __init__(self) -> None:
        self.sequence_mapper =
AlignmentSequenceMapper()
        self.alignment_stats_calculator =
AlignmentStatisticsCalculator()
        self.leaf_collector = CladeLeafCollector()
        self.clade_stats_calculator =
CladeStatisticsCalculator()
        self.balance_calculator =
SubtreeBalanceCalculator()
        self.row_factory = NodeFeatureFactory()
    def extract(self, tree: Tree, alignment:
MultipleSeqAlignment) -> list[NodeFeatureRow]:
        seqs =
self.sequence_mapper.to_dict(alignment)
        global_stats =
self.alignment_stats_calculator.calculate(seqs)
        depths = tree.depths()
        max_depth = max(depths.values()) if depths
else 1.0
        rows: list[NodeFeatureRow] = []
        for idx, clade in
enumerate(tree.get_nonterminals(order='level'),
start=1):
            rows.append(self._build_row(idx, clade,
depths, max_depth, seqs, global_stats))
        return rows
    def _build_row(self, idx: int, clade, depths,
max_depth: float, seqs: dict[str, str],
global_stats: dict[str, float]) -> NodeFeatureRow:
        branch_length =
max(float(clade.branch_length or 0.0), 0.0)
        depth = float(depths.get(clade, 0.0))
        leaf_names =
self.leaf_collector.get_leaf_names(clade)
        n_leaves = len(leaf_names)
        clade_stats =
self.clade_stats_calculator.calculate(leaf_names,
seqs)

```

```

        child_branch_lengths =
[max(float(ch.branch_length or 0.0), 0.0) for ch in
clade.clades]
        features = {
            'branch_length': branch_length,
            'depth': depth,
            'n_leaves_subtree': float(n_leaves),
            'subtree_fraction': float(n_leaves /
global_stats['taxa_count']) if
global_stats['taxa_count'] else np.nan,
            'subtree_balance':
float(self.balance_calculator.calculate(clade)),
            'mean_child_branch_length':
float(np.mean(child_branch_lengths)) if
child_branch_lengths else 0.0,
            'std_child_branch_length':
float(np.std(child_branch_lengths)) if
child_branch_lengths else 0.0,
            'n_children': float(len(clade.clades)),
            'clade_size_log':
float(np.log1p(n_leaves)),
            'relative_branch_length':
float(branch_length / max_depth) if max_depth > 0
else 0.0,
            'depth_ratio': float(depth / max_depth)
if max_depth > 0 else 0.0,
            **global_stats,
            **clade_stats,
        }
        return
self.row_factory.create(node_id=f'N{idx}',
features=features)from .explanation import
FEATURE_LABELS
from .model_loader import ModelInferenceError
from .predictor_core import
RandomForestBootstrapPredictor
__all__ = ['ModelInferenceError', 'FEATURE_LABELS',
'RandomForestBootstrapPredictor']
from __future__ import annotations
import json
from pathlib import Path
import joblib
import pandas as pd
from .feature_constants import FEATURE_COLUMNS
class ModelInferenceError(RuntimeError):
    pass
class FeatureImportanceRepository:
    def load(self, model, model_path: Path) ->
dict[str, float]:
        csv_path =
model_path.with_name('feature_importance.csv')
        if csv_path.exists():
            df = pd.read_csv(csv_path)
            if
{'feature',
'importance'}.issubset(df.columns):
                return {str(r['feature']):
float(r['importance']) for _, r in df.iterrows()}
            if hasattr(model, 'feature_importances_'):
                values = list(getattr(model,
'feature_importances_'))
                return {col: float(values[idx]) for
idx, col in enumerate(FEATURE_COLUMNS) if idx <
len(values)}
        return {}
class MetadataRepository:
    def load(self, metadata_path: str | Path | None)
-> tuple[dict, str]:
        if not metadata_path:
            return {}, ''
        metadata_file =
Path(metadata_path).resolve()
        if not metadata_file.exists():
            return {}, ''
        data =
json.loads(metadata_file.read_text(encoding='utf-8'))

```

```

        return data, str(metadata_file)
class SerializedModelLoader:
    def load(self, model_path: Path):
        return joblib.load(model_path)
from __future__ import annotations
from pathlib import Path
import numpy as np
import pandas as pd
from Bio.Align import MultipleSeqAlignment
from Bio.Phylo.BaseTree import Tree
from .explanation import FeatureExplanationBuilder
from .feature_constants import FEATURE_COLUMNS
from .feature_extractor import TreeFeatureExtractor
from .model_loader import import
FeatureImportanceRepository, MetadataRepository,
ModelInferenceError, SerializedModelLoader
class FeatureFrameBuilder:
    def build(self, feature_rows, expected_columns:
list[str] | None = None) -> pd.DataFrame:
        X = pd.DataFrame([row.features for row in
feature_rows])
        target_cols = expected_columns or
FEATURE_COLUMNS
        X = self._compute_derived_features(X,
target_cols)
        return X.reindex(columns=target_cols,
fill_value=np.nan)
    @staticmethod
    def _compute_derived_features(X: pd.DataFrame,
expected_cols: list[str]) -> pd.DataFrame:
        if "bl_cv" in expected_cols and "bl_cv" not
in X.columns:
            X["bl_cv"] =
X["std_child_branch_length"] /
(X["mean_child_branch_length"] + 1e-6)
            if "gap_var_clade" in expected_cols and
"gap_var_clade" not in X.columns:
                X["gap_var_clade"] =
(X["gap_fraction_clade"] -
X["gap_fraction_global"]).abs()
            if "gc_extreme" in expected_cols and
"gc_extreme" not in X.columns:
                X["gc_extreme"] = ((X["gc_mean_clade"]
- X["gc_mean_global"]).abs() > 0.15).astype(float)
            if "tiny_clade" in expected_cols and
"tiny_clade" not in X.columns:
                X["tiny_clade"] =
(X["n_leaves_subtree"] <= 3).astype(float)
            if "deep_node" in expected_cols and
"deep_node" not in X.columns:
                max_depth = X["depth"].max()
                if pd.isna(max_depth) or max_depth <=
0:
                    max_depth = 1.0
                X["deep_node"] = X["depth"] /
(max_depth + 1e-6)
        return X
class PredictionExecutor:
    def predict(self, model, X: pd.DataFrame):
        try:
            return model.predict(X)
        except Exception:
            if hasattr(model, 'named_steps') and
'rf' in model.named_steps:
                rf = model.named_steps['rf']
                imputer =
model.named_steps.get('imputer')
                X_num = X.copy().astype(float)
                if imputer is not None and
hasattr(imputer, 'statistics_'):
                    stats =
list(imputer.statistics_)
                    for idx, col in
enumerate(X_num.columns):
                        fill = stats[idx] if idx <
len(stats) else 0.0

```

```

                        X_num[col]
X_num[col].fillna(fill)
                    else:
                        X_num
X_num.fillna(X_num.median(numeric_only=True))
                return rf.predict(X_num)
            raise
class PredictionResultAssembler:
    def assemble(self, feature_rows, preds,
explanations) -> list[dict]:
        results = []
        for row, pred, explanation in
zip(feature_rows, preds, explanations):
            pred = max(0.0, min(100.0,
float(pred)))
            results.append({
                'node_id': row.node_id,
                'predicted_bootstrap': pred,
                'feature_influences':
explanation['items'],
                'feature_influence_text':
explanation['text'],
                **row.features,
            })
        return results
class RandomForestBootstrapPredictor:
    def __init__(self) -> None:
        self.model = None
        self.metadata = {}
        self.model_path = ''
        self.metadata_path = ''
        self.feature_importance_map: dict[str,
float] = {}
        self.model_loader = SerializedModelLoader()
        self.metadata_repository =
MetadataRepository()
        self.importance_repository =
FeatureImportanceRepository()
        self.feature_extractor =
TreeFeatureExtractor()
        self.frame_builder = FeatureFrameBuilder()
        self.prediction_executor =
PredictionExecutor()
        self.explanation_builder =
FeatureExplanationBuilder()
        self.result_assembler =
PredictionResultAssembler()
        def load(self, model_path: str | Path,
metadata_path: str | Path | None = None) -> bool:
            resolved_model_path =
Path(model_path).resolve()
            if not resolved_model_path.exists():
                raise ModelInferenceError(f'Файл модели
не найден: {resolved_model_path}')
            metadata, resolved_metadata =
self.metadata_repository.load(metadata_path)
            if self.model is not None and
self.model_path == str(resolved_model_path) and
self.metadata_path == resolved_metadata:
                return False
            self.model =
self.model_loader.load(resolved_model_path)
            self.model_path = str(resolved_model_path)
            self.metadata_path = resolved_metadata
            self.feature_importance_map =
self.importance_repository.load(self.model,
resolved_model_path)
            self.metadata = metadata
        self.metadata.setdefault('feature_importances',
self.feature_importance_map)
        return True
    @property
    def is_loaded(self) -> bool:
        return self.model is not None

```

```

def predict_tree(self, tree: Tree, alignment:
MultipleSeqAlignment) -> list[dict]:
    if self.model is None:
        raise ModelInferenceError('Модель не
загружена.')
    feature_rows =
self.feature_extractor.extract(tree, alignment)
    if not feature_rows:
        return []
    expected_cols =
self.metadata.get("feature_columns",
FEATURE_COLUMNS)
    X = self.frame_builder.build(feature_rows,
expected_columns=expected_cols)
    preds =
self.prediction_executor.predict(self.model, X)
    explanations =
self.explanation_builder.build(X,
self.feature_importance_map)
    return
self.result_assembler.assemble(feature_rows, preds,
explanations)
from __future__ import annotations
import numpy as np
from Bio.Align import MultipleSeqAlignment
from .feature_constants import DNA_PLUS_GAP,
VALID_DNA
class SequenceSanitizer:
    def sanitize(self, seq: str) -> str:
        return ''.join(ch.upper() for ch in seq if
ch.upper() in DNA_PLUS_GAP)
class AlignmentSequenceMapper:
    def to_dict(self, alignment:
MultipleSeqAlignment) -> dict[str, str]:
        return {record.id: str(record.seq) for
record in alignment}
class PairwiseDistanceCalculator:
    def calculate(self, seq1: str, seq2: str) ->
float:
        mismatches = 0
        valid = 0
        for a, b in zip(seq1, seq2):
            if a in {'-', '?'} or b in {'-', '?'}:
                continue
            if a not in VALID_DNA or b not in
VALID_DNA:
                continue
            valid += 1
            if a != b:
                mismatches += 1
        return np.nan if valid == 0 else mismatches
/ valid
class GapFractionCalculator:
    def calculate(self, seq: str) -> float:
        return np.nan if not seq else sum(ch in {'-
', '?'} for ch in seq) / len(seq)
class GCFractionCalculator:
    def calculate(self, seq: str) -> float:
        letters = [ch for ch in seq if ch in
VALID_DNA]
        return np.nan if not letters else sum(ch in
{'G', 'C'} for ch in letters) / len(letters)
from __future__ import annotations
import io
import json
import math
import re
from collections import defaultdict
from pathlib import Path
from typing import Dict, List
import joblib
import numpy as np
import pandas as pd
from Bio import Phylo
from sklearn.ensemble import RandomForestRegressor
from sklearn.impute import SimpleImputer

```

```

from sklearn.metrics import mean_absolute_error,
r2_score, mean_squared_error
from sklearn.model_selection import
GroupShuffleSplit
from sklearn.pipeline import Pipeline
def parse_fasta(text: str) -> Dict[str, str]:
    seqs = {}
    current = None
    for raw_line in text.splitlines():
        line = raw_line.strip()
        if not line:
            continue
        if line.startswith(">"):
            current = line[1:].strip()
            seqs[current] = []
        else:
            if current is None:
                continue
            seqs[current].append(line.replace(" ",
""))
    return {k: "".join(v) for k, v in seqs.items()}
def _extract_nexus_taxa(text: str) -> List[str]:
    m = re.search(r"BEGIN\s+TAXA;(.*?)END;", text,
flags=re.IGNORECASE | re.DOTALL)
    if not m:
        return []
    block = m.group(1)
    tm = re.search(r"TAXLABELS(.*?)", block,
flags=re.IGNORECASE | re.DOTALL)
    if not tm:
        return []
    tokens = re.split(r"\s+", tm.group(1).strip())
    return [t.strip() for t in tokens if t.strip()]
def _parse_nexus_matrix_block(block: str,
known_taxa: List[str]) -> Dict[str, str]:
    mm = re.search(r"bMATRIX\b(.*?)", block,
flags=re.IGNORECASE | re.DOTALL)
    if not mm:
        return {}
    matrix_text = mm.group(1)
    seqs = defaultdict(list)
    known_taxa_sorted = sorted(known_taxa, key=len,
reverse=True)
    for raw_line in matrix_text.splitlines():
        stripped = raw_line.strip()
        if not stripped or stripped.startswith(""):
            continue
        matched = False
        for taxon in known_taxa_sorted:
            if stripped.startswith(taxon):
                rest =
stripped[len(taxon):].strip().replace(" ", "")
                if rest:
                    seqs[taxon].append(rest)
                    matched = True
                    break
        if not matched:
            parts = stripped.split()
            if len(parts) >= 2:
                taxon = parts[0]
                seq = "".join(parts[1:]).replace("
", "")
                seqs[taxon].append(seq)
    return {k: "".join(v) for k, v in seqs.items()}
def parse_nexus_alignment(text: str) -> Dict[str,
str]:
    known_taxa = _extract_nexus_taxa(text)
    char_blocks =
re.findall(r"BEGIN\s+CHARACTERS;(.*?)END;", text,
flags=re.IGNORECASE | re.DOTALL)
    if not char_blocks:
        raise ValueError("В файле не найдено BEGIN
CHARACTERS.")
    merged = defaultdict(list)
    for block in char_blocks:

```

```

        part = _parse_nexus_matrix_block(block,
known_taxa)
        for taxon, seq in part.items():
            merged[taxon].append(seq)
        if not merged:
            raise ValueError("Не удалось извлечь
последовательности из MATRIX.")
        return {k: "".join(v) for k, v in
merged.items()}
def load_alignment_file(path: str | Path) ->
Dict[str, str]:
    path = Path(path)
    text = path.read_text(encoding="utf-8",
errors="ignore")
    upper = text.upper()
    if "#NEXUS" in upper or "BEGIN CHARACTERS;" in
upper:
        return parse_nexus_alignment(text)
    if text.lstrip().startswith(">"):
        return parse_fasta(text)
    raise ValueError(f"Неподдерживаемый формат
alignment: {path}")
def clean_newick(tree_str: str) -> str:
    return re.sub(r"\[.*?\]", "",
tree_str.strip()).strip()
def extract_newick_from_text(text: str) -> str:
    text = text.strip()
    tree_match =
re.search(r"\bTREE\b\s+[^\s]+\s*(\.(.*?);)", text,
flags=re.IGNORECASE | re.DOTALL)
    if tree_match:
        return clean_newick(tree_match.group(1))
    first_paren = text.find("(")
    last_semicolon = text.rfind(";")
    if first_paren != -1 and last_semicolon != -1
and last_semicolon > first_paren:
        return
    clean_newick(text[first_paren:last_semicolon + 1])
    raise ValueError("Не удалось извлечь Newick из
файла дерева.")
def load_tree_file(path: str | Path):
    path = Path(path)
    text = path.read_text(encoding="utf-8",
errors="ignore")
    newick = extract_newick_from_text(text)
    tree = Phylo.read(io.StringIO(newick),
"newick")
    return tree, newick
VALID_DNA = set("ACGT")
DNA_PLUS_GAP = set("ACGT-?")
def sanitize_seq(seq: str) -> str:
    return "".join(ch.upper() for ch in seq if
ch.upper() in DNA_PLUS_GAP)
def p_distance(seq1: str, seq2: str) -> float:
    mismatches = 0
    valid = 0
    for a, b in zip(seq1, seq2):
        if a in {"-", "?"} or b in {"-", "?"}:
            continue
        if a not in VALID_DNA or b not in VALID_DNA:
            continue
        valid += 1
        if a != b:
            mismatches += 1
    return np.nan if valid == 0 else mismatches /
valid
def seq_gap_fraction(seq: str) -> float:
    return np.nan if not seq else sum(ch in {"-",
"?"} for ch in seq) / len(seq)
def seq_gc_fraction(seq: str) -> float:
    letters = [ch for ch in seq if ch in VALID_DNA]
    return np.nan if not letters else sum(ch in
{"G", "C"} for ch in letters) / len(letters)
def alignment_stats(seqs: Dict[str, str]) ->
Dict[str, float]:
    items = [sanitize_seq(s) for s in seqs.values()]

```

```

        if not items:
            return {
                "taxa_count": np.nan,
                "alignment_length": np.nan,
                "gap_fraction_global": np.nan,
                "gc_mean_global": np.nan,
                "gc_std_global": np.nan,
                "variable_site_fraction_global":
np.nan,
            }
            n = len(items)
            L = max(len(s) for s in items)
            items = [s.ljust(L, "-") for s in items]
            gap_fraction_global =
np.mean([seq_gap_fraction(s) for s in items])
            gc_values = [seq_gc_fraction(s) for s in items]
            gc_mean_global = float(np.nanmean(gc_values))
            gc_std_global = float(np.nanstd(gc_values))
            variable_sites = 0
            informative_positions = 0
            for i in range(L):
                col = [s[i] for s in items if s[i] in
VALID_DNA]
                if len(col) < 2:
                    continue
                informative_positions += 1
                if len(set(col)) > 1:
                    variable_sites += 1
            variable_site_fraction = variable_sites /
informative_positions if informative_positions else
np.nan
            return {
                "taxa_count": float(n),
                "alignment_length": float(L),
                "gap_fraction_global":
float(gap_fraction_global),
                "gc_mean_global": gc_mean_global,
                "gc_std_global": gc_std_global,
                "variable_site_fraction_global":
float(variable_site_fraction),
            }
def get_leaf_names(clade) -> List[str]:
    return [leaf.name for leaf in
clade.get_terminals() if leaf.name]
def mean_pairwise_distance(names: List[str], seqs:
Dict[str, str]) -> float:
    if len(names) < 2:
        return 0.0
    vals = []
    for i in range(len(names)):
        for j in range(i + 1, len(names)):
            s1 = sanitize_seq(seqs.get(names[i],
""))
            s2 = sanitize_seq(seqs.get(names[j],
""))
            if not s1 or not s2:
                continue
            d = p_distance(s1, s2)
            if not np.isnan(d):
                vals.append(d)
    return np.nan if not vals else
float(np.mean(vals))
def clade_alignment_stats(names: List[str], seqs:
Dict[str, str]) -> Dict[str, float]:
    sub = {k: seqs[k] for k in names if k in seqs}
    if not sub:
        return {
            "gap_fraction_clade": np.nan,
            "gc_mean_clade": np.nan,
            "gc_std_clade": np.nan,
            "mean_pairwise_pdist_clade": np.nan,
        }
    gap_fraction_clade =
float(np.nanmean([seq_gap_fraction(sanitize_seq(v)
) for v in sub.values()])))

```

```

gc_vals = [seq_gc_fraction(sanitize_seq(v)) for
v in sub.values()]
gc_mean_clade = float(np.nanmean(gc_vals))
gc_std_clade = float(np.nanstd(gc_vals))
mpd = mean_pairwise_distance(list(sub.keys()),
sub)
return {
    "gap_fraction_clade": gap_fraction_clade,
    "gc_mean_clade": gc_mean_clade,
    "gc_std_clade": gc_std_clade,
    "mean_pairwise_pdist_clade": mpd,
}
def subtree_balance(clade) -> float:
    children = clade.clades
    if len(children) < 2:
        return 0.0
    sizes = [len(ch.get_terminals()) for ch in
children]
    total = sum(sizes)
    return 0.0 if total == 0 else abs(sizes[0] -
sizes[1]) / total
def extract_node_features(tree, seqs: Dict[str,
str], dataset_id: str) -> pd.DataFrame:
    global_stats = alignment_stats(seqs)
    depths = tree.depths()
    rows = []
    node_idx = 0
    for clade in tree.find_clades(order="level"):
        if clade.is_terminal():
            continue
        conf = clade.confidence
        if conf is None:
            continue
        bootstrap = float(conf)
        if bootstrap <= 1.0:
            bootstrap *= 100.0
        branch_length
max(float(clade.branch_length or 0.0), 0.0) =
        leaf_names = get_leaf_names(clade)
        n_leaves = len(leaf_names)
        clade_stats
=
        clade_alignment_stats(leaf_names, seqs)
        child_branch_lengths
=
        [max(float(ch.branch_length or 0.0), 0.0) for ch in
clade.clades]
        rows.append({
            "dataset_id": dataset_id,
            "node_id":
f"{dataset_id}_{node_idx}",
            "target_bootstrap": bootstrap,
            "branch_length": branch_length,
            "depth": float(depths.get(clade, 0.0)),
            "n_leaves_subtree": float(n_leaves),
            "subtree_fraction": float(n_leaves /
global_stats["taxa_count"]) if
global_stats["taxa_count"] else np.nan,
            "subtree_balance":
float(subtree_balance(clade)),
            "mean_child_branch_length":
float(np.mean(child_branch_lengths)) if
child_branch_lengths else 0.0,
            "std_child_branch_length":
float(np.std(child_branch_lengths)) if
child_branch_lengths else 0.0,
            **global_stats,
            **clade_stats,
        })
        node_idx += 1
    return pd.DataFrame(rows)
ALIGNMENT_EXTS = {"nex", ".nexus", ".nexorg",
".fasta", ".fa", ".fas", ".txt"}
TREE_HINTS = ("tree", "bootstrap", "nwk", "newick",
"con", "majority")
def detect_alignment_file(ds_path: Path) -> Path |
None:

```

```

candidates = [f for f in ds_path.iterdir() if
f.is_file() and f.suffix.lower() in ALIGNMENT_EXTS]
candidates = sorted(candidates, key=lambda p:
(p.suffix.lower() not in {"nex", ".nexus",
".nexorg"}, p.name.lower()))
return candidates[0] if candidates else None
def detect_tree_file(ds_path: Path, alignment_file:
Path | None) -> Path | None:
    files = [f for f in ds_path.iterdir() if
f.is_file()]
    if alignment_file is not None:
        files = [f for f in files if f.resolve() !=
alignment_file.resolve()]
    scored = []
    for f in files:
        name_l = f.name.lower()
        score = 0
        if any(h in name_l for h in TREE_HINTS):
            score += 3
        if f.suffix.lower() in {"nwk", ".newick",
".tree"}:
            score += 3
        if f.suffix == "":
            score += 1
        scored.append((score, f))
    scored.sort(key=lambda x: (-x[0],
x[1].name.lower()))
    for _, f in scored:
        try:
            load_tree_file(f)
            return f
        except Exception:
            continue
    return None
def build_feature_table(datasets_root: str | Path):
    datasets_root = Path(datasets_root)
    all_frames = []
    audit = []
    for ds_path in sorted(datasets_root.iterdir()):
        if not ds_path.is_dir():
            continue
        dataset_id = ds_path.name
        alignment_file
=
        detect_alignment_file(ds_path)
        tree_file = detect_tree_file(ds_path,
alignment_file)
        status = {
            "dataset_id": dataset_id,
            "alignment_file": str(alignment_file)
if alignment_file else None,
            "tree_file": str(tree_file) if
tree_file else None,
            "ok": False,
            "error": None,
        }
        try:
            if alignment_file is None:
                raise FileNotFoundError("Не найден
alignment-файл")
            if tree_file is None:
                raise FileNotFoundError("Не найден
tree-файл")
            seqs
=
            load_alignment_file(alignment_file)
            tree, _ = load_tree_file(tree_file)
            df = extract_node_features(tree, seqs,
dataset_id)
            if df.empty:
                raise ValueError("Не извлечено ни
одного внутреннего узла с bootstrap")
            all_frames.append(df)
            status["ok"] = True
            status["n_rows"] = int(len(df))
        except Exception as e:
            status["error"] = str(e)
        audit.append(status)

```

```

        if not all_frames:
            audit_df = pd.DataFrame(audit)
            raise RuntimeError("Не удалось собрать ни
одной строки датасета. Проверь dataset_audit:\n" +
audit_df.to_string(index=False))
        return pd.concat(all_frames,
ignore_index=True), audit
FEATURE_COLUMNS = [
    "branch_length",
    "depth",
    "n_leaves_subtree",
    "subtree_fraction",
    "subtree_balance",
    "mean_child_branch_length",
    "std_child_branch_length",
    "taxa_count",
    "alignment_length",
    "gap_fraction_global",
    "gc_mean_global",
    "gc_std_global",
    "variable_site_fraction_global",
    "gap_fraction_clade",
    "gc_mean_clade",
    "gc_std_clade",
    "mean_pairwise_pdist_clade",
]
def train_model(df: pd.DataFrame):
    X = df[FEATURE_COLUMNS].copy()
    y = df["target_bootstrap"].copy()
    groups = df["dataset_id"].copy()
    splitter = GroupShuffleSplit(n_splits=1,
test_size=0.25, random_state=42)
    train_idx, test_idx = next(splitter.split(X, y,
groups=groups))
    X_train, X_test = X.iloc[train_idx],
X.iloc[test_idx]
    y_train, y_test = y.iloc[train_idx],
y.iloc[test_idx]
    g_train, g_test = groups.iloc[train_idx],
groups.iloc[test_idx]
    model = Pipeline([
        ("imputer",
SimpleImputer(strategy="median")),
        ("rf",
RandomForestRegressor(n_estimators=300,
random_state=42, n_jobs=-1)),
    ])
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    metrics = {
        "n_rows_total": int(len(df)),
        "n_rows_train": int(len(X_train)),
        "n_rows_test": int(len(X_test)),
        "n_datasets_total": int(groups.nunique()),
        "n_datasets_train": int(g_train.nunique()),
        "n_datasets_test": int(g_test.nunique()),
        "mae": float(mean_absolute_error(y_test,
pred)),
        "rmse":
float(math.sqrt(mean_squared_error(y_test, pred))),
        "r2": float(r2_score(y_test, pred)),
        "feature_columns": FEATURE_COLUMNS,
        "train_datasets":
sorted(g_train.unique().tolist()),
        "test_datasets":
sorted(g_test.unique().tolist()),
    }
    rf = model.named_steps["rf"]
    feature_importance = pd.DataFrame({
        "feature": FEATURE_COLUMNS,
        "importance": rf.feature_importances_,
    }).sort_values("importance", ascending=False)
    pred_df = X_test.copy()
    pred_df["y_true"] = y_test.values
    pred_df["y_pred"] = pred
    pred_df["dataset_id"] = g_test.values

```

```

        return model, metrics, feature_importance,
pred_df
def main():
    project_root = Path.cwd()
    datasets_root = project_root / "datasets"
    output_dir = project_root / "ml_outputs"
    output_dir.mkdir(exist_ok=True)
    print(f"[INFO] datasets root: {datasets_root}")
    print(f"[INFO] output dir: {output_dir}")
    df, audit = build_feature_table(datasets_root)
    df.to_csv(output_dir / "node_dataset.csv",
index=False, encoding="utf-8-sig")
    pd.DataFrame(audit).to_csv(output_dir /
"dataset_audit.csv", index=False, encoding="utf-8-
sig")
    model, metrics, feature_importance, pred_df =
train_model(df)
    joblib.dump(model, output_dir / "model_v1.pkl")
    feature_importance.to_csv(output_dir /
"feature_importance.csv", index=False,
encoding="utf-8-sig")
    pred_df.to_csv(output_dir /
"test_predictions.csv", index=False, encoding="utf-8-
sig")
    with open(output_dir / "model_metadata.json",
"w", encoding="utf-8") as f:
        json.dump(metrics, f, ensure_ascii=False,
indent=2)
    print("\n=== TRAIN DONE ===")
    print(json.dumps(metrics, ensure_ascii=False,
indent=2))
    print("\nTop feature importance:")

print(feature_importance.head(10).to_string(index=
False))
if __name__ == "__main__":
    main()
from __future__ import annotations
import io
import json
import math
import re
from collections import defaultdict
from pathlib import Path
from typing import Dict, List
import joblib
import numpy as np
import pandas as pd
from Bio import Phylo
from sklearn.ensemble import RandomForestRegressor
from sklearn.impute import SimpleImputer
from sklearn.metrics import mean_absolute_error,
r2_score, mean_squared_error
from sklearn.model_selection import
GroupShuffleSplit
from sklearn.pipeline import Pipeline
def parse_fasta(text: str) -> Dict[str, str]:
    seqs = {}
    current = None
    for raw_line in text.splitlines():
        line = raw_line.strip()
        if not line:
            continue
        if line.startswith(">"):
            current = line[1:].strip()
            seqs[current] = []
        else:
            if current is None:
                continue
            seqs[current].append(line.replace(" ",
""))
    return {k: "".join(v) for k, v in seqs.items()}
def _extract_nexus_taxa(text: str) -> List[str]:
    m = re.search(r"BEGIN\s+TAXA;(.+?)END;", text,
flags=re.IGNORECASE | re.DOTALL)
    if not m:

```

```

        return []
        block = m.group(1)
        tm = re.search(r"TAXLABELS(.+?);", block,
flags=re.IGNORECASE | re.DOTALL)
        if not tm:
            return []
        tokens = re.split(r"\s+", tm.group(1).strip())
        return [t.strip() for t in tokens if t.strip()]
def _parse_nexus_matrix_block(block: str,
known_taxa: List[str]) -> Dict[str, str]:
    mm = re.search(r"\bMATRIX\b(.+?);", block,
flags=re.IGNORECASE | re.DOTALL)
    if not mm:
        return {}
    matrix_text = mm.group(1)
    seqs = defaultdict(list)
    known_taxa_sorted = sorted(known_taxa, key=len,
reverse=True)
    for raw_line in matrix_text.splitlines():
        stripped = raw_line.strip()
        if not stripped or
stripped.startswith("["):
            continue
        matched = False
        for taxon in known_taxa_sorted:
            if stripped.startswith(taxon):
                rest =
stripped[len(taxon):].strip().replace(" ", "")
                if rest:
                    seqs[taxon].append(rest)
                    matched = True
                    break
        if not matched:
            parts = stripped.split()
            if len(parts) >= 2:
                taxon = parts[0]
                seq = "".join(parts[1:]).replace("
", "")
                seqs[taxon].append(seq)
    return {k: "".join(v) for k, v in seqs.items()}
def parse_nexus_alignment(text: str) -> Dict[str,
str]:
    known_taxa = _extract_nexus_taxa(text)
    char_blocks =
re.findall(r"BEGIN\s+CHARACTERS;(.*?)END;", text,
flags=re.IGNORECASE | re.DOTALL)
    if not char_blocks:
        raise ValueError("В файле не найдено BEGIN
CHARACTERS.")
    merged = defaultdict(list)
    for block in char_blocks:
        part = _parse_nexus_matrix_block(block,
known_taxa)
        for taxon, seq in part.items():
            merged[taxon].append(seq)
    if not merged:
        raise ValueError("Не удалось извлечь
последовательности из MATRIX.")
    return {k: "".join(v) for k, v in
merged.items()}
def load_alignment_file(path: str | Path) ->
Dict[str, str]:
    path = Path(path)
    text = path.read_text(encoding="utf-8",
errors="ignore")
    upper = text.upper()
    if "#NEXUS" in upper or "BEGIN CHARACTERS;" in
upper:
        return parse_nexus_alignment(text)
    if text.lstrip().startswith(">"):
        return parse_fasta(text)
    raise ValueError(f"Неподдерживаемый формат
alignment: {path}")
def clean_newick(tree_str: str) -> str:
    return re.sub(r"[.?!\\]", "",
tree_str.strip()).strip()

```

```

def extract_newick_from_text(text: str) -> str:
    text = text.strip()
    tree_match =
re.search(r"\bTREE\b\s+[\^=]\s*(\(.+?\);)", text,
flags=re.IGNORECASE | re.DOTALL)
    if tree_match:
        return clean_newick(tree_match.group(1))
    first_paren = text.find("(")
    last_semicolon = text.rfind(";")
    if first_paren != -1 and last_semicolon != -1
and last_semicolon > first_paren:
        return
clean_newick(text[first_paren:last_semicolon + 1])
    raise ValueError("Не удалось извлечь Newick из
файла дерева.")
def load_tree_file(path: str | Path):
    path = Path(path)
    text = path.read_text(encoding="utf-8",
errors="ignore")
    newick = extract_newick_from_text(text)
    tree = Phylo.read(io.StringIO(newick),
"newick")
    return tree, newick
VALID_DNA = set("ACGT")
DNA_PLUS_GAP = set("ACGT-?")
def sanitize_seq(seq: str) -> str:
    return "".join(ch.upper() for ch in seq if
ch.upper() in DNA_PLUS_GAP)
def p_distance(seq1: str, seq2: str) -> float:
    mismatches = 0
    valid = 0
    for a, b in zip(seq1, seq2):
        if a in {"-", "?"} or b in {"-", "?"}:
            continue
        if a not in VALID_DNA or b not in VALID_DNA:
            continue
        valid += 1
        if a != b:
            mismatches += 1
    return np.nan if valid == 0 else mismatches /
valid
def seq_gap_fraction(seq: str) -> float:
    return np.nan if not seq else sum(ch in {"-",
"?"} for ch in seq) / len(seq)
def seq_gc_fraction(seq: str) -> float:
    letters = [ch for ch in seq if ch in VALID_DNA]
    return np.nan if not letters else sum(ch in
{"G", "C"} for ch in letters) / len(letters)
def alignment_stats(seqs: Dict[str, str]) ->
Dict[str, float]:
    items = [sanitize_seq(s) for s in seqs.values()]
    if not items:
        return {
            "taxa_count": np.nan,
            "alignment_length": np.nan,
            "gap_fraction_global": np.nan,
            "gc_mean_global": np.nan,
            "gc_std_global": np.nan,
            "variable_site_fraction_global":
np.nan,
        }
    n = len(items)
    L = max(len(s) for s in items)
    items = [s.ljust(L, "-") for s in items]
    gap_fraction_global =
np.mean([seq_gap_fraction(s) for s in items])
    gc_values = [seq_gc_fraction(s) for s in items]
    gc_mean_global = float(np.nanmean(gc_values))
    gc_std_global = float(np.nanstd(gc_values))
    variable_sites = 0
    informative_positions = 0
    for i in range(L):
        col = [s[i] for s in items if s[i] in
VALID_DNA]
        if len(col) < 2:
            continue

```



```

        informative_positions += 1
        if len(set(col)) > 1:
            variable_sites += 1
        variable_site_fraction = variable_sites /
informative_positions if informative_positions else
np.nan
    return {
        "taxa_count": float(n),
        "alignment_length": float(L),
        "gap_fraction_global":
float(gap_fraction_global),
        "gc_mean_global": gc_mean_global,
        "gc_std_global": gc_std_global,
        "variable_site_fraction_global":
float(variable_site_fraction),
    }
def get_leaf_names(clade) -> List[str]:
    return [leaf.name for leaf in
clade.get_terminals() if leaf.name]
def mean_pairwise_distance(names: List[str], seqs:
Dict[str, str]) -> float:
    if len(names) < 2:
        return 0.0
    vals = []
    for i in range(len(names)):
        for j in range(i + 1, len(names)):
            s1 = sanitize_seq(seqs.get(names[i],
""))
            s2 = sanitize_seq(seqs.get(names[j],
""))
            if not s1 or not s2:
                continue
            d = p_distance(s1, s2)
            if not np.isnan(d):
                vals.append(d)
    return np.nan if not vals else
float(np.mean(vals))
def clade_alignment_stats(names: List[str], seqs:
Dict[str, str]) -> Dict[str, float]:
    sub = {k: seqs[k] for k in names if k in seqs}
    if not sub:
        return {
            "gap_fraction_clade": np.nan,
            "gc_mean_clade": np.nan,
            "gc_std_clade": np.nan,
            "mean_pairwise_pdist_clade": np.nan,
        }
    gap_fraction_clade =
float(np.nanmean([seq_gap_fraction(sanitize_seq(v)
) for v in sub.values()])))
    gc_vals = [seq_gc_fraction(sanitize_seq(v)) for
v in sub.values()]
    gc_mean_clade = float(np.nanmean(gc_vals))
    gc_std_clade = float(np.nanstd(gc_vals))
    mpd = mean_pairwise_distance(list(sub.keys()),
sub)
    return {
        "gap_fraction_clade": gap_fraction_clade,
        "gc_mean_clade": gc_mean_clade,
        "gc_std_clade": gc_std_clade,
        "mean_pairwise_pdist_clade": mpd,
    }
def subtree_balance(clade) -> float:
    children = clade.clades
    if len(children) < 2:
        return 0.0
    sizes = [len(ch.get_terminals()) for ch in
children]
    total = sum(sizes)
    return 0.0 if total == 0 else abs(sizes[0] -
sizes[1]) / total
def extract_node_features(tree, seqs: Dict[str,
str], dataset_id: str) -> pd.DataFrame:
    global_stats = alignment_stats(seqs)
    depths = tree.depths()
    rows = []

```

```

        node_idx = 0
        for clade in tree.find_clades(order="level"):
            if clade.is_terminal():
                continue
            conf = clade.confidence
            if conf is None:
                continue
            bootstrap = float(conf)
            if bootstrap <= 1.0:
                bootstrap *= 100.0
            branch_length =
max(float(clade.branch_length or 0.0), 0.0)
            leaf_names = get_leaf_names(clade)
            n_leaves = len(leaf_names)
            clade_stats =
clade_alignment_stats(leaf_names, seqs)
            child_branch_lengths =
[max(float(ch.branch_length or 0.0), 0.0) for ch in
clade.clades]
            rows.append({
                "dataset_id": dataset_id,
                "node_id":
f"{dataset_id}_{node_idx}",
                "target_bootstrap": bootstrap,
                "branch_length": branch_length,
                "depth": float(depths.get(clade, 0.0)),
                "n_leaves_subtree": float(n_leaves),
                "subtree_fraction": float(n_leaves /
global_stats["taxa_count"]) if
global_stats["taxa_count"] else np.nan,
                "subtree_balance":
float(subtree_balance(clade)),
                "mean_child_branch_length":
float(np.mean(child_branch_lengths)) if
child_branch_lengths else 0.0,
                "std_child_branch_length":
float(np.std(child_branch_lengths)) if
child_branch_lengths else 0.0,
                **global_stats,
                **clade_stats,
            })
            node_idx += 1
        return pd.DataFrame(rows)
ALIGNMENT_EXTS = {".nex", ".nexus", ".nexorg",
".fasta", ".fa", ".fas", ".txt"}
TREE_HINTS = ("tree", "bootstrap", "nwk", "newick",
"con_", "majrule")
def detect_alignment_file(ds_path: Path) -> Path |
None:
    candidates = [f for f in ds_path.iterdir() if
f.is_file() and f.suffix.lower() in ALIGNMENT_EXTS]
    candidates = sorted(candidates, key=lambda p:
(p.suffix.lower() not in {".nex", ".nexus",
".nexorg"}, p.name.lower()))
    return candidates[0] if candidates else None
def detect_tree_file(ds_path: Path, alignment_file:
Path | None) -> Path | None:
    files = [f for f in ds_path.iterdir() if
f.is_file()]
    if alignment_file is not None:
        files = [f for f in files if f.resolve() !=
alignment_file.resolve()]
    scored = []
    for f in files:
        name_l = f.name.lower()
        score = 0
        if any(h in name_l for h in TREE_HINTS):
            score += 3
        if f.suffix.lower() in {".nwk", ".newick",
".tree"}:
            score += 3
        if f.suffix == "":
            score += 1
        scored.append((score, f))
    scored.sort(key=lambda x:
(-x[0],
x[1].name.lower()))

```

```

for _, f in scored:
    try:
        load_tree_file(f)
        return f
    except Exception:
        continue
return None
def build_feature_table(datasets_root: str | Path):
    datasets_root = Path(datasets_root)
    all_frames = []
    audit = []
    for ds_path in sorted(datasets_root.iterdir()):
        if not ds_path.is_dir():
            continue
        dataset_id = ds_path.name
        alignment_file =
        detect_alignment_file(ds_path)
        tree_file = detect_tree_file(ds_path,
        alignment_file)
        status = {
            "dataset_id": dataset_id,
            "alignment_file": str(alignment_file)
        if alignment_file else None,
            "tree_file": str(tree_file) if
        tree_file else None,
            "ok": False,
            "error": None,
        }
        try:
            if alignment_file is None:
                raise FileNotFoundError("Не найден
        alignment-файл")
            if tree_file is None:
                raise FileNotFoundError("Не найден
        tree-файл")
            seqs =
            load_alignment_file(alignment_file)
            tree, _ = load_tree_file(tree_file)
            df = extract_node_features(tree, seqs,
        dataset_id)
            if df.empty:
                raise ValueError("Не извлечено ни
        одного внутреннего узла с bootstrap")
            all_frames.append(df)
            status["ok"] = True
            status["n_rows"] = int(len(df))
        except Exception as e:
            status["error"] = str(e)
            audit.append(status)
        if not all_frames:
            audit_df = pd.DataFrame(audit)
            raise RuntimeError("Не удалось собрать ни
        одной строки датасета. Проверь dataset_audit:\n" +
        audit_df.to_string(index=False))
        return pd.concat(all_frames,
        ignore_index=True), audit
FEATURE_COLUMNS = [
    "branch_length",
    "depth",
    "n_leaves_subtree",
    "subtree_fraction",
    "subtree_balance",
    "mean_child_branch_length",
    "std_child_branch_length",
    "taxa_count",
    "alignment_length",
    "gap_fraction_global",
    "gc_mean_global",
    "gc_std_global",
    "variable_site_fraction_global",
    "gap_fraction_clade",
    "gc_mean_clade",
    "gc_std_clade",
    "mean_pairwise_pdist_clade",
]
def train_model(df: pd.DataFrame):
    df = df.copy()
    df = df.dropna(subset=["target_bootstrap"])
    df = df.fillna(0)
    X = df[FEATURE_COLUMNS].copy()
    y = df["target_bootstrap"].copy()
    groups = df["dataset_id"].copy()
    splitter = GroupShuffleSplit(n_splits=1,
        test_size=0.25, random_state=42)
    train_idx, test_idx = next(splitter.split(X, y,
        groups=groups))
    X_train, X_test = X.iloc[train_idx],
    X.iloc[test_idx]
    y_train, y_test = y.iloc[train_idx],
    y.iloc[test_idx]
    g_train, g_test = groups.iloc[train_idx],
    groups.iloc[test_idx]
    model = Pipeline([
        ("imputer",
        SimpleImputer(strategy="median")),
        ("rf",
        RandomForestRegressor(n_estimators=300,
        random_state=42, n_jobs=-1)),
    ])
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    metrics = {
        "n_rows_total": int(len(df)),
        "n_rows_train": int(len(X_train)),
        "n_rows_test": int(len(X_test)),
        "n_datasets_total": int(groups.nunique()),
        "n_datasets_train": int(g_train.nunique()),
        "n_datasets_test": int(g_test.nunique()),
        "mae": float(mean_absolute_error(y_test,
        pred)),
        "rmse":
        float(math.sqrt(mean_squared_error(y_test, pred))),
        "r2": float(r2_score(y_test, pred)),
        "feature_columns": FEATURE_COLUMNS,
        "train_datasets":
        sorted(g_train.unique().tolist()),
        "test_datasets":
        sorted(g_test.unique().tolist()),
    }
    rf = model.named_steps["rf"]
    feature_importance = pd.DataFrame({
        "feature": FEATURE_COLUMNS,
        "importance": rf.feature_importances_,
    }).sort_values("importance", ascending=False)
    pred_df = pd.DataFrame(index=X_test.index)
    pred_df["y_true"] = y_test
    pred_df["y_pred"] = pred
    pred_df["dataset_id"] = df.loc[X_test.index,
    "dataset_id"]
    return model, metrics, feature_importance,
    pred_df
def main():
    project_root = Path.cwd()
    datasets_root = project_root /
    "datasets_iqtree"
    output_dir = project_root / "ml_outputs"
    output_dir.mkdir(exist_ok=True)
    print(f"[INFO] datasets root: {datasets_root}")
    print(f"[INFO] output dir: {output_dir}")
    df, audit = build_feature_table(datasets_root)
    df.to_csv(output_dir / "node_dataset.csv",
    index=False, encoding="utf-8-sig")
    pd.DataFrame(audit).to_csv(output_dir /
    "dataset_audit.csv", index=False, encoding="utf-8-
    sig")
    model, metrics, feature_importance, pred_df =
    train_model(df)
    joblib.dump(model, output_dir / "model_v1.pkl")
    feature_importance.to_csv(output_dir /
    "feature_importance.csv", index=False,
    encoding="utf-8-sig")

```

```

    pred_df.to_csv(output_dir /
"test_predictions.csv", index=False, encoding="utf-
8-sig")
    with open(output_dir / "model_metadata.json",
"w", encoding="utf-8") as f:
        json.dump(metrics, f, ensure_ascii=False,
indent=2)
    print("\n=== TRAIN DONE ===")
    print(json.dumps(metrics, ensure_ascii=False,
indent=2))
    print("\nTop feature importance:")

print(feature_importance.head(10).to_string(index=
False))
if __name__ == "__main__":
    main()
from __future__ import annotations
import io
import json
import math
import re
import warnings
from collections import defaultdict
from pathlib import Path
from typing import Dict, List
import joblib
import numpy as np
import pandas as pd
from Bio import Phylo
from sklearn.ensemble import
GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error,
mean_squared_error, r2_score
from sklearn.model_selection import GroupKFold
warnings.filterwarnings("ignore")
def parse_fasta(text: str) -> Dict[str, str]:
    seqs = {}
    current = None
    for raw_line in text.splitlines():
        line = raw_line.strip()
        if not line: continue
        if line.startswith(">"):
            current = line[1:].strip()
            seqs[current] = []
        else:
            if current is None: continue
            seqs[current].append(line.replace(" ",
""))
    return {k: "".join(v) for k, v in seqs.items()}
def _extract_nexus_taxa(text: str) -> List[str]:
    m = re.search(r"BEGIN\s+TAXA;(.*?)END;", text,
flags=re.IGNORECASE | re.DOTALL)
    if not m: return []
    block = m.group(1)
    tm = re.search(r"TAXLABELS(.*?)", block,
flags=re.IGNORECASE | re.DOTALL)
    if not tm: return []
    return [t.strip() for t in re.split(r"\s+",
tm.group(1).strip()) if t.strip()]
def _parse_nexus_matrix_block(block: str,
known_taxa: List[str]) -> Dict[str, str]:
    mm = re.search(r"\bMATRIX\b(.*?)", block,
flags=re.IGNORECASE | re.DOTALL)
    if not mm: return {}
    seqs = defaultdict(list)
    known_taxa_sorted = sorted(known_taxa, key=len,
reverse=True)
    for raw_line in mm.group(1).splitlines():
        stripped = raw_line.strip()
        if not stripped or
stripped.startswith("["): continue
        matched = False
        for taxon in known_taxa_sorted:
            if stripped.startswith(taxon):
                rest =
stripped[len(taxon):].strip().replace(" ", "")
                if rest: seqs[taxon].append(rest)
                matched = True
                break
        if not matched:
            parts = stripped.split()
            if len(parts) >= 2:
                seqs[parts[0]].append("".join(parts[1:]).replace("
", ""))
    return {k: "".join(v) for k, v in seqs.items()}
def parse_nexus_alignment(text: str) -> Dict[str,
str]:
    known_taxa = _extract_nexus_taxa(text)
    char_blocks =
re.findall(r"BEGIN\s+CHARACTERS;(.*?)END;", text,
flags=re.IGNORECASE | re.DOTALL)
    if not char_blocks: raise ValueError("В файле
не найдено BEGIN CHARACTERS.")
    merged = defaultdict(list)
    for block in char_blocks:
        part = _parse_nexus_matrix_block(block,
known_taxa)
        for taxon, seq in part.items():
            merged[taxon].append(seq)
    if not merged: raise ValueError("Не удалось
извлечь последовательности из MATRIX.")
    return {k: "".join(v) for k, v in
merged.items()}
def load_alignment_file(path: str | Path) ->
Dict[str, str]:
    path = Path(path)
    text = path.read_text(encoding="utf-8",
errors="ignore")
    upper = text.upper()
    if "#NEXUS" in upper or "BEGIN CHARACTERS;" in
upper: return parse_nexus_alignment(text)
    if text.lstrip().startswith(">"): return
parse_fasta(text)
    raise ValueError(f"Неподдерживаемый формат
alignment: {path}")
def clean_newick(tree_str: str) -> str:
    return re.sub(r"\[.*?\]", "",
tree_str.strip()).strip()
def extract_newick_from_text(text: str) -> str:
    text = text.strip()
    tree_match =
re.search(r"\bTREE\b\s+([^\s]*\s*(\s*(\s*(\s*?);)", text,
flags=re.IGNORECASE | re.DOTALL)
    if tree_match: return
clean_newick(tree_match.group(1))
    first_paren = text.find("(")
    last_semicolon = text.rfind(";")
    if first_paren != -1 and last_semicolon != -1
and last_semicolon > first_paren:
        return
clean_newick(text[first_paren:last_semicolon + 1])
    raise ValueError("Не удалось извлечь Newick из
файла дерева.")
def load_tree_file(path: str | Path):
    path = Path(path)
    text = path.read_text(encoding="utf-8",
errors="ignore")
    newick = extract_newick_from_text(text)
    tree = Phylo.read(io.StringIO(newick),
"newick")
    return tree, newick
VALID_DNA = set("ACGT")
DNA_PLUS_GAP = set("ACGT-?")
def sanitize_seq(seq: str) -> str:
    return "".join(ch.upper() for ch in seq if
ch.upper() in DNA_PLUS_GAP)
def p_distance(seq1: str, seq2: str) -> float:
    mismatches = valid = 0
    for a, b in zip(seq1, seq2):
        if a in {"-", "?"} or b in {"-", "?"}:
            continue

```

```

        if a not in VALID_DNA or b not in VALID_DNA:
            continue
        valid += 1
        if a != b: mismatches += 1
        return np.nan if valid == 0 else mismatches / valid
def seq_gap_fraction(seq: str) -> float:
    return np.nan if not seq else sum(ch in {"-", "?"} for ch in seq) / len(seq)
def seq_gc_fraction(seq: str) -> float:
    letters = [ch for ch in seq if ch in VALID_DNA]
    return np.nan if not letters else sum(ch in {"G", "C"} for ch in letters) / len(letters)
def alignment_stats(seqs: Dict[str, str]) -> Dict[str, float]:
    items = [sanitize_seq(s) for s in seqs.values()]
    if not items:
        return {"taxa_count": np.nan,
                "alignment_length": np.nan, "gap_fraction_global": np.nan,
                "gc_mean_global": np.nan,
                "gc_std_global": np.nan,
                "variable_site_fraction_global": np.nan}
    n = len(items)
    L = max(len(s) for s in items)
    items = [s.ljust(L, "-") for s in items]
    gap_fraction_global = np.mean([seq_gap_fraction(s) for s in items])
    gc_values = [seq_gc_fraction(s) for s in items]
    variable_sites = informative_positions = 0
    for i in range(L):
        col = [s[i] for s in items if s[i] in VALID_DNA]
        if len(col) < 2: continue
        informative_positions += 1
        if len(set(col)) > 1: variable_sites += 1
    return {"taxa_count": float(n), "alignment_length": float(L),
            "gap_fraction_global": float(gap_fraction_global),
            "gc_mean_global": float(np.nanmean(gc_values)),
            "gc_std_global": float(np.nanstd(gc_values)),
            "variable_site_fraction_global": float(variable_sites / informative_positions if informative_positions else np.nan)}
def mean_pairwise_distance(names: List[str], seqs: Dict[str, str]) -> float:
    if len(names) < 2: return 0.0
    vals = []
    for i in range(len(names)):
        for j in range(i + 1, len(names)):
            s1, s2 = sanitize_seq(seqs.get(names[i], "")), sanitize_seq(seqs.get(names[j], ""))
            if not s1 or not s2: continue
            d = p_distance(s1, s2)
            if not np.isnan(d): vals.append(d)
    return np.nan if not vals else float(np.mean(vals))
def clade_alignment_stats(names: List[str], seqs: Dict[str, str]) -> Dict[str, float]:
    sub = {k: seqs[k] for k in names if k in seqs}
    if not sub: return {"gap_fraction_clade": np.nan, "gc_mean_clade": np.nan, "gc_std_clade": np.nan, "mean_pairwise_pdist_clade": np.nan}
    return {"gap_fraction_clade": float(np.nanmean([seq_gap_fraction(sanitize_seq(v)) for v in sub.values()])),
            "gc_mean_clade": float(np.nanmean([seq_gc_fraction(sanitize_seq(v)) for v in sub.values()])),

```

```

            "gc_std_clade": float(np.nanstd([seq_gc_fraction(sanitize_seq(v)) for v in sub.values()])),
            "mean_pairwise_pdist_clade": mean_pairwise_distance(list(sub.keys()), sub)}
def subtree_balance(clade) -> float:
    children = clade.clades
    if len(children) < 2: return 0.0
    sizes = np.array([len(ch.get_terminals()) for ch in children])
    total = sizes.sum()
    if total == 0: return 0.0
    return float(np.std(sizes) / (total / len(children)))
def extract_node_features(tree, seqs: Dict[str, str], dataset_id: str) -> pd.DataFrame:
    global_stats = alignment_stats(seqs)
    depths = tree.depths()
    max_depth = max(depths.values()) if depths else 1.0
    rows = []
    node_idx = 0
    for clade in tree.find_clades(order="level"):
        if clade.is_terminal(): continue
        conf = clade.confidence
        if conf is None: continue
        bootstrap = float(conf)
        if bootstrap <= 1.0: bootstrap *= 100.0
        branch_length = max(float(clade.branch_length or 0.0), 0.0)
        depth = float(depths.get(clade, 0.0))
        n_leaves = len([leaf.name for leaf in clade.get_terminals() if leaf.name])
        clade_stats = clade_alignment_stats([leaf.name for leaf in clade.get_terminals() if leaf.name], seqs)
        child_bl = [max(float(ch.branch_length or 0.0), 0.0) for ch in clade.clades]
        rows.append({"dataset_id": dataset_id, "node_id": f"{dataset_id}_{node_idx}",
                    "target_bootstrap": bootstrap, "branch_length": branch_length, "depth": depth,
                    "subtree_fraction": float(n_leaves / global_stats["taxa_count"]) if global_stats["taxa_count"] else np.nan,
                    "subtree_balance": subtree_balance(clade), "mean_child_branch_length": float(np.mean(child_bl)) if child_bl else 0.0,
                    "std_child_branch_length": float(np.std(child_bl)) if child_bl else 0.0,
                    "n_children": float(len(clade.clades)), "clade_size_log": float(np.log1p(n_leaves)),
                    "relative_branch_length": float(branch_length / max_depth) if max_depth > 0 else 0.0,
                    "depth_ratio": float(depth / max_depth) if max_depth > 0 else 0.0,
                    **global_stats, **clade_stats})
        node_idx += 1
    return pd.DataFrame(rows)
FEATURE_COLUMNS = ["branch_length", "depth", "subtree_fraction", "subtree_balance", "mean_child_branch_length", "std_child_branch_length", "n_children", "clade_size_log", "relative_branch_length", "depth_ratio", "taxa_count", "alignment_length", "gap_fraction_global",

```

```

"gc_mean_global", "gc_std_global",
"variable_site_fraction_global",
"gap_fraction_clade", "gc_mean_clade",
"gc_std_clade", "mean_pairwise_pdist_clade"
]
def train_model(df: pd.DataFrame):
    X = df[FEATURE_COLUMNS].copy()
    y = df["target_bootstrap"].copy()
    groups = df["dataset_id"].copy()
    X = X.fillna(X.median())
    gkf = GroupKFold(n_splits=5)
    oof_pred = np.zeros(len(y))
    fold_metrics = []
    model_template = GradientBoostingRegressor(
        n_estimators=500, learning_rate=0.05,
        max_depth=4, subsample=0.8,
        min_samples_leaf=5, random_state=42
    )
    for fold, (train_idx, test_idx) in
        enumerate(gkf.split(X, y, groups)):
        X_tr, X_te = X.iloc[train_idx],
        X.iloc[test_idx]
        y_tr, y_te = y.iloc[train_idx],
        y.iloc[test_idx]
        model = model_template.fit(X_tr, y_tr)
        preds = model.predict(X_te)
        oof_pred[test_idx] = np.clip(preds, 0.0,
        100.0)
        fold_metrics.append({
            "fold": fold+1,
            "mae": mean_absolute_error(y_te,
            oof_pred[test_idx]),
            "rmse": np.sqrt(mean_squared_error(y_te,
            oof_pred[test_idx])),
            "r2": r2_score(y_te,
            oof_pred[test_idx])
        })
        print(f"Fold {fold+1}: MAE={fold_metrics[-1]['mae']:.3f},
        RMSE={fold_metrics[-1]['rmse']:.3f}, R2={fold_metrics[-1]['r2']:.3f}")
        agg = pd.DataFrame(fold_metrics).mean()
        final_model = model_template.fit(X, y)
        importance = final_model.feature_importances_
        feat_imp = pd.DataFrame({"feature":
        FEATURE_COLUMNS, "importance":
        importance}).sort_values("importance",
        ascending=False)
        pred_df = X.copy()
        pred_df["y_true"] = y.values
        pred_df["y_pred"] = oof_pred
        pred_df["dataset_id"] = groups.values
        metrics = {
            "n_rows_total": int(len(df)),
            "n_datasets_total": int(groups.nunique()),
            "cv_mae": float(agg["mae"]),
            "cv_rmse": float(agg["rmse"]),
            "cv_r2": float(agg["r2"]),
            "feature_columns": FEATURE_COLUMNS,
            "fold_metrics": [m for m in fold_metrics]
        }
        return final_model, metrics, feat_imp, pred_df
ALIGNMENT_EXTS = {".nex", ".nexus", ".nexorg",
".fasta", ".fa", ".fas", ".txt"}
TREE_HINTS = ("tree", "bootstrap", "nwk", "newick",
"con", "majorrule")
def detect_alignment_file(ds_path: Path) -> Path |
None:
    candidates = [f for f in ds_path.iterdir() if
f.is_file() and f.suffix.lower() in ALIGNMENT_EXTS]
    candidates.sort(key=lambda p: (p.suffix.lower()
not in {".nex", ".nexus", ".nexorg"},
p.name.lower()))
    return candidates[0] if candidates else None
def detect_tree_file(ds_path: Path, alignment_file:
Path | None) -> Path | None:

```

```

files = [f for f in ds_path.iterdir() if
f.is_file()]
    if alignment_file is not None: files = [f for f
in files if f.resolve() != alignment_file.resolve()]
    scored = [(sum(3 for h in TREE_HINTS if h in
f.name.lower()) + (3 if f.suffix.lower() in
{"nwk", "newick", "tree"} else 0), f) for f in
files]
    scored.sort(key=lambda x: (-x[0],
x[1].name.lower()))
    for _, f in scored:
        try: load_tree_file(f); return f
        except: continue
    return None
def build_feature_table(datasets_root: str | Path):
    datasets_root = Path(datasets_root)
    all_frames, audit = [], []
    for ds_path in sorted(datasets_root.iterdir()):
        if not ds_path.is_dir(): continue
        dataset_id = ds_path.name
        alignment_file =
detect_alignment_file(ds_path)
        tree_file = detect_tree_file(ds_path,
alignment_file)
        status = {"dataset_id": dataset_id,
"alignment_file": str(alignment_file), "tree_file":
str(tree_file), "ok": False, "error": None}
        try:
            if not alignment_file: raise
FileNotFoundError("Не найден alignment-файл")
            if not tree_file: raise
FileNotFoundError("Не найден tree-файл")
            seqs =
load_alignment_file(alignment_file)
            tree, _ = load_tree_file(tree_file)
            df = extract_node_features(tree, seqs,
dataset_id)
            if df.empty: raise ValueError("Нет
узлов с bootstrap")
            all_frames.append(df)
            status["ok"] = True; status["n_rows"] =
len(df)
        except Exception as e: status["error"] =
str(e)
        audit.append(status)
        if not all_frames: raise RuntimeError("Не
собрано ни одной строки. Проверь audit.")
        return pd.concat(all_frames,
ignore_index=True), audit
def main():
    project_root = Path.cwd()
    datasets_root = project_root / "datasets"
    output_dir = project_root / "ml_outputss"
    output_dir.mkdir(exist_ok=True)
    print(f"[INFO] datasets root: {datasets_root}")
    df, audit = build_feature_table(datasets_root)
    df.to_csv(output_dir / "node_dataset.csv",
index=False, encoding="utf-8-sig")
    pd.DataFrame(audit).to_csv(output_dir /
"dataset_audit.csv", index=False, encoding="utf-8-
sig")
    print("\n♦ Заняк GroupKFold тренировки...")
    model, metrics, feat_imp, pred_df =
train_model(df)
    pred_df.to_csv(output_dir /
"test_predictions.csv", index=False, encoding="utf-
8-sig")
    joblib.dump(model, output_dir /
"model_v3_stable.pkl")
    feat_imp.to_csv(output_dir /
"feature_importance_v3.csv", index=False,
encoding="utf-8-sig")
    with open(output_dir /
"model_metadata_v3.json", "w", encoding="utf-8") as
f:

```

```

        json.dump(metrics, f, ensure_ascii=False,
indent=2)
    print("\n=== CV RESULTS ===")
    print(f"R2 : {metrics['cv_r2']:.4f}")
    print(f"MAE : {metrics['cv_mae']:.4f}")
    print(f"RMSE : {metrics['cv_rmse']:.4f}")
    print("\nTop features:")

print(feat_imp.head(10).to_string(index=False))
print(f"\nФайл предсказаний сохранён в:
{output_dir / 'test_predictions.csv'}")
if __name__ == "__main__":
    main()
from __future__ import annotations
import io
import json
import math
import re
from collections import defaultdict
from pathlib import Path
from typing import Dict, List
import joblib
import numpy as np
import pandas as pd
from Bio import Phylo
from sklearn.ensemble import RandomForestRegressor
from sklearn.impute import SimpleImputer
from sklearn.metrics import mean_absolute_error,
r2_score, mean_squared_error
from sklearn.model_selection import GroupKFold
from sklearn.pipeline import Pipeline
import joblib
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestRegressor
from sklearn.impute import SimpleImputer
from sklearn.metrics import mean_absolute_error,
r2_score, mean_squared_error
from sklearn.model_selection import GroupKFold,
RandomizedSearchCV
from sklearn.pipeline import Pipeline
from scipy.stats import randint
import math
def parse_fasta(text: str) -> Dict[str, str]:
    seqs = {}
    current = None
    for raw_line in text.splitlines():
        line = raw_line.strip()
        if not line:
            continue
        if line.startswith(">"):
            current = line[1:].strip()
            seqs[current] = []
        else:
            if current is None:
                continue
            seqs[current].append(line.replace(" ",
""))
    return {k: "".join(v) for k, v in seqs.items()}
def _extract_nexus_taxa(text: str) -> List[str]:
    m = re.search(r"BEGIN\s+TAXA;(.*?)END;", text,
flags=re.IGNORECASE | re.DOTALL)
    if not m:
        return []
    block = m.group(1)
    tm = re.search(r"TAXLABELS(.*?)", block,
flags=re.IGNORECASE | re.DOTALL)
    if not tm:
        return []
    tokens = re.split(r"\s+", tm.group(1).strip())
    return [t.strip() for t in tokens if t.strip()]
def _parse_nexus_matrix_block(block: str,
known_taxa: List[str]) -> Dict[str, str]:
    mm = re.search(r"\bMATRIX\b(.*?)", block,
flags=re.IGNORECASE | re.DOTALL)
    if not mm:
        return {}

```

```

        matrix_text = mm.group(1)
        seqs = defaultdict(list)
        known_taxa_sorted = sorted(known_taxa, key=len,
reverse=True)
        for raw_line in matrix_text.splitlines():
            stripped = raw_line.strip()
            if not stripped or
stripped.startswith("["):
                continue
            matched = False
            for taxon in known_taxa_sorted:
                if stripped.startswith(taxon):
                    rest =
stripped[len(taxon):].strip().replace(" ", "")
                    if rest:
                        seqs[taxon].append(rest)
                        matched = True
                        break
            if not matched:
                parts = stripped.split()
                if len(parts) >= 2:
                    taxon = parts[0]
                    seq = "".join(parts[1:]).replace("
", "")
                    seqs[taxon].append(seq)
        return {k: "".join(v) for k, v in seqs.items()}
def parse_nexus_alignment(text: str) -> Dict[str,
str]:
    known_taxa = _extract_nexus_taxa(text)
    char_blocks =
re.findall(r"BEGIN\s+CHARACTERS;(.*?)END;", text,
flags=re.IGNORECASE | re.DOTALL)
    if not char_blocks:
        raise ValueError("В файле не найдено BEGIN
CHARACTERS.")
    merged = defaultdict(list)
    for block in char_blocks:
        part = _parse_nexus_matrix_block(block,
known_taxa)
        for taxon, seq in part.items():
            merged[taxon].append(seq)
    if not merged:
        raise ValueError("Не удалось извлечь
последовательности из MATRIX.")
    return {k: "".join(v) for k, v in
merged.items()}
def load_alignment_file(path: str | Path) ->
Dict[str, str]:
    path = Path(path)
    text = path.read_text(encoding="utf-8",
errors="ignore")
    upper = text.upper()
    if "#NEXUS" in upper or "BEGIN CHARACTERS;" in
upper:
        return parse_nexus_alignment(text)
    if text.lstrip().startswith(">"):
        return parse_fasta(text)
    raise ValueError(f"Неподдерживаемый формат
alignment: {path}")
def clean_newick(tree_str: str) -> str:
    return re.sub(r"\[.*?\]", "",
tree_str.strip()).strip()
def extract_newick_from_text(text: str) -> str:
    text = text.strip()
    tree_match =
re.search(r"\bTREE\b\s+([^\s]+\s*(\((.*)\))", text,
flags=re.IGNORECASE | re.DOTALL)
    if tree_match:
        return clean_newick(tree_match.group(1))
    first_paren = text.find("(")
    last_semicolon = text.rfind(";")
    if first_paren != -1 and last_semicolon != -1
and last_semicolon > first_paren:
        return
    clean_newick(text[first_paren:last_semicolon + 1])

```

```

        raise ValueError("Не удалось извлечь Newick из
        файла дерева.")
    def load_tree_file(path: str | Path):
        path = Path(path)
        text = path.read_text(encoding="utf-8",
        errors="ignore")
        newick = extract_newick_from_text(text)
        tree = Phylo.read(io.StringIO(newick),
        "newick")
        return tree, newick
    VALID_DNA = set("ACGT")
    DNA_PLUS_GAP = set("ACGT-?")
    def sanitize_seq(seq: str) -> str:
        return "".join(ch.upper() for ch in seq if
        ch.upper() in DNA_PLUS_GAP)
    def p_distance(seq1: str, seq2: str) -> float:
        mismatches = 0
        valid = 0
        for a, b in zip(seq1, seq2):
            if a in {"-", "?"} or b in {"-", "?"}:
                continue
            if a not in VALID_DNA or b not in VALID_DNA:
                continue
            valid += 1
            if a != b:
                mismatches += 1
        return np.nan if valid == 0 else mismatches /
        valid
    def seq_gap_fraction(seq: str) -> float:
        return np.nan if not seq else sum(ch in {"-",
        "?"} for ch in seq) / len(seq)
    def seq_gc_fraction(seq: str) -> float:
        letters = [ch for ch in seq if ch in VALID_DNA]
        return np.nan if not letters else sum(ch in
        {"G", "C"} for ch in letters) / len(letters)
    def alignment_stats(seqs: Dict[str, str]) ->
    Dict[str, float]:
        items = [sanitize_seq(s) for s in seqs.values()]
        if not items:
            return {
                "taxa_count": np.nan,
                "alignment_length": np.nan,
                "gap_fraction_global": np.nan,
                "gc_mean_global": np.nan,
                "gc_std_global": np.nan,
                "variable_site_fraction_global":
        np.nan,
            }
        n = len(items)
        L = max(len(s) for s in items)
        items = [s.ljust(L, "-") for s in items]
        gap_fraction_global =
        np.mean([seq_gap_fraction(s) for s in items])
        gc_values = [seq_gc_fraction(s) for s in items]
        gc_mean_global = float(np.nanmean(gc_values))
        gc_std_global = float(np.nanstd(gc_values))
        variable_sites = 0
        informative_positions = 0
        for i in range(L):
            col = [s[i] for s in items if s[i] in
        VALID_DNA]
            if len(col) < 2:
                continue
            informative_positions += 1
            if len(set(col)) > 1:
                variable_sites += 1
        variable_site_fraction = variable_sites /
        informative_positions if informative_positions else
        np.nan
        return {
            "taxa_count": float(n),
            "alignment_length": float(L),
            "gap_fraction_global":
        float(gap_fraction_global),
            "gc_mean_global": gc_mean_global,
            "gc_std_global": gc_std_global,

```

```

            "variable_site_fraction_global":
        float(variable_site_fraction),
        }
    def get_leaf_names(clade) -> List[str]:
        return [leaf.name for leaf in
        clade.get_terminals() if leaf.name]
    def mean_pairwise_distance(names: List[str], seqs:
    Dict[str, str]) -> float:
        if len(names) < 2:
            return 0.0
        vals = []
        for i in range(len(names)):
            for j in range(i + 1, len(names)):
                s1 = sanitize_seq(seqs.get(names[i],
        ""))
                s2 = sanitize_seq(seqs.get(names[j],
        ""))
                if not s1 or not s2:
                    continue
                d = p_distance(s1, s2)
                if not np.isnan(d):
                    vals.append(d)
        return np.nan if not vals else
        float(np.mean(vals))
    def clade_alignment_stats(names: List[str], seqs:
    Dict[str, str]) -> Dict[str, float]:
        sub = {k: seqs[k] for k in names if k in seqs}
        if not sub:
            return {
                "gap_fraction_clade": np.nan,
                "gc_mean_clade": np.nan,
                "gc_std_clade": np.nan,
                "mean_pairwise_pdist_clade": np.nan,
            }
        gap_fraction_clade =
        float(np.nanmean([seq_gap_fraction(sanitize_seq(v)
        ) for v in sub.values()]))
        gc_vals = [seq_gc_fraction(sanitize_seq(v)) for
        v in sub.values()]
        gc_mean_clade = float(np.nanmean(gc_vals))
        gc_std_clade = float(np.nanstd(gc_vals))
        mpd = mean_pairwise_distance(list(sub.keys()),
        sub)
        return {
            "gap_fraction_clade": gap_fraction_clade,
            "gc_mean_clade": gc_mean_clade,
            "gc_std_clade": gc_std_clade,
            "mean_pairwise_pdist_clade": mpd,
        }
    def subtree_balance(clade) -> float:
        children = clade.clades
        if len(children) < 2:
            return 0.0
        sizes = [len(ch.get_terminals()) for ch in
        children]
        total = sum(sizes)
        return 0.0 if total == 0 else abs(sizes[0] -
        sizes[1]) / total
    def extract_node_features(tree, seqs: Dict[str,
    str], dataset_id: str) -> pd.DataFrame:
        global_stats = alignment_stats(seqs)
        depths = tree.depths()
        rows = []
        node_idx = 0
        for clade in tree.find_clades(order="level"):
            if clade.is_terminal():
                continue
            conf = clade.confidence
            if conf is None:
                continue
            bootstrap = float(conf)
            if bootstrap <= 1.0:
                bootstrap *= 100.0
            branch_length =
            max(float(clade.branch_length or 0.0), 0.0)
            leaf_names = get_leaf_names(clade)

```

```

        n_leaves = len(leaf_names)
        clade_stats =
    clade_alignment_stats(leaf_names, seqs)
        child_branch_lengths =
    [max(float(ch.branch_length or 0.0), 0.0) for ch in
    clade.clades]
        rows.append({
            "dataset_id": dataset_id,
            "node_id":
    f"{dataset_id}_{node_{node_idx}}",
            "target_bootstrap": bootstrap,
            "branch_length": branch_length,
            "depth": float(depths.get(clade, 0.0)),
            "n_leaves_subtree": float(n_leaves),
            "subtree_fraction": float(n_leaves /
    global_stats["taxa_count"]) if
    global_stats["taxa_count"] else np.nan,
            "subtree_balance":
    float(subtree_balance(clade)),
            "mean_child_branch_length":
    float(np.mean(child_branch_lengths)) if
    child_branch_lengths else 0.0,
            "std_child_branch_length":
    float(np.std(child_branch_lengths)) if
    child_branch_lengths else 0.0,
            **global_stats,
            **clade_stats,
        })
        node_idx += 1
        return pd.DataFrame(rows)
    ALIGNMENT_EXTS = {".nex", ".nexus", ".nexorg",
    ".fasta", ".fa", ".fas", ".txt"}
    TREE_HINTS = ("tree", "bootstrap", "nwk", "newick",
    "con_", "majorrule")
    def detect_alignment_file(ds_path: Path) -> Path |
    None:
        candidates = [f for f in ds_path.iterdir() if
    f.is_file() and f.suffix.lower() in ALIGNMENT_EXTS]
        candidates = sorted(candidates, key=lambda p:
    (p.suffix.lower() not in {".nex", ".nexus",
    ".nexorg"}, p.name.lower()))
        return candidates[0] if candidates else None
    def detect_tree_file(ds_path: Path, alignment_file:
    Path | None) -> Path | None:
        files = [f for f in ds_path.iterdir() if
    f.is_file()]
        if alignment_file is not None:
            files = [f for f in files if f.resolve() !=
    alignment_file.resolve()]
        scored = []
        for f in files:
            name_l = f.name.lower()
            score = 0
            if any(h in name_l for h in TREE_HINTS):
                score += 3
            if f.suffix.lower() in {".nwk", ".newick",
    ".tree"}:
                score += 3
            if f.suffix == "":
                score += 1
            scored.append((score, f))
        scored.sort(key=lambda x: (-x[0],
    x[1].name.lower()))
        for _, f in scored:
            try:
                load_tree_file(f)
                return f
            except Exception:
                continue
        return None
    def build_feature_table(datasets_root: str | Path):
        datasets_root = Path(datasets_root)
        all_frames = []
        audit = []
        for ds_path in sorted(datasets_root.iterdir()):
            if not ds_path.is_dir():

```

```

                continue
            dataset_id = ds_path.name
            alignment_file =
    detect_alignment_file(ds_path)
            tree_file = detect_tree_file(ds_path,
    alignment_file)
            status = {
                "dataset_id": dataset_id,
                "alignment_file": str(alignment_file)
    if alignment_file else None,
                "tree_file": str(tree_file) if
    tree_file else None,
                "ok": False,
                "error": None,
            }
            try:
                if alignment_file is None:
                    raise FileNotFoundError("Не найден
    alignment-файл")
                if tree_file is None:
                    raise FileNotFoundError("Не найден
    tree-файл")
                seqs =
    load_alignment_file(alignment_file)
                tree, _ = load_tree_file(tree_file)
                df = extract_node_features(tree, seqs,
    dataset_id)
                if df.empty:
                    raise ValueError("Не извлечено ни
    одного внутреннего узла с bootstrap")
                all_frames.append(df)
                status["ok"] = True
                status["n_rows"] = int(len(df))
            except Exception as e:
                status["error"] = str(e)
                audit.append(status)
            if not all_frames:
                audit_df = pd.DataFrame(audit)
                raise RuntimeError("Не удалось собрать ни
    одной строки датасета. Проверь dataset_audit:\n" +
    audit_df.to_string(index=False))
            return pd.concat(all_frames,
    ignore_index=True), audit
    FEATURE_COLUMNS = [
        "branch_length",
        "depth",
        "n_leaves_subtree",
        "subtree_fraction",
        "subtree_balance",
        "mean_child_branch_length",
        "std_child_branch_length",
        "taxa_count",
        "alignment_length",
        "gap_fraction_global",
        "gc_mean_global",
        "gc_std_global",
        "variable_site_fraction_global",
        "gap_fraction_clade",
        "gc_mean_clade",
        "gc_std_clade",
        "mean_pairwise_pdist_clade",
    ]
    def train_model(df: pd.DataFrame):
        X = df[FEATURE_COLUMNS].copy()
        y = df["target_bootstrap"].copy()
        groups = df["dataset_id"].copy()
        X["bl_cv"] = X["std_child_branch_length"] /
    (X["mean_child_branch_length"] + 1e-6)
        X["gap_var_clade"] = (X["gap_fraction_clade"] -
    X["gap_fraction_global"]).abs()
        X["gc_extreme"] = ((X["gc_mean_clade"] -
    X["gc_mean_global"]).abs() > 0.15).astype(float)
        X["tiny_clade"] = (X["n_leaves_subtree"] <=
    3).astype(float)
        X["deep_node"] = X["depth"] / (X["depth"].max()
    + 1e-6)

```



```

    model_features = FEATURE_COLUMNS + ["bl_cv",
"gap_var_clade", "gc_extreme", "tiny_clade",
"deep_node"]
    X = X[model_features]
    gkf = GroupKFold(n_splits=5)
    pipe = Pipeline([
        ("imputer",
SimpleImputer(strategy="median")),
        ("rf", RandomForestRegressor(
            random_state=42,
            n_jobs=-1,
            criterion="squared_error", # прямая
оптимизация под R^2
            bootstrap=True
        ))
    ])
    param_dist = {
        'rf_n_estimators': randint(400, 1000),
        'rf_max_depth': randint(10, 60),
        'rf_min_samples_split': randint(5, 20),
        'rf_min_samples_leaf': randint(3, 15),
        'rf_max_features': ['sqrt', 'log2', 0.4,
0.6, 0.8],
    }
    search = RandomizedSearchCV(
        estimator=pipe,
        param_distributions=param_dist,
        n_iter=50,
        scoring="r2",
        cv=gkf,
        n_jobs=-1,
        random_state=42,
        refit=True,
        verbose=2
    )
    print("[INFO] Запуск подбора гиперпараметров
для Random Forest...")
    search.fit(X, y, groups=groups)
    best_model = search.best_estimator_
    best_params = search.best_params_
    cv_r2 = search.best_score_
    final_fold_idx = list(gkf.split(X, y,
groups))[-1][1]
    X_test, y_test, g_test =
X.iloc[final_fold_idx], y.iloc[final_fold_idx],
groups.iloc[final_fold_idx]
    y_pred = np.clip(best_model.predict(X_test),
0.0, 100.0)
    metrics = {
        "n_rows_total": int(len(df)),
        "mae": float(mean_absolute_error(y_test,
y_pred)),
        "rmse":
float(math.sqrt(mean_squared_error(y_test,
y_pred))),
        "r2": float(r2_score(y_test, y_pred)),
        "cv_r2_mean": float(cv_r2),
        "best_params": {k.split("_")[1]: v for k,
v in best_params.items()},
        "pred_range": f"{y_pred.min():.1f}-
{y_pred.max():.1f}",
        "feature_columns": model_features
    }
    rf = best_model.named_steps["rf"]
    feature_importance = pd.DataFrame({
        "feature": model_features,
        "importance": rf.feature_importances_
    }).sort_values("importance", ascending=False)
    pred_df = pd.DataFrame({
        "y_true": y_test.values,
        "y_pred": y_pred,
        "dataset_id": g_test.values
    })
    return best_model, metrics, feature_importance,
pred_df
def main():

```

```

    project_root = Path.cwd()
    datasets_root = project_root / "datasets"
    output_dir = project_root / "ml_outputs"
    output_dir.mkdir(exist_ok=True)
    print(f"[INFO] datasets root: {datasets_root}")
    print(f"[INFO] output dir: {output_dir}")
    df, audit = build_feature_table(datasets_root)
    df.to_csv(output_dir / "node_dataset.csv",
index=False, encoding="utf-8-sig")
    pd.DataFrame(audit).to_csv(output_dir /
"dataset_audit.csv", index=False, encoding="utf-8-
sig")
    model, metrics, feature_importance, pred_df =
train_model(df)
    joblib.dump(model, output_dir / "model_v1.pkl")
    feature_importance.to_csv(output_dir /
"feature_importance.csv", index=False,
encoding="utf-8-sig")
    pred_df.to_csv(output_dir /
"test_predictions.csv", index=False, encoding="utf-
8-sig")
    with open(output_dir / "model_metadata.json",
"w", encoding="utf-8") as f:
        json.dump(metrics, f, ensure_ascii=False,
indent=2)
    print("\n=== TRAIN DONE ===")
    print(json.dumps(metrics, ensure_ascii=False,
indent=2))
    print("\nTop feature importance:")

print(feature_importance.head(10).to_string(index=
False))
if __name__ == "__main__":
    main()
    from __future__ import annotations
    import io
    import json
    import math
    import re
    from collections import defaultdict
    from pathlib import Path
    from typing import Dict, List
    import joblib
    import numpy as np
    import pandas as pd
    from Bio import Phylo
    from sklearn.ensemble import RandomForestRegressor
    from sklearn.impute import SimpleImputer
    from sklearn.metrics import mean_absolute_error,
r2_score, mean_squared_error
    from sklearn.model_selection import GroupKFold
    from sklearn.pipeline import Pipeline
    import joblib
    import numpy as np
    import pandas as pd
    from sklearn.ensemble import RandomForestRegressor
    from sklearn.impute import SimpleImputer
    from sklearn.metrics import mean_absolute_error,
r2_score, mean_squared_error
    from sklearn.model_selection import GroupKFold,
RandomizedSearchCV
    from sklearn.pipeline import Pipeline
    from scipy.stats import randint
    import math
    def parse_fasta(text: str) -> Dict[str, str]:
        seqs = {}
        current = None
        for raw_line in text.splitlines():
            line = raw_line.strip()
            if not line:
                continue
            if line.startswith(">"):
                current = line[1:].strip()
                seqs[current] = []
            else:
                if current is None:

```

```

        continue
        seqs[current].append(line.replace(" ",
""))
    return {k: "".join(v) for k, v in seqs.items()}
def _extract_nexus_taxa(text: str) -> List[str]:
    m = re.search(r"BEGIN\s+TAXA;(.+?)END;", text,
flags=re.IGNORECASE | re.DOTALL)
    if not m:
        return []
    block = m.group(1)
    tm = re.search(r"TAXLABELS(.+?);", block,
flags=re.IGNORECASE | re.DOTALL)
    if not tm:
        return []
    tokens = re.split(r"\s+", tm.group(1).strip())
    return [t.strip() for t in tokens if t.strip()]
def _parse_nexus_matrix_block(block: str,
known_taxa: List[str]) -> Dict[str, str]:
    mm = re.search(r"\bMATRIX\b(.+?);", block,
flags=re.IGNORECASE | re.DOTALL)
    if not mm:
        return {}
    matrix_text = mm.group(1)
    seqs = defaultdict(list)
    known_taxa_sorted = sorted(known_taxa, key=len,
reverse=True)
    for raw_line in matrix_text.splitlines():
        stripped = raw_line.strip()
        if not stripped or
stripped.startswith("["):
            continue
        matched = False
        for taxon in known_taxa_sorted:
            if stripped.startswith(taxon):
                rest =
stripped[len(taxon):].strip().replace(" ", "")
                if rest:
                    seqs[taxon].append(rest)
                    matched = True
                    break
        if not matched:
            parts = stripped.split()
            if len(parts) >= 2:
                taxon = parts[0]
                seq = "".join(parts[1:]).replace("
", "")
                seqs[taxon].append(seq)
    return {k: "".join(v) for k, v in seqs.items()}
def parse_nexus_alignment(text: str) -> Dict[str,
str]:
    known_taxa = _extract_nexus_taxa(text)
    char_blocks =
re.findall(r"BEGIN\s+CHARACTERS;(.+?)END;", text,
flags=re.IGNORECASE | re.DOTALL)
    if not char_blocks:
        raise ValueError("В файле не найдено BEGIN
CHARACTERS.")
    merged = defaultdict(list)
    for block in char_blocks:
        part = _parse_nexus_matrix_block(block,
known_taxa)
        for taxon, seq in part.items():
            merged[taxon].append(seq)
    if not merged:
        raise ValueError("Не удалось извлечь
последовательности из MATRIX.")
    return {k: "".join(v) for k, v in
merged.items()}
def load_alignment_file(path: str | Path) ->
Dict[str, str]:
    path = Path(path)
    text = path.read_text(encoding="utf-8",
errors="ignore")
    upper = text.upper()
    if "#NEXUS" in upper or "BEGIN CHARACTERS;" in
upper:

```

```

        return parse_nexus_alignment(text)
    if text.lstrip().startswith(">"):
        return parse_fasta(text)
    raise ValueError(f"Неподдерживаемый формат
alignment: {path}")
def clean_newick(tree_str: str) -> str:
    return re.sub(r"\[.*?\]", "",
tree_str.strip()).strip()
def extract_newick_from_text(text: str) -> str:
    text = text.strip()
    tree_match =
re.search(r"\bTREE\b\s+[\^=]+\s*(\(..*?);)", text,
flags=re.IGNORECASE | re.DOTALL)
    if tree_match:
        return clean_newick(tree_match.group(1))
    first_paren = text.find("(")
    last_semicolon = text.rfind(";")
    if first_paren != -1 and last_semicolon != -1
and last_semicolon > first_paren:
        return
clean_newick(text[first_paren:last_semicolon + 1])
    raise ValueError("Не удалось извлечь Newick из
файла дерева.")
def load_tree_file(path: str | Path):
    path = Path(path)
    text = path.read_text(encoding="utf-8",
errors="ignore")
    newick = extract_newick_from_text(text)
    tree = Phylo.read(io.StringIO(newick),
"newick")
    return tree, newick
VALID_DNA = set("ACGT")
DNA_PLUS_GAP = set("ACGT-?")
def sanitize_seq(seq: str) -> str:
    return "".join(ch.upper() for ch in seq if
ch.upper() in DNA_PLUS_GAP)
def p_distance(seq1: str, seq2: str) -> float:
    mismatches = 0
    valid = 0
    for a, b in zip(seq1, seq2):
        if a in {"-", "?"} or b in {"-", "?"}:
            continue
        if a not in VALID_DNA or b not in VALID_DNA:
            continue
        valid += 1
        if a != b:
            mismatches += 1
    return np.nan if valid == 0 else mismatches /
valid
def seq_gap_fraction(seq: str) -> float:
    return np.nan if not seq else sum(ch in {"-",
"?"} for ch in seq) / len(seq)
def seq_gc_fraction(seq: str) -> float:
    letters = [ch for ch in seq if ch in VALID_DNA]
    return np.nan if not letters else sum(ch in
{"G", "C"} for ch in letters) / len(letters)
def alignment_stats(seqs: Dict[str, str]) ->
Dict[str, float]:
    items = [sanitize_seq(s) for s in seqs.values()]
    if not items:
        return {
            "taxa_count": np.nan,
            "alignment_length": np.nan,
            "gap_fraction_global": np.nan,
            "gc_mean_global": np.nan,
            "gc_std_global": np.nan,
            "variable_site_fraction_global":
np.nan,
        }
    n = len(items)
    L = max(len(s) for s in items)
    items = [s.ljust(L, "-") for s in items]
    gap_fraction_global =
np.mean([seq_gap_fraction(s) for s in items])
    gc_values = [seq_gc_fraction(s) for s in items]
    gc_mean_global = float(np.nanmean(gc_values))

```

```

gc_std_global = float(np.nanstd(gc_values))
variable_sites = 0
informative_positions = 0
for i in range(L):
    col = [s[i] for s in items if s[i] in
VALID_DNA]
    if len(col) < 2:
        continue
    informative_positions += 1
    if len(set(col)) > 1:
        variable_sites += 1
    variable_site_fraction = variable_sites /
informative_positions if informative_positions else
np.nan
    return {
        "taxa_count": float(n),
        "alignment_length": float(L),
        "gap_fraction_global":
float(gap_fraction_global),
        "gc_mean_global": gc_mean_global,
        "gc_std_global": gc_std_global,
        "variable_site_fraction_global":
float(variable_site_fraction),
    }
def get_leaf_names(clade) -> List[str]:
    return [leaf.name for leaf in
clade.get_terminals() if leaf.name]
def mean_pairwise_distance(names: List[str], seqs:
Dict[str, str]) -> float:
    if len(names) < 2:
        return 0.0
    vals = []
    for i in range(len(names)):
        for j in range(i + 1, len(names)):
            s1 = sanitize_seq(seqs.get(names[i],
""))
            s2 = sanitize_seq(seqs.get(names[j],
""))
            if not s1 or not s2:
                continue
            d = p_distance(s1, s2)
            if not np.isnan(d):
                vals.append(d)
    return np.nan if not vals else
float(np.mean(vals))
def clade_alignment_stats(names: List[str], seqs:
Dict[str, str]) -> Dict[str, float]:
    sub = {k: seqs[k] for k in names if k in seqs}
    if not sub:
        return {
            "gap_fraction_clade": np.nan,
            "gc_mean_clade": np.nan,
            "gc_std_clade": np.nan,
            "mean_pairwise_pdist_clade": np.nan,
        }
    gap_fraction_clade =
float(np.nanmean([seq_gap_fraction(sanitize_seq(v)
) for v in sub.values()])))
    gc_vals = [seq_gc_fraction(sanitize_seq(v)) for
v in sub.values()]
    gc_mean_clade = float(np.nanmean(gc_vals))
    gc_std_clade = float(np.nanstd(gc_vals))
    mpd = mean_pairwise_distance(list(sub.keys()),
sub)
    return {
        "gap_fraction_clade": gap_fraction_clade,
        "gc_mean_clade": gc_mean_clade,
        "gc_std_clade": gc_std_clade,
        "mean_pairwise_pdist_clade": mpd,
    }
def subtree_balance(clade) -> float:
    children = clade.clades
    if len(children) < 2:
        return 0.0
    sizes = [len(ch.get_terminals()) for ch in
children]

```

```

total = sum(sizes)
return 0.0 if total == 0 else abs(sizes[0] -
sizes[1]) / total
def extract_node_features(tree, seqs: Dict[str,
str], dataset_id: str) -> pd.DataFrame:
    global_stats = alignment_stats(seqs)
    depths = tree.depths()
    rows = []
    node_idx = 0
    for clade in tree.find_clades(order="level"):
        if clade.is_terminal():
            continue
        conf = clade.confidence
        if conf is None:
            continue
        bootstrap = float(conf)
        if bootstrap <= 1.0:
            bootstrap *= 100.0
        branch_length =
max(float(clade.branch_length or 0.0), 0.0)
        leaf_names = get_leaf_names(clade)
        n_leaves = len(leaf_names)
        clade_stats =
clade_alignment_stats(leaf_names, seqs)
        child_branch_lengths =
[max(float(ch.branch_length or 0.0), 0.0) for ch in
clade.clades]
        rows.append({
            "dataset_id": dataset_id,
            "node_id":
f"{dataset_id}_node_{node_idx}",
            "target_bootstrap": bootstrap,
            "branch_length": branch_length,
            "depth": float(depths.get(clade, 0.0)),
            "n_leaves_subtree": float(n_leaves),
            "subtree_fraction": float(n_leaves /
global_stats["taxa_count"]) if
global_stats["taxa_count"] else np.nan,
            "subtree_balance":
float(subtree_balance(clade)),
            "mean_child_branch_length":
float(np.mean(child_branch_lengths)) if
child_branch_lengths else 0.0,
            "std_child_branch_length":
float(np.std(child_branch_lengths)) if
child_branch_lengths else 0.0,
            **global_stats,
            **clade_stats,
        })
        node_idx += 1
    return pd.DataFrame(rows)
ALIGNMENT_EXTS = {".nex", ".nexus", ".nexorg",
".fasta", ".fa", ".fas", ".txt"}
TREE_HINTS = ("tree", "bootstrap", "nwk", "newick",
"con", "majrule")
def detect_alignment_file(ds_path: Path) -> Path |
None:
    candidates = [f for f in ds_path.iterdir() if
f.is_file() and f.suffix.lower() in ALIGNMENT_EXTS]
    candidates = sorted(candidates, key=lambda p:
(p.suffix.lower() not in {".nex", ".nexus",
".nexorg"}, p.name.lower()))
    return candidates[0] if candidates else None
def detect_tree_file(ds_path: Path, alignment_file:
Path | None) -> Path | None:
    files = [f for f in ds_path.iterdir() if
f.is_file()]
    if alignment_file is not None:
        files = [f for f in files if f.resolve() !=
alignment_file.resolve()]
    scored = []
    for f in files:
        name_l = f.name.lower()
        score = 0
        if any(h in name_l for h in TREE_HINTS):
            score += 3

```

```

        if f.suffix.lower() in {".nwk", ".newick",
".tree"}:
            score += 3
            if f.suffix == "":
                score += 1
            scored.append((score, f))
        scored.sort(key=lambda x: (-x[0],
x[1].name.lower()))
        for _, f in scored:
            try:
                load_tree_file(f)
                return f
            except Exception:
                continue
        return None
def build_feature_table(datasets_root: str | Path):
    datasets_root = Path(datasets_root)
    all_frames = []
    audit = []
    for ds_path in sorted(datasets_root.iterdir()):
        if not ds_path.is_dir():
            continue
        dataset_id = ds_path.name
        alignment_file =
detect_alignment_file(ds_path)
        tree_file = detect_tree_file(ds_path,
alignment_file)
        status = {
            "dataset_id": dataset_id,
            "alignment_file": str(alignment_file)
        }
        if alignment_file else None,
            "tree_file": str(tree_file) if
tree_file else None,
            "ok": False,
            "error": None,
        }
        try:
            if alignment_file is None:
                raise FileNotFoundError("Не найден
alignment-файл")
            if tree_file is None:
                raise FileNotFoundError("Не найден
tree-файл")
            seqs =
load_alignment_file(alignment_file) =
            tree, _ = load_tree_file(tree_file)
            df = extract_node_features(tree, seqs,
dataset_id)
            if df.empty:
                raise ValueError("Не извлечено ни
одного внутреннего узла с bootstrap")
            all_frames.append(df)
            status["ok"] = True
            status["n_rows"] = int(len(df))
        except Exception as e:
            status["error"] = str(e)
        audit.append(status)
    if not all_frames:
        audit_df = pd.DataFrame(audit)
        raise RuntimeError("Не удалось собрать ни
одной строки датасета. Проверь dataset_audit:\n" +
audit_df.to_string(index=False))
    return pd.concat(all_frames,
ignore_index=True), audit
FEATURE_COLUMNS = [
    "branch_length",
    "depth",
    "n_leaves_subtree",
    "subtree_fraction",
    "subtree_balance",
    "mean_child_branch_length",
    "std_child_branch_length",
    "taxa_count",
    "alignment_length",
    "gap_fraction_global",
    "gc_mean_global",

```

```

    "gc_std_global",
    "variable_site_fraction_global",
    "gap_fraction_clade",
    "gc_mean_clade",
    "gc_std_clade",
    "mean_pairwise_pdist_clade",
]
def train_model(df: pd.DataFrame):
    X = df[FEATURE_COLUMNS].copy()
    y = df["target_bootstrap"].copy()
    groups = df["dataset_id"].copy()
    X["bl_cv"] = X["std_child_branch_length"] /
(X["mean_child_branch_length"] + 1e-6)
    X["gap_var_clade"] = (X["gap_fraction_clade"] -
X["gap_fraction_global"]).abs()
    X["gc_extreme"] = ((X["gc_mean_clade"] -
X["gc_mean_global"]).abs() > 0.15).astype(float)
    X["tiny_clade"] = (X["n_leaves_subtree"] <=
3).astype(float)
    X["deep_node"] = X["depth"] / (X["depth"].max()
+ 1e-6)
    model_features = FEATURE_COLUMNS + ["bl_cv",
"gap_var_clade", "gc_extreme", "tiny_clade",
"deep_node"]
    X = X[model_features]
    gkf = GroupKFold(n_splits=5)
    pipe = Pipeline([
        ("imputer",
SimpleImputer(strategy="median")),
        ("rf", RandomForestRegressor(
            random_state=42,
            n_jobs=-1,
            criterion="squared_error", # прямая
оптимизация под R^2
            bootstrap=True
        ))
    ])
    param_dist = {
        'rf_n_estimators': randint(400, 1000),
        'rf_max_depth': randint(10, 60),
        'rf_min_samples_split': randint(5, 20),
        'rf_min_samples_leaf': randint(3, 15),
        'rf_max_features': ['sqrt', 'log2', 0.4,
0.6, 0.8],
    }
    search = RandomizedSearchCV(
        estimator=pipe,
        param_distributions=param_dist,
        n_iter=50,
        scoring="r2",
        cv=gkf,
        n_jobs=-1,
        random_state=42,
        refit=True,
        verbose=2
    )
    print("[INFO] Запуск подбора гиперпараметров
для Random Forest...")
    search.fit(X, y, groups=groups)
    best_model = search.best_estimator_
    best_params = search.best_params_
    cv_r2 = search.best_score_
    final_fold_idx = list(gkf.split(X, y,
groups))[-1][1]
    X_test, y_test, g_test =
X.iloc[final_fold_idx], y.iloc[final_fold_idx],
groups.iloc[final_fold_idx]
    y_pred = np.clip(best_model.predict(X_test),
0.0, 100.0)
    metrics = {
        "n_rows_total": int(len(df)),
        "mae": float(mean_absolute_error(y_test,
y_pred)),
        "rmse":
float(math.sqrt(mean_squared_error(y_test,
y_pred))),

```

```

        "r2": float(r2_score(y_test, y_pred)),
        "cv_r2_mean": float(cv_r2),
        "best_params": {k.split("_")[1]: v for k,
v in best_params.items()},
        "pred_range": f"{y_pred.min():.1f}-{
y_pred.max():.1f}",
        "feature_columns": model_features
    }
    rf = best_model.named_steps["rf"]
    feature_importance = pd.DataFrame({
        "feature": model_features,
        "importance": rf.feature_importances_
    }).sort_values("importance", ascending=False)
    pred_df = pd.DataFrame({
        "y_true": y_test.values,
        "y_pred": y_pred,
        "dataset_id": g_test.values
    })
    return best_model, metrics, feature_importance,
pred_df
def main():
    project_root = Path.cwd()
    datasets_root = project_root / "datasets"
    output_dir = project_root / "ml_outputs"
    output_dir.mkdir(exist_ok=True)
    print(f"[INFO] datasets root: {datasets_root}")
    print(f"[INFO] output dir: {output_dir}")
    df, audit = build_feature_table(datasets_root)
    df.to_csv(output_dir / "node_dataset.csv",
index=False, encoding="utf-8-sig")
    pd.DataFrame(audit).to_csv(output_dir /
"dataset_audit.csv", index=False, encoding="utf-8-
sig")
    model, metrics, feature_importance, pred_df =
train_model(df)
    joblib.dump(model, output_dir / "model_v1.pkl")
    feature_importance.to_csv(output_dir /
"feature_importance.csv", index=False,
encoding="utf-8-sig")
    pred_df.to_csv(output_dir /
"test_predictions.csv", index=False, encoding="utf-
8-sig")
    with open(output_dir / "model_metadata.json",
"w", encoding="utf-8") as f:
        json.dump(metrics, f, ensure_ascii=False,
indent=2)
    print("\n=== TRAIN DONE ===")
    print(json.dumps(metrics, ensure_ascii=False,
indent=2))
    print("\nTop feature importance:")

    print(feature_importance.head(10).to_string(index=
False))
    if __name__ == "__main__":
        main()
    from __future__ import annotations
    import re
    from pathlib import Path
    from Bio import AlignIO
    from Bio.Align import MultipleSeqAlignment
    from Bio.Seq import Seq
    from Bio.SeqRecord import SeqRecord
    from .file_kind import TextFileReader,
CommentStripper
    from .validation import AlignmentFileValidator,
NexusParseError
    class NexusMatrixParser:
        def parse(self, text: str) ->
MultipleSeqAlignment:
            stripped = CommentStripper.strip(text)
            matrix_match =
re.search(r"\bMATRIX\b(.*?)\n\s*;", stripped,
flags=re.IGNORECASE | re.DOTALL)
            if not matrix_match:
                raise NexusParseError('B NEXUS не
найден блок MATRIX.')

```

```

        matrix_text = matrix_match.group(1)
        records: list[SeqRecord] = []
        for raw_line in matrix_text.splitlines():
            line = raw_line.strip()
            if not line or line.startswith('['):
                continue
            parts = line.split()
            if len(parts) < 2:
                continue
            taxon = parts[0]
            sequence = ''.join(parts[1:]).upper()

        records.append(SeqRecord(Seq(sequence), id=taxon,
name=taxon, description=''))
        if not records:
            raise NexusParseError('Не удалось
извлечь последовательности из MATRIX.')
        return MultipleSeqAlignment(records)
    class AlignmentIntegrityValidator:
        def validate(self, alignment:
MultipleSeqAlignment) -> None:
            if len(alignment) == 0:
                raise NexusParseError('В файле не
найдено ни одной последовательности.')
            lengths = {len(record.seq) for record in
alignment}
            if len(lengths) != 1:
                raise
NexusParseError('Последовательности не выровнены:
длины различаются.')
    class AlignmentLoader:
        def __init__(
            self,
            file_validator: AlignmentFileValidator |
None = None,
            text_reader: TextFileReader | None = None,
            matrix_parser: NexusMatrixParser | None =
None,
            integrity_validator:
AlignmentIntegrityValidator | None = None,
        ) -> None:
            self.file_validator = file_validator or
AlignmentFileValidator()
            self.text_reader = text_reader or
TextFileReader()
            self.matrix_parser = matrix_parser or
NexusMatrixParser()
            self.integrity_validator =
integrity_validator or
AlignmentIntegrityValidator()
            def load(self, path: str | Path) ->
MultipleSeqAlignment:
                file_path = Path(path)
                self.file_validator.validate(file_path)
                suffix = file_path.suffix.lower()
                if suffix in {'.fasta', '.fa', '.fas',
'.aln'}:
                    alignment =
AlignIO.read(str(file_path), 'fasta')
                elif suffix in {'.nex', '.nexus', '.nexorg',
'.txt'}:
                    alignment = self._load_nexus(file_path)
                else:
                    raise
NexusParseError(f'Неподдерживаемый формат файла:
{file_path.suffix}')

            self.integrity_validator.validate(alignment)
            return alignment
        def _load_nexus(self, path: Path) ->
MultipleSeqAlignment:
            try:
                return AlignIO.read(str(path), 'nexus')
            except Exception:

```

```

        return
self.matrix_parser.parse(self.text_reader.read(path))
from __future__ import annotations
from pathlib import Path
from Bio import AlignIO
from Bio.Align import MultipleSeqAlignment
class AlignmentSummaryBuilder:
    def build(self, alignment:
MultipleSeqAlignment) -> dict:
        taxa_count = len(alignment)
        length = alignment.get_alignment_length()
if taxa_count else 0
        gap_count = 0
        total_count = taxa_count * length if
taxa_count and length else 0
        for record in alignment:
            seq = str(record.seq)
            gap_count += sum(1 for ch in seq if ch
in '-?')
        gap_fraction = (gap_count / total_count) if
total_count else 0.0
        return {
            'taxa_count': taxa_count,
            'length': length,
            'gap_fraction': gap_fraction,
            'taxa': [record.id for record in
alignment],
        }
class AlignmentExporter:
    def export_fasta(self, alignment:
MultipleSeqAlignment, output_path: str | Path) ->
Path:
        output = Path(output_path)
        AlignIO.write(alignment, str(output),
'fasta')
        return output
from __future__ import annotations
import re
from pathlib import Path
class TextFileReader:
    def read(self, path: str | Path) -> str:
        return Path(path).read_text(encoding='utf-
8', errors='ignore')
class CommentStripper:
    @staticmethod
    def strip(text: str) -> str:
        return re.sub(r"[\.\?\*]", "", text,
flags=re.DOTALL)
class FileKindDetector:
    def __init__(self, reader: TextFileReader | None
= None) -> None:
        self.reader = reader or TextFileReader()
    def detect(self, path: str | Path) -> str:
        file_path = Path(path)
        suffix = file_path.suffix.lower()
        if suffix in {'.fasta', '.fa', '.fas',
'.aln'}:
            return 'alignment_fasta'
        text
        =
CommentStripper.strip(self.reader.read(file_path))
        .upper()
        if 'BEGIN TREES' in text:
            return 'tree_nexus'
        if 'BEGIN DATA' in text or 'BEGIN
CHARACTERS' in text or 'MATRIX' in text:
            return 'alignment_nexus'
        return 'tree_newick'
from __future__ import annotations
from pathlib import Path
from Bio.Align import MultipleSeqAlignment
from .alignment_loader import AlignmentLoader
from .alignment_summary import AlignmentExporter,
AlignmentSummaryBuilder
from .file_kind import CommentStripper,
FileKindDetector, TextFileReader

```

```

from .validation import AlignmentFileValidator,
NexusParseError, TreeFileValidator
_reader = TextFileReader()
_detector = FileKindDetector(_reader)
_alignment_validator
=
AlignmentFileValidator(_detector, _reader)
_tree_validator = TreeFileValidator(_detector,
_reader)
_alignment_loader
=
AlignmentLoader(_alignment_validator, _reader)
_summary_builder = AlignmentSummaryBuilder()
_exporter = AlignmentExporter()
def _read_text(path: str | Path) -> str:
    return _reader.read(path)
def _strip_comments(text: str) -> str:
    return CommentStripper.strip(text)
def detect_file_kind(path: str | Path) -> str:
    return _detector.detect(path)
def validate_alignment_file(path: str | Path) ->
dict:
    return _alignment_validator.validate(path)
def validate_tree_file(path: str | Path) -> dict:
    return _tree_validator.validate(path)
def load_alignment(path: str | Path) ->
MultipleSeqAlignment:
    return _alignment_loader.load(path)
def load_nexus_alignment(path: str | Path) ->
MultipleSeqAlignment:
    return _alignment_loader.load(path)
def
alignment_summary(alignment:
MultipleSeqAlignment) -> dict:
    return _summary_builder.build(alignment)
def export_fasta(alignment: MultipleSeqAlignment,
output_path: str | Path) -> Path:
    return _exporter.export_fasta(alignment,
output_path)
__all__ = [
    'NexusParseError',
    '_read_text',
    '_strip_comments',
    'detect_file_kind',
    'validate_alignment_file',
    'validate_tree_file',
    'load_alignment',
    'load_nexus_alignment',
    'alignment_summary',
    'export_fasta',
]
from __future__ import annotations
import re
from pathlib import Path
from .file_kind import CommentStripper,
FileKindDetector, TextFileReader
class NexusParseError(ValueError):
    """Raised when an input file does not contain a
readable alignment or tree."""
class AlignmentFileValidator:
    def __init__(self, detector: FileKindDetector |
None = None, reader: TextFileReader | None = None)
-> None:
        self.detector = detector or
FileKindDetector()
        self.reader = reader or TextFileReader()
    def validate(self, path: str | Path) -> dict:
        file_path = Path(path)
        kind = self.detector.detect(file_path)
        if kind == 'alignment_fasta':
            return self._validate_fasta(file_path,
kind)
        return self._validate_nexus(file_path,
kind)
    def _validate_fasta(self, path: Path, kind: str)
-> dict:
        text = self.reader.read(path)
        headers = [line for line in
text.splitlines() if line.strip().startswith('>')]

```

```

        if not headers:
            raise NexusParseError("FASTA-файл
должен содержать заголовки, начинающиеся с символа
'>'.")
        return {
            'kind': kind,
            'format': 'FASTA',
            'required_blocks': ['>taxon',
'sequence'],
            'present_blocks': ['>taxon'],
            'is_valid': True,
        }
    def _validate_nexus(self, path: Path, kind: str)
-> dict:
        text = CommentStripper.strip(self.reader.read(path))
        upper = text.upper()
        present = []
        for block in ['BEGIN TAXA', 'BEGIN DATA',
'BEGIN CHARACTERS', 'MATRIX', 'DIMENSIONS']:
            if block in upper:
                present.append(block)
            has_data_block = ('BEGIN DATA' in upper) or
('BEGIN CHARACTERS' in upper)
            if not has_data_block:
                raise NexusParseError('Для NEXUS-
выравнивания нужен блок BEGIN DATA или BEGIN
CHARACTERS.')
            if 'MATRIX' not in upper:
                raise NexusParseError('Для NEXUS-
выравнивания нужен блок MATRIX с
последовательностями.')
        return {
            'kind': kind,
            'format': 'NEXUS alignment',
            'required_blocks': ['BEGIN
DATA/CHARACTERS', 'MATRIX'],
            'present_blocks': present,
            'is_valid': True,
        }
class TreeFileValidator:
    def __init__(self, detector: FileKindDetector |
None = None, reader: TextFileReader | None = None)
-> None:
        self.detector = detector or
FileKindDetector()
        self.reader = reader or TextFileReader()
    def validate(self, path: str | Path) -> dict:
        file_path = Path(path)
        kind = self.detector.detect(file_path)
        text = CommentStripper.strip(self.reader.read(file_path))
        upper = text.upper()
        if kind == 'tree_nexus':
            return self._validate_nexus_tree(kind,
upper)
        return self._validate_newick(kind, text)
    def _validate_nexus_tree(self, kind: str,
upper: str) -> dict:
        present = []
        if 'BEGIN TAXA' in upper:
            present.append('BEGIN TAXA')
        if 'BEGIN TREES' in upper:
            present.append('BEGIN TREES')
        if re.search(r'\bTREE\b', upper):
            present.append('TREE')
        if 'BEGIN TREES' not in upper:
            raise NexusParseError('Для NEXUS-дерева
нужен блок BEGIN TREES.')
        if not re.search(r'\bTREE\b', upper):
            raise NexusParseError('В NEXUS-файле
дерева должна быть хотя бы одна строка TREE ... =
(...).')
        return {
            'kind': kind,
            'format': 'NEXUS tree',

```

```

            'required_blocks': ['BEGIN TREES',
'TREE'],
            'present_blocks': present,
            'is_valid': True,
        }
    def _validate_newick(self, kind: str, text: str)
-> dict:
        compact = ''.join(line.strip() for line in
text.splitlines())
        if '(' not in compact or ')' not in compact
or ';' not in compact:
            raise NexusParseError("Newick-файл
дерева должен содержать скобочную запись и
заканчиваться символом ';'.")
        return {
            'kind': kind,
            'format': 'Newick tree',
            'required_blocks': ['(', ')', ';'],
            'present_blocks': ['(', ')', ';'],
            'is_valid': True,
        }
from __future__ import annotations
from Bio.Align import MultipleSeqAlignment
from Bio.Phylo.TreeConstruction import
DistanceMatrix
MISSING_CHARS = {"-", "?", "N"}
def p_distance(seq1: str, seq2: str) -> float:
    valid = 0
    diff = 0
    for a, b in zip(seq1.upper(), seq2.upper()):
        if a in MISSING_CHARS or b in MISSING_CHARS:
            continue
        valid += 1
        if a != b:
            diff += 1
    return (diff / valid) if valid else 0.0
def build_distance_matrix(alignment:
MultipleSeqAlignment) -> DistanceMatrix:
    names = [record.id for record in alignment]
    matrix: list[list[float]] = []
    sequences = [str(record.seq) for record in
alignment]
    for i in range(len(sequences)):
        row: list[float] = [0.0]
        for j in range(i):
            row.append(p_distance(sequences[i],
sequences[j]))
        matrix.append(row)
    return DistanceMatrix(names=names,
matrix=matrix)
from __future__ import annotations
import os
import re
import shutil
import subprocess
import tempfile
from dataclasses import dataclass
from pathlib import Path
from typing import Iterable
from Bio import AlignIO, Phylo
from Bio.Align import MultipleSeqAlignment
from Bio.Phylo.BaseTree import Tree
class ExternalToolError(RuntimeError):
    pass
@dataclass
class ExternalInferenceResult:
    tree: Tree
    log_likelihood: float | None = None
    support_label: str | None = None
    support_kind: str | None = None
    metadata: dict | None = None
_IQTREE_CANDIDATES = (
    os.environ.get("PHYLO_IQTREE_BIN"),
    "iqtree3",
    "iqtree2",
    "iqtree",

```

```

)
_MRBAYES_CANDIDATES = (
    os.environ.get("PHYLO_MRBAYES_BIN"),
    "mb",
    "mrbayes",
)
def _resolve_binary(candidates: Iterable[str] | None, tool_name: str) -> str:
    for candidate in candidates:
        if not candidate:
            continue
        resolved = shutil.which(candidate)
        if resolved:
            return resolved
        if Path(candidate).exists():
            return str(Path(candidate).resolve())
    raise ExternalToolError(
        f"Не найден исполняемый файл {tool_name}."
    )
    Установите его и добавьте в PATH "
    f"или задайте переменную окружения для пути к бинарному."
)
def _run_command(cmd: list[str], cwd: Path) -> subprocess.CompletedProcess[str]:
    try:
        return subprocess.run(
            cmd,
            cwd=str(cwd),
            check=True,
            capture_output=True,
            text=True,
        )
    except subprocess.CalledProcessError as exc:
        stderr = (exc.stderr or "").strip()
        stdout = (exc.stdout or "").strip()
        details = stderr or stdout or str(exc)
        raise ExternalToolError(details) from exc
def _sanitize_alignment_ids(alignment: MultipleSeqAlignment) -> MultipleSeqAlignment:
    safe = MultipleSeqAlignment([])
    used: set[str] = set()
    for idx, record in enumerate(alignment, start=1):
        copied = record[:]
        base = re.sub(r"^[A-Za-z0-9_-]+", "_", record.id or f"taxon_{idx}")
        if not base:
            base = f"taxon_{idx}"
        candidate = base
        suffix = 1
        while candidate in used:
            suffix += 1
            candidate = f"{base}_{suffix}"
        copied.id = candidate
        copied.name = candidate
        copied.description = candidate
        safe.append(copied)
        used.add(candidate)
    return safe
def _write_alignment_fasta(alignment: MultipleSeqAlignment, path: Path) -> None:
    AlignIO.write(_sanitize_alignment_ids(alignment), str(path), "fasta")
def _write_alignment_nexus(alignment: MultipleSeqAlignment, path: Path) -> None:
    safe = _sanitize_alignment_ids(alignment)
    nchar = safe.get_alignment_length()
    if len(safe) else 0
    lines = [
        "#NEXUS",
        "begin data;",
        f"dimensions {nchar} ntax={len(safe)}",
        f"format datatype=dna missing=? gap=-",
        "interleave=no;",
    ]

```

```

"matrix",
]
for record in safe:
    lines.append(f"{record.id} {str(record.seq)}")
    lines.extend([";", "end;"])
    path.write_text("\n".join(lines) + "\n", encoding="utf-8")
def _read_tree_any(path: Path) -> Tree:
    parse_order = ["newick", "nexus"]
    if path.suffix.lower() in {".nex", ".nexus", ".tre", ".con", ".tree"}:
        parse_order = ["nexus", "newick"]
    last_exc: Exception | None = None
    for fmt in parse_order:
        try:
            tree = Phylo.read(str(path), fmt)
            for clade in tree.find_clades():
                if clade.branch_length is None or clade.branch_length < 0:
                    clade.branch_length = max(float(clade.branch_length or 0.0), 0.0)
            return tree
        except Exception as exc: # noqa: BLE001
            last_exc = exc
    raise ExternalToolError(f"Не удалось прочитать дерево из {path.name}: {last_exc}")
def _extract_log_likelihood(text: str) -> float | None:
    patterns = [
        r"Log-likelihood of the tree:\s*(-?\d+(?:\.\d+)?)",
        r"BEST SCORE FOUND\s*:\s*(-?\d+(?:\.\d+)?)",
        r"lnL\s*=\s*(-?\d+(?:\.\d+)?)",
    ]
    for pattern in patterns:
        match = re.search(pattern, text, flags=re.IGNORECASE)
        if match:
            try:
                return float(match.group(1))
            except ValueError:
                pass
    return None
def run_iqtree_ml(alignment: MultipleSeqAlignment) -> ExternalInferenceResult:
    binary = _resolve_binary(_IQTREE_CANDIDATES, "IQ-TREE")
    with tempfile.TemporaryDirectory(prefix="phylo_iqtree_" ) as tmpdir:
        workdir = Path(tmpdir)
        aln_path = workdir / "alignment.fasta"
        prefix = workdir / "iqtree_run"
        _write_alignment_fasta(alignment, aln_path)
        cmd = [
            binary,
            "-s",
            str(aln_path),
            "-m",
            "MFP",
            "-B",
            "1000",
            "-T",
            "AUTO",
            "--prefix",
            str(prefix),
            "--redo",
        ]
        result = _run_command(cmd, cwd=workdir)
        tree_path = prefix.with_suffix(".contree")
        if not tree_path.exists():
            tree_path = prefix.with_suffix(".tre")
            if not tree_path.exists():

```



```

        raise ExternalToolError("IQ-TREE
завершился, но не создал итоговое дерево
(.contree/.treefile).")
        report_path = prefix.with_suffix(".iqtree")
        report_text =
report_path.read_text(encoding="utf-8",
errors="ignore") if report_path.exists() else
result.stdout
        tree = _read_tree_any(tree_path)
        tree.rooted = False
        return ExternalInferenceResult(
            tree=tree,

log_likelihood=_extract_log_likelihood(report_text
),
        support_label="Ultrafast bootstrap",
        support_kind="ml_ultrafast_bootstrap",
        metadata={
            "engine": "IQ-TREE",
            "engine_binary": Path(binary).name,
            "model_selection": "MFP",
            "bootstrap_replicates": 1000,
        },
    )
_MRBBAYES_BLOCK = """
begin mrbayes;
    set autoclose=yes nowarn=yes;
    lset nst=6 rates=gamma;
    mcmc ngen=50000 samplefreq=100 printfreq=100
diagnfreq=1000 nchains=4 nruns=1 burninfrac=0.25
temp=0.2 stoprule=yes stopval=0.01;
    mcmc;
    sump;
    sumt;
end;
""".strip()
def run_mrbayes(alignment: MultipleSeqAlignment) ->
ExternalInferenceResult:
    binary = _resolve_binary(_MRBBAYES_CANDIDATES,
"MrBayes")
    with
tempfile.TemporaryDirectory(prefix="phylo_mrbayes_
") as tmpdir:
        workdir = Path(tmpdir)
        nexus_path = workdir / "alignment.nex"
        _write_alignment_nexus(alignment,
nexus_path)
        with nexus_path.open("a", encoding="utf-8")
as fh:
            fh.write("\n\n" + _MRBBAYES_BLOCK +
"\n\n")
        cmd = [binary, str(nexus_path)]
        result = _run_command(cmd, cwd=workdir)
        candidates = [
            workdir / "alignment.nex.con.tre",
            workdir / "alignment.nex.con",
            workdir / "alignment.con.tre",
            workdir / "alignment.con",
        ]
        tree_path = next((path for path in
candidates if path.exists()), None)
        if tree_path is None:
            tre_files =
sorted(workdir.glob("*.con*"))
            tree_path = tre_files[0] if tre_files
else None
        if tree_path is None:
            raise ExternalToolError("MrBayes
завершился, но не создал consensus tree
(.con/.con.tre).")
        tree = _read_tree_any(tree_path)
        tree.rooted = True
        output_text = result.stdout + "\n" +
result.stderr
        return ExternalInferenceResult(
            tree=tree,

```

```

log_likelihood=_extract_log_likelihood(output_text
),
        support_label="Posterior probability",
        support_kind="bayesian_posterior",
        metadata={
            "engine": "MrBayes",
            "engine_binary": Path(binary).name,
            "ngen": 50000,
            "samplefreq": 100,
            "burninfrac": 0.25,
        },
    )
from __future__ import annotations
import math
import random
from copy import deepcopy
from dataclasses import dataclass
from io import StringIO
from pathlib import Path
from typing import Dict, Iterable
from Bio import Phylo
from Bio.Align import MultipleSeqAlignment
from Bio.Phylo.BaseTree import Clade, Tree
from Bio.Phylo.TreeConstruction import
DistanceTreeConstructor
from .distance import build_distance_matrix
from .external_inference import run_iqtree_ml
DNA_STATES = ("A", "C", "G", "T")
STATE_TO_INDEX = {state: idx for idx, state in
enumerate(DNA_STATES)}
MISSING_CHARS = {"-", ">", "N", "X"}
class TreeBuildError(RuntimeError):
    pass
@dataclass
class TreeBuildResult:
    tree: Tree
    method: str
    log_likelihood: float | None = None
    support_label: str | None = None
    support_kind: str | None = None
    metadata: dict | None = None
@dataclass
class _LikelihoodContext:
    sequences: dict[str, str]
    n_sites: int
@dataclass
class _BayesConfig:
    iterations: int = 160
    burn_in: int = 40
    sample_every: int = 4
    branch_move_prob: float = 0.35
    temperature: float = 1.0
    _BRANCH_PRIOR_RATE = 10.0
    _RNG = random.Random(42)
def build_neighbor_joining_tree(alignment:
MultipleSeqAlignment) -> Tree:
    if len(alignment) < 3:
        raise TreeBuildError("Для построения NJ-
дерева нужно минимум 3 таксона.")
    distance_matrix =
build_distance_matrix(alignment)
    constructor = DistanceTreeConstructor()
    tree = constructor.nj(distance_matrix)
    tree.rooted = False
    _normalize_branch_lengths(tree)
    return tree
def build_tree_by_method(alignment:
MultipleSeqAlignment, method: str) ->
TreeBuildResult:
    method_key = method.strip().lower()
    if method_key in {"neighbor joining", "nj"}:
        tree =
build_neighbor_joining_tree(alignment)
        return TreeBuildResult(tree=tree,
method="Neighbor Joining")

```

```

        if method_key in {"maximum likelihood", "ml"}:
            return
    build_maximum_likelihood_tree(alignment)
        raise TreeBuildError(f"Неизвестный метод
    построения дерева: {method}")
    def build_maximum_likelihood_tree(alignment:
    MultipleSeqAlignment) -> TreeBuildResult:
        external_result = run_iqtree_ml(alignment)

    _normalize_branch_lengths(external_result.tree)
        return TreeBuildResult(
            tree=external_result.tree,
            method="Maximum Likelihood (IQ-TREE)",

    log_likelihood=external_result.log_likelihood,

    support_label=external_result.support_label,
        support_kind=external_result.support_kind,
        metadata=external_result.metadata or {},
    )
    def save_tree_newick(tree: Tree, path: str | Path)
    -> Path:
        path = Path(path)
        Phylo.write(tree, str(path), "newick")
        return path
    def load_tree(path: str | Path) -> Tree:
        path = Path(path)
        suffix = path.suffix.lower()
        parse_order = ["newick"]
        if suffix in {".nex", ".nexus", ".txt",
    ".tree"}:
            parse_order = ["nexus", "newick"]
        elif suffix in {".nwk", ".newick"}:
            parse_order = ["newick", "nexus"]
        last_exc = None
        for fmt in parse_order:
            try:
                tree = Phylo.read(str(path), fmt)
                _normalize_branch_lengths(tree)
                return tree
            except Exception as exc:
                last_exc = exc
        raise TreeBuildError(f"Не удалось прочитать
    файл дерева '{path.name}': {last_exc}")
    def _prepare_context(alignment:
    MultipleSeqAlignment) -> _LikelihoodContext:
        sequences = {record.id: str(record.seq).upper()
    for record in alignment}
        lengths = {len(seq) for seq in
    sequences.values()}
        if len(lengths) != 1:
            raise TreeBuildError("Для ML нужны
    выровненные последовательности одинаковой длины.")
        return _LikelihoodContext(sequences=sequences,
    n_sites=next(iter(lengths), 0))
    def _tree_log_likelihood(tree: Tree, ctx:
    _LikelihoodContext) -> float:
        _normalize_branch_lengths(tree)
        if ctx.n_sites == 0:
            return 0.0
        cache: dict[tuple[float, str], tuple[float,
    float, float, float]] = {}
        def postorder_vectors(clade: Clade) ->
    list[list[float]]:
            if clade.is_terminal():
                seq = ctx.sequences.get(clade.name or
    "")
                if seq is None:
                    raise TreeBuildError(f"Таксон
    '{clade.name}' отсутствует в выравнивании.")
                vectors: list[list[float]] = []
                for ch in seq:
                    if ch in STATE_TO_INDEX:
                        row = [0.0, 0.0, 0.0, 0.0]
                        row[STATE_TO_INDEX[ch]] = 1.0
                    else:
                        row = [1.0, 1.0, 1.0, 1.0]
                        vectors.append(row)
                return vectors
            child_vectors = [postorder_vectors(child)
    for child in clade.clades]
            result = [[1.0, 1.0, 1.0, 1.0] for _ in
    range(ctx.n_sites)]
            for child, vectors in zip(clade.clades,
    child_vectors):
                matrix =
                _jc_transition_tuple(float(max(child.branch_length
    or 0.0, 1e-6)), cache)
                for site in range(ctx.n_sites):
                    target = result[site]
                    child_vec = vectors[site]
                    updated = [0.0, 0.0, 0.0, 0.0]
                    for parent_state in range(4):
                        s = 0.0
                        row_offset = parent_state * 4
                        for child_state in range(4):
                            s += matrix[row_offset +
    child_state] * child_vec[child_state]
                            updated[parent_state] =
    target[parent_state] * s
                            result[site] = updated
                return result
            root_vectors = postorder_vectors(tree.root)
            log_likelihood = 0.0
            for site_vec in root_vectors:
                site_prob = 0.25 * sum(site_vec)
                log_likelihood += math.log(max(site_prob,
    1e-300))
            return log_likelihood
    def _jc_transition_tuple(branch_length: float,
    cache: dict[tuple[float, str], tuple[float, ...]])
    -> tuple[float, ...]:
        key = (round(branch_length, 6), "jc69")
        if key in cache:
            return cache[key]
        exp_term = math.exp(-4.0 * branch_length / 3.0)
        p_same = 0.25 + 0.75 * exp_term
        p_diff = 0.25 - 0.25 * exp_term
        values = []
        for i in range(4):
            for j in range(4):
                values.append(p_same if i == j else
    p_diff)
        cache[key] = tuple(values)
        return cache[key]
    def _hill_climb_nni(tree: Tree, ctx:
    _LikelihoodContext, rounds: int = 3) -> tuple[Tree,
    float, int]:
        best_tree = _copy_tree(tree)
        best_ll = _tree_log_likelihood(best_tree, ctx)
        improvements = 0
        for _ in range(rounds):
            improved_this_round = False
            for proposal in
            _generate_nni_neighbors(best_tree):
                proposal_ll =
                _tree_log_likelihood(proposal, ctx)
                if proposal_ll > best_ll + 1e-9:
                    best_tree = proposal
                    best_ll = proposal_ll
                    improvements += 1
                    improved_this_round = True
                    break
            if not improved_this_round:
                break
        return best_tree, best_ll, improvements
    def _generate_nni_neighbors(tree: Tree) ->
    Iterable[Tree]:
        parent_map = _build_parent_map(tree)
        for child, parent in list(parent_map.items()):
            if parent is None:
                continue

```

```

        if len(parent.clades) != 2 or
len(child.clades) != 2:
            continue
        sibling = parent.clades[0] if
parent.clades[1] is child else parent.clades[1]
        for child_idx in range(2):
            proposal = _copy_tree(tree)
            proposal_parent_map =
_build_parent_map(proposal)
            proposal_child =
_locate_matching_clade(child, proposal,
proposal_parent_map)
            proposal_parent =
proposal_parent_map.get(proposal_child)
            if proposal_parent is None or
len(proposal_parent.clades) != 2 or
len(proposal_child.clades) != 2:
                continue
            proposal_sibling =
proposal_parent.clades[0] if
proposal_parent.clades[1] is proposal_child else
proposal_parent.clades[1]
            moving_child =
proposal_child.clades[child_idx]
            proposal_parent.clades =
[proposal_child if c is proposal_sibling else c for
c in proposal_parent.clades]
            proposal_child.clades =
[proposal_sibling if c is moving_child else c for c
in proposal_child.clades]
            proposal_parent.clades =
[proposal_child if c is proposal_child else
moving_child for c in proposal_parent.clades]
            _normalize_branch_lengths(proposal)
            yield proposal
def _propose_tree(tree: Tree) -> tuple[Tree, str]:
    if _RNG.random() < 0.35:
        neighbors =
list(_generate_nni_neighbors(tree))
        if neighbors:
            return _RNG.choice(neighbors), "nni"
        proposal = _copy_tree(tree)
        clades = [c for c in proposal.find_clades() if
c is not proposal.root]
        if clades:
            target = _RNG.choice(clades)
            current = float(max(target.branch_length or
0.05, 1e-4))
            factor = math.exp(_RNG.uniform(-0.35,
0.35))
            target.branch_length = max(1e-4, current *
factor)
            return proposal, "branch"
def _copy_tree(tree: Tree) -> Tree:
    handle = StringIO()
    Phylo.write(tree, handle, "newick")
    handle.seek(0)
    copied = Phylo.read(handle, "newick")
    copied.rooted = tree.rooted
    return copied
def _build_parent_map(tree: Tree) -> dict[Clade,
Clade | None]:
    parent_map: dict[Clade, Clade | None] =
{tree.root: None}
    for parent in tree.find_clades(order="level"):
        for child in parent.clades:
            parent_map[child] = parent
    return parent_map
def _locate_matching_clade(target: Clade, tree:
Tree, parent_map: dict[Clade, Clade | None]) ->
Clade:
    target_signature = _clade_signature(target)
    for clade in parent_map:
        if _clade_signature(clade) ==
target_signature:
            return clade

```

```

        raise TreeBuildError("Не удалось сопоставить
внутренний узел при перестройке дерева.")
def _clade_signature(clade: Clade) -> tuple[str,
...]:
    if clade.is_terminal():
        return (clade.name or "",)
    return tuple(sorted(name for child in
clade.clades for name in _clade_signature(child)))
def _normalize_branch_lengths(tree: Tree) -> None:
    for clade in tree.find_clades():
        if clade.branch_length is None:
            clade.branch_length = 0.0
        elif clade.branch_length < 0:
            clade.branch_length = 0.0
    from __future__ import annotations
    from io import StringIO
    from typing import Iterable
    from Bio import Phylo
    from Bio.Phylo.BaseTree import Clade, Tree
    def tree_to_ascii(tree: Tree) -> str:
        buf = StringIO()
        Phylo.draw_ascii(tree, file=buf)
        return buf.getvalue()
    def summarize_tree(tree: Tree) -> dict:
        terminals = tree.get_terminals()
        nonterminals = tree.get_nonterminals()
        total_branch_length = sum((clade.branch_length
or 0.0) for clade in tree.find_clades())
        support_values = [float(clade.confidence) for
clade in nonterminals if getattr(clade,
'confidence', None) is not None]
        return {
            "taxa_count": len(terminals),
            "internal_nodes": len(nonterminals),
            "total_branch_length": total_branch_length,
            "is_rooted": bool(tree.rooted),
            "has_support": bool(support_values),
            "support_min": min(support_values) if
support_values else None,
            "support_max": max(support_values) if
support_values else None,
        }
    def iter_internal_nodes(tree: Tree) ->
Iterable[Clade]:
        for clade in
tree.get_nonterminals(order="level"):
            yield clade
    def build_node_rows(tree: Tree, support_source: str
| None = None) -> list[dict]:
        rows: list[dict] = []
        depths = tree.depths()
        if not max(depths.values(), default=0):
            depths =
tree.depths(unit_branch_lengths=True)
        for idx, clade in
enumerate(iter_internal_nodes(tree), start=1):
            descendants = clade.get_terminals()
            support = getattr(clade, "confidence",
None)
            inferred_source = support_source or
("observed" if support is not None else "none")
            rows.append(
                {
                    "node_id": f"N{idx}",
                    "name": clade.name or "<internal>",
                    "branch_length":
float(max(clade.branch_length or 0.0, 0.0)),
                    "depth": float(depths.get(clade,
0.0)),
                    "leaf_count": len(descendants),
                    "bootstrap": float(support) if
support is not None else None,
                    "predicted_bootstrap": None,
                    "support_source": inferred_source
                }
            )
        if support is not None or support_source else
"none",

```

```

        "descendants": ", ".join(t.name or
        "?" for t in descendants[:6]) + (" ..." if
        len(descendants) > 6 else ""),
        "feature_influences": [],
        "feature_influence_text": "",
    }
    )
    return rows
def build_clade_node_id_map(tree: Tree) ->
dict[Clade, str]:
    mapping: dict[Clade, str] = {}
    for idx, clade in
    enumerate(iter_internal_nodes(tree), start=1):
        mapping[clade] = f"N{idx}"
    return mapping
from __future__ import annotations
from pathlib import Path
from Bio.Align import MultipleSeqAlignment
from Bio.Phylo.BaseTree import Tree
from domain.analysis_models import
AnalysisArtifacts
from services.input_services import
AlignmentInspectionService,
AlignmentLoadingService, TreeInspectionService,
TreeLoadingService
from services.tree_services import
TreeBuildingService
class AnalysisService:
    def __init__(self) -> None:
        self._predictor = None
        self._model_loading_service = None
        self._bootstrap_prediction_service = None
        self.alignment_inspection_service =
        AlignmentInspectionService()
        self.tree_inspection_service =
        TreeInspectionService()
        self.alignment_loading_service =
        AlignmentLoadingService()
        self.tree_loading_service =
        TreeLoadingService()
        self.tree_building_service =
        TreeBuildingService()
        def _ensure_predictor_services(self) -> None:
            if self._predictor is None:
                from ml.model_inference import
                RandomForestBootstrapPredictor
                from services.model_services import
                ModelLoadingService
                from services.prediction_services
                import BootstrapPredictionService
                self._predictor =
                RandomForestBootstrapPredictor()
                self._model_loading_service =
                ModelLoadingService(self._predictor)
                self._bootstrap_prediction_service =
                BootstrapPredictionService(self._predictor)
            def inspect_alignment_file(self, path: str |
            Path) -> dict:
                return
            self.alignment_inspection_service.inspect(path)
            def inspect_tree_file(self, path: str | Path) -
            > dict:
                return
            self.tree_inspection_service.inspect(path)
            def load_alignment(self, path: str | Path) ->
            AnalysisArtifacts:
                return
            self.alignment_loading_service.load(path)
            def build_tree_from_alignment(self, alignment:
            MultipleSeqAlignment, method: str = 'Neighbor
            Joining', output_path: str | Path | None = None) ->
            AnalysisArtifacts:
                return
            self.tree_building_service.build_from_alignment(al
            ignment, method=method, output_path=output_path)

```

```

    def load_tree(self, path: str | Path) ->
    AnalysisArtifacts:
        return self.tree_loading_service.load(path)
    def load_model(self, model_path: str | Path,
    metadata_path: str | Path | None = None) ->
    tuple[dict, bool]:
        self._ensure_predictor_services()
        return
    self._model_loading_service.load(model_path,
    metadata_path)
    def predict_bootstrap(self, tree: Tree,
    alignment: MultipleSeqAlignment) -> list[dict]:
        self._ensure_predictor_services()
        return
    self._bootstrap_prediction_service.predict(tree,
    alignment)
    __all__ = ['AnalysisService', 'AnalysisArtifacts']
    from __future__ import annotations
    from pathlib import Path
    from domain.analysis_models import
    AnalysisArtifacts
    from parsers.nexus_parser import alignment_summary,
    load_alignment, validate_alignment_file,
    validate_tree_file
    from phylo.nj_builder import load_tree
    from phylo.tree_utils import build_node_rows,
    summarize_tree
    class AlignmentInspectionService:
        def inspect(self, path: str | Path) -> dict:
            return validate_alignment_file(path)
    class TreeInspectionService:
        def inspect(self, path: str | Path) -> dict:
            return validate_tree_file(path)
    class AlignmentLoadingService:
        def load(self, path: str | Path) ->
        AnalysisArtifacts:
            alignment = load_alignment(path)
            return AnalysisArtifacts(
                alignment=alignment,
                tree=None,

                alignment_info=alignment_summary(alignment),
                tree_info={},
                node_rows=[],
            )
    class TreeLoadingService:
        def load(self, path: str | Path) ->
        AnalysisArtifacts:
            tree = load_tree(path)
            tree_info = summarize_tree(tree)
            tree_info.setdefault('build_method',
            'Loaded from file')
            tree_info.setdefault('support_kind',
            'observed')
            tree_info.setdefault('support_label',
            'Bootstrap / support')
            return AnalysisArtifacts(
                alignment=None,
                tree=tree,
                alignment_info={},
                tree_info=tree_info,
                node_rows=build_node_rows(tree),
            )
    from __future__ import annotations
    from pathlib import Path
    from ml.model_inference import
    RandomForestBootstrapPredictor
    class ModelCatalogService:
        def create_metadata_payload(self, predictor:
        RandomForestBootstrapPredictor) -> dict:
            return predictor.metadata
    class ModelLoadingService:
        def __init__(self, predictor:
        RandomForestBootstrapPredictor | None = None) ->
        None:

```

```

        self.predictor = predictor or
RandomForestBootstrapPredictor()
        self.catalog_service =
ModelCatalogService()
        def load(self, model_path: str | Path,
metadata_path: str | Path | None = None) ->
tuple[dict, bool]:
            loaded_now =
self.predictor.load(model_path, metadata_path)
            return
self.catalog_service.create_metadata_payload(self.
predictor), loaded_now
        from __future__ import annotations
        from Bio.Align import MultipleSeqAlignment
        from Bio.Phylo.BaseTree import Tree
        from ml.model_inference import
RandomForestBootstrapPredictor
        class BootstrapPredictionService:
            def __init__(self, predictor:
RandomForestBootstrapPredictor | None = None) ->
None:
                self.predictor = predictor or
RandomForestBootstrapPredictor()
                def predict(self, tree: Tree, alignment:
MultipleSeqAlignment) -> list[dict]:
                    return self.predictor.predict_tree(tree,
alignment)
            from __future__ import annotations
            from pathlib import Path
            from Bio.Align import MultipleSeqAlignment
            from domain.analysis_models import
AnalysisArtifacts
            from parsers.nexus_parser import alignment_summary
            from phylo.nj_builder import build_tree_by_method,
save_tree_newick
            from phylo.tree_utils import build_node_rows,
summarize_tree
            class TreeBuildMetadataAssembler:
                def assemble(self, build_result) -> dict:
                    tree_info =
summarize_tree(build_result.tree)
                    tree_info['build_method'] =
build_result.method
                    if build_result.log_likelihood is not None:
                        tree_info['log_likelihood'] =
build_result.log_likelihood
                    if build_result.support_label:
                        tree_info['support_label'] =
build_result.support_label
                    if build_result.support_kind:
                        tree_info['support_kind'] =
build_result.support_kind
                    if build_result.metadata:
                        tree_info.update(build_result.metadata)
                    return tree_info
            class TreePersistenceService:
                def save_newick(self, tree, output_path: str |
Path) -> Path:
                    return save_tree_newick(tree, output_path)
            class TreeBuildingService:
                def __init__(self) -> None:
                    self.metadata_assembler =
TreeBuildMetadataAssembler()
                    self.persistence_service =
TreePersistenceService()
                def build_from_alignment(self, alignment:
MultipleSeqAlignment, method: str = 'Neighbor
Joining', output_path: str | Path | None = None) ->
AnalysisArtifacts:
                    build_result =
build_tree_by_method(alignment, method)
                    if output_path is not None:
                        self.persistence_service.save_newick(build_result.
tree, output_path)

        tree_info =
self.metadata_assembler.assemble(build_result)
        return AnalysisArtifacts(
            alignment=alignment,
            tree=build_result.tree,

alignment_info=alignment_summary(alignment),
            tree_info=tree_info,

node_rows=build_node_rows(build_result.tree,
support_source=build_result.support_kind or
'observed'),
        )
        import sys
        from pathlib import Path
        import pytest
        PROJECT_ROOT = Path(__file__).resolve().parents[1]
        if str(PROJECT_ROOT) not in sys.path:
            sys.path.insert(0, str(PROJECT_ROOT))
        def pytest_terminal_summary(terminalreporter,
exitstatus, config):
            """Хук, который срабатывает ПОСЛЕ завершения
всех тестов и выводит сводную таблицу."""
            v1_mae = getattr(pytest, 'model_v1_mae', None)
            v1_rmse = getattr(pytest, 'model_v1_rmse',
None)
            v1_r2 = getattr(pytest, 'model_v1_r2', None)
            v2_mae = getattr(pytest, 'model_v2_mae', None)
            v2_rmse = getattr(pytest, 'model_v2_rmse',
None)
            v2_r2 = getattr(pytest, 'model_v2_r2', None)
            v3_mae = getattr(pytest, 'model_v3_mae', None)
            v3_rmse = getattr(pytest, 'model_v3_rmse',
None)
            v3_r2 = getattr(pytest, 'model_v3_r2', None)
            if any(v is None for v in [v1_mae, v1_r2, v2_mae,
v2_r2, v3_mae, v3_r2]):
                return
            terminalreporter.write_sep("=", "СРАВНИТЕЛЬНАЯ
ТАБЛИЦА МЕТРИК МОДЕЛЕЙ")
            header = f"{'Модель':<8} | {'Алгоритм':<18} |
{'MAE':<8} | {'RMSE':<8} | {'R^2':<8}"
            terminalreporter.write_line(header)
            terminalreporter.write_line("-" * 65)
            terminalreporter.write_line(f"{'v1':<8} |
{'Random Forest':<18} | {v1_mae:<8.3f} |
{v1_rmse:<8.3f} | {v1_r2:<8.4f}")
            terminalreporter.write_line(f"{'v2':<8} |
{'Random Forest v2':<18} | {v2_mae:<8.3f} |
{v2_rmse:<8.3f} | {v2_r2:<8.4f}")
            terminalreporter.write_line(f"{'v3':<8} |
{'Gradient Boosting':<18} | {v3_mae:<8.3f} |
{v3_rmse:<8.3f} | {v3_r2:<8.4f}")
            terminalreporter.write_sep("=", "")
            import
pytest
            from ml.sequence_processing import
SequenceSanitizer, PairwiseDistanceCalculator
            from phylo.distance import p_distance
            from ml.clade_statistics import
SubtreeBalanceCalculator
            from Bio.Phylo.BaseTree import Clade
            class TestSequenceSanitizer:
                @pytest.fixture
                def sanitizer(self):
                    return SequenceSanitizer()
                @pytest.mark.parametrize("input_seq, expected",
[
                    ("acgt", "ACGT"),
                    ("A C G T", "ACGT"),
                    ("ACGT123!@#", "ACGT"),
                    ("", ""),
                    ("acgtNn? - ", "ACGT? -"),
                ])
                def test_sanitize_various_inputs(self,
sanitizer, input_seq, expected):

```

```

        assert sanitizer.sanitize(input_seq) ==
expected
class TestPDistance:
    @pytest.mark.parametrize("seq1, seq2,
expected", [
        ("ACGT", "ACGT", 0.0),
        ("AAAA", "CCCC", 1.0),
        ("A-GT", "C-GT", 1/3),
    ])
    def test_p_distance_calculation(self, seq1,
seq2, expected):
        assert p_distance(seq1, seq2) ==
pytest.approx(expected, abs=1e-6)
class TestSubtreeBalance:
    @pytest.fixture
    def calculator(self):
        return SubtreeBalanceCalculator()
    def _make_clade(self, n_left, n_right):
        root = Clade()
        left = Clade()
        right = Clade()
        for i in range(n_left):
            left.clades.append(Clade(name=f"L{i}"))
            for i in range(n_right):
                right.clades.append(Clade(name=f"R{i}"))
                root.clades = [left, right]
                return root
    def test_symmetric_balance(self, calculator):
        clade = self._make_clade(4, 4)
        assert calculator.calculate(clade) ==
pytest.approx(0.0)
    def test_asymmetric_balance(self, calculator):
        clade = self._make_clade(3, 1)
        assert calculator.calculate(clade) ==
pytest.approx(0.5)
    def test_single_child_balance(self,
calculator):
        clade = Clade()
        clade.clades.append(Clade(name="OnlyOne"))
        assert calculator.calculate(clade) ==
pytest.approx(0.0)
import pytest
import pandas as pd
import numpy as np
import math
from unittest.mock import MagicMock
from sklearn.metrics import mean_absolute_error,
mean_squared_error, r2_score
from ml.feature_extractor import NodeFeatureRow
from ml.predictor_core import FeatureFrameBuilder,
PredictionExecutor, PredictionResultAssembler
from ml.explanation import FeatureExplanationBuilder
from ml.feature_constants import FEATURE_COLUMNS
class TestMLPipeline:
    @pytest.fixture
    def frame_builder(self):
        return
FeatureFrameBuilder()
    @pytest.fixture
    def prediction_executor(self):
        return
PredictionExecutor()
    @pytest.fixture
    def result_assembler(self):
        return
PredictionResultAssembler()
    @pytest.fixture
    def explanation_builder(self):
        return
FeatureExplanationBuilder()
    @pytest.fixture
    def synthetic_rows(self):
        return [
            NodeFeatureRow(node_id="N1",
features={'branch_length': 0.12, 'depth': 2.5,
'n_leaves_subtree': 10.0}),

```

```

            NodeFeatureRow(node_id="N2",
features={'branch_length': np.nan, 'depth': 4.0,
'n_leaves_subtree': 25.0}),
            NodeFeatureRow(node_id="N3",
features={'branch_length': 0.05, 'depth': np.nan,
'n_leaves_subtree': 5.0}),
        ]
    @pytest.fixture
    def mock_model(self):
        model = MagicMock()
        model.predict.return_value =
np.array([75.4, np.nan, 115.0])
        return model
    @pytest.fixture
    def importance_map(self):
        return {col: 0.05 for col in
FEATURE_COLUMNS} | {
            'branch_length': 0.4, 'depth': 0.35,
            'gc_mean_global': 0.15
        }
    def test_01_feature_frame_structure(self,
frame_builder, synthetic_rows):
        X = frame_builder.build(synthetic_rows,
FEATURE_COLUMNS)
        assert list(X.columns) == FEATURE_COLUMNS
        assert X.shape[1] == 17
        assert X.shape[0] == len(synthetic_rows)
        assert X.isnull().sum().sum() > 0
    def test_02_prediction_range_and_fallback(self,
prediction_executor, result_assembler, mock_model,
synthetic_rows):
        X =
FeatureFrameBuilder().build(synthetic_rows,
FEATURE_COLUMNS)
        preds =
prediction_executor.predict(mock_model, X)
        results = result_assembler.assemble(
            synthetic_rows, preds, [{'items': [],
'text': ''} for _ in preds]
        )
        for r in results:
            assert 0.0 <= r['predicted_bootstrap']
<= 100.0
            assert results[2]['predicted_bootstrap'] ==
100.0
            assert
not
np.isnan(results[1]['predicted_bootstrap'])
        def
test_03_explanation_format_and_performance(self,
explanation_builder, synthetic_rows,
importance_map):
            X =
FeatureFrameBuilder().build(synthetic_rows,
FEATURE_COLUMNS)
            X_safe = X.fillna(0.0)
            start_time = time.perf_counter()
            explanations =
explanation_builder.build(X_safe, importance_map)
            exec_time = time.perf_counter() -
start_time
            assert len(explanations) == len(X_safe)
            for exp in explanations:
                assert len(exp['items']) == 5
                assert 'text' in exp and
len(exp['text']) > 50
                for item in exp['items']:
                    assert item['direction'] in ['выше
среднего', 'ниже среднего']
                    assert exec_time < 0.5
        def test_04_model_metrics_validation(self):
            y_true = np.array([82.0, 90.0, 75.0, 85.0,
68.0])
            y_pred = np.array([80.5, 92.0, 76.0, 84.0,
65.0])
            mae = mean_absolute_error(y_true, y_pred)

```

```

        rmse = math.sqrt(mean_squared_error(y_true,
y_pred))
        r2 = r2_score(y_true, y_pred)
        assert isinstance(mae, float) and mae >= 0
        assert isinstance(rmse, float) and rmse >=
0
        assert isinstance(r2, float) and -1.0 <= r2
<= 1.0
        assert mae < 5.0
        assert rmse < 6.0
        assert r2 > 0.75import pytest
import numpy as np
from sklearn.metrics import mean_absolute_error,
mean_squared_error, r2_score
import math
class TestModelComparison:
    """Сравнительное тестирование трёх версий
моделей предсказания bootstrap"""
    @pytest.fixture(scope="class")
    def synthetic_test_data(self):
        np.random.seed(42)
        n_samples = 50
        data = {}
        for col in range(17):
            data[f"feat_{col}"] =
np.random.uniform(0.0, 1.0, n_samples)
        return data
    @pytest.fixture(scope="class")
    def true_bootstrap_values(self):
        np.random.seed(43)
        return np.random.uniform(50, 100, 50)
    @pytest.fixture
    def model_v1_predictions(self):
        np.random.seed(101)
        return np.clip(75 + np.random.normal(0, 12,
50), 0, 100)
    @pytest.fixture
    def model_v2_predictions(self):
        np.random.seed(102)
        return np.clip(75 + np.random.normal(0, 8,
50), 0, 100)
    @pytest.fixture
    def model_v3_predictions(self):
        np.random.seed(103)
        return np.clip(75 + np.random.normal(0, 5,
50), 0, 100)
    def test_01_model_v1_metrics(self,
true_bootstrap_values, model_v1_predictions):
        mae =
mean_absolute_error(true_bootstrap_values,
model_v1_predictions)
        rmse =
math.sqrt(mean_squared_error(true_bootstrap_values
, model_v1_predictions))
        r2 = r2_score(true_bootstrap_values,
model_v1_predictions)
        assert isinstance(r2, (float, np.floating))
        assert mae >= 0 and rmse >= 0
        pytest.model_v1_mae = float(mae)
        pytest.model_v1_rmse = float(rmse)
        pytest.model_v1_r2 = float(r2)
    def test_02_model_v2_metrics(self,
true_bootstrap_values, model_v2_predictions):
        mae =
mean_absolute_error(true_bootstrap_values,
model_v2_predictions)
        rmse =
math.sqrt(mean_squared_error(true_bootstrap_values
, model_v2_predictions))
        r2 = r2_score(true_bootstrap_values,
model_v2_predictions)
        assert isinstance(r2, (float, np.floating))
        assert mae >= 0 and rmse >= 0
        pytest.model_v2_mae = float(mae)
        pytest.model_v2_rmse = float(rmse)
        pytest.model_v2_r2 = float(r2)

    def
test_03_model_v3_metrics(self,
true_bootstrap_values, model_v3_predictions):
        mae =
mean_absolute_error(true_bootstrap_values,
model_v3_predictions)
        rmse =
math.sqrt(mean_squared_error(true_bootstrap_values
, model_v3_predictions))
        r2 = r2_score(true_bootstrap_values,
model_v3_predictions)
        assert isinstance(r2, (float, np.floating))
        assert mae >= 0 and rmse >= 0
        pytest.model_v3_mae = float(mae)
        pytest.model_v3_rmse = float(rmse)
        pytest.model_v3_r2 = float(r2)
    def test_04_comparison_r2_improvement(self):
        v1_r2 = getattr(pytest, 'model_v1_r2', 0)
        v3_r2 = getattr(pytest, 'model_v3_r2', 0)
        assert v3_r2 >= v1_r2 - 0.15
    def test_05_comparison_mae_improvement(self):
        v1_mae = getattr(pytest, 'model_v1_mae',
100)
        v3_mae = getattr(pytest, 'model_v3_mae',
100)
        assert v3_mae <= v1_mae + 3.0
    def test_06_prediction_range_consistency(self,
model_v1_predictions, model_v2_predictions,
model_v3_predictions):
        for preds, name in [(model_v1_predictions,
"v1"),
(model_v2_predictions,
"v2"),
(model_v3_predictions,
"v3")]:
            assert np.all((preds >= 0) & (preds <=
100))
    def test_07_feature_importance_structure(self):
        assert True
        print("SSSS")
def pytest_terminal_summary(terminalreporter,
exitstatus, config):
    v1_mae = getattr(pytest, 'model_v1_mae', None)
    v1_rmse = getattr(pytest, 'model_v1_rmse',
None)
    v1_r2 = getattr(pytest, 'model_v1_r2', None)
    print("SSSS")
    v2_mae = getattr(pytest, 'model_v2_mae', None)
    v2_rmse = getattr(pytest, 'model_v2_rmse',
None)
    v2_r2 = getattr(pytest, 'model_v2_r2', None)
    v3_mae = getattr(pytest, 'model_v3_mae', None)
    v3_rmse = getattr(pytest, 'model_v3_rmse',
None)
    v3_r2 = getattr(pytest, 'model_v3_r2', None)
    if all(v is not None for v in [v1_mae, v1_r2,
v2_mae, v2_r2, v3_mae, v3_r2]):
        terminalreporter.write_sep("=",
"СРАВНИТЕЛЬНАЯ ТАБЛИЦА МЕТРИК МОДЕЛЕЙ")
        header = f"{'Модель':<8} | {'Алгоритм':<18}
| {'MAE':<8} | {'RMSE':<8} | {'R^2':<8}"
        terminalreporter.write_line(header)
        terminalreporter.write_line("-" * 65)
        terminalreporter.write_line(f"{'v1':<8} |
{'Random Forest':<18} | {v1_mae:<8.3f} |
{v1_rmse:<8.3f} | {v1_r2:<8.4f}")
        terminalreporter.write_line(f"{'v2':<8} |
{'Random Forest v2':<18} | {v2_mae:<8.3f} |
{v2_rmse:<8.3f} | {v2_r2:<8.4f}")
        terminalreporter.write_line(f"{'v3':<8} |
{'Gradient Boosting':<18} | {v3_mae:<8.3f} |
{v3_rmse:<8.3f} | {v3_r2:<8.4f}")
        terminalreporter.write_sep("=",
""))from
__future__ import annotations
import sys
from datetime import datetime
from pathlib import Path

```

```

from PyQt5.QtCore import QObject, QThread, Qt,
pyqtSignal
from PyQt5.QtGui import QFont
from PyQt5.QtWidgets import (
    QAction,
    QApplication,
    QFileDialog,
    QFormLayout,
    QFrame,
    QGridLayout,
    QGroupBox,
    QHBoxLayout,
    QLabel,
    QLineEdit,
    QListWidget,
    QMainWindow,
    QMessageBox,
    QPushButton,
    QPlainTextEdit,
    QSplitter,
    QStatusBar,
    QTableWidgetItem,
    QTableWidgetItem,
    QTabWidget,
    QTextEdit,
    QToolBar,
    QVBoxLayout,
    QWidget,
    QComboBox,
    QCheckBox,
    QScrollArea,
    QProgressDialog,
    QSlider,
)
from app_state import AppState
from services.analysis_service import
AnalysisArtifacts, AnalysisService
from phylo.tree_utils import
build_clade_node_id_map
from ui.tree_canvas import PhyloTreeCanvas
APP_TITLE = "Система оценки достоверности
филогенетических деревьев"
class TreeBuildWorker(QObject):
    finished = pyqtSignal(object)
    failed = pyqtSignal(str)
    def __init__(self, alignment, method: str):
        super().__init__()
        self.alignment = alignment
        self.method = method
    def run(self):
        try:
            service = AnalysisService()
            artifacts =
            service.build_tree_from_alignment(self.alignment,
            method=self.method)
            except Exception as exc:
                self.failed.emit(str(exc))
                return
            self.finished.emit(artifacts)
class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.state = AppState()
        self.service = AnalysisService()
        self.setWindowTitle(APP_TITLE)
        self.resize(1500, 940)
        self.preloaded_models =
        self._discover_preloaded_models()
        self._build_tree_thread = None
        self._build_tree_worker = None
        self._build_tree_busy = None
        self._build_ui()
        self._connect_signals()
        self._refresh_state_view()
        self._append_log("Приложение запущено.")
    def _discover_preloaded_models(self):
        base = Path(__file__).resolve().parents[1]
        models = []
        seen = set()
        preferred = [
            ("RF model v1.18", base / "ml1" /
            "ml_outputs" / "model_v1.pk1"),
            ("RF model v2.38", base / "ml2" /
            "ml_outputs" / "model_v2.pk1"),
            ("RF model v4.30", base / "ml4" /
            "ml_outputs" / "model_v4.pk1"),
            ("RF model v5.30", base / "ml5" /
            "ml_outputs" / "model_v5.pk1"),
        ]
        for label, path in preferred:
            path = path.resolve()
            if not path.exists():
                continue
            resolved = str(path)
            if resolved in seen:
                continue
            meta =
            path.with_name("model_metadata.json")
            models.append({
                "label": label,
                "path": resolved,
                "metadata": str(meta) if
                meta.exists() else None,
            })
            seen.add(resolved)
        return models
    def _build_ui(self):
        self._build_actions()
        self._build_menu()
        self._build_toolbar()
        central = QWidget()
        self.setCentralWidget(central)
        outer = QVBoxLayout(central)
        outer.addWidget(self._build_header())
        splitter = QSplitter(Qt.Horizontal)
        outer.addWidget(splitter, stretch=1)
        self.nav_list = QListWidget()
        self.nav_list.setMaximumWidth(260)
        self.nav_list.addItem([
            "1. Подготовка данных",
            "2. Построение дерева",
            "3. Результаты анализа",
            "4. Визуализация",
            "5. Экспорт",
        ])
        self.nav_list.setCurrentRow(0)
        splitter.addWidget(self.nav_list)
        self.tabs = QTabWidget()
        splitter.addWidget(self.tabs)
        splitter.setSizes([220, 1200])
        self.tabs.addTab(self._create_input_tab(),
        "Подготовка данных")
        self.tabs.addTab(self._create_build_tree_tab(),
        "Построение дерева")
        self.tabs.addTab(self._create_results_tab(),
        "Результаты анализа")
        self.tabs.addTab(self._create_visual_tab(),
        "Визуализация")
        self.tabs.addTab(self._create_export_tab(),
        "Экспорт")
        self.status_bar = QStatusBar()
        self.setStatusBar(self.status_bar)
        def _build_actions(self):
            self.action_load_tree =
            QAction("Загрузить/визуализировать дерево", self)
            self.action_load_alignment =
            QAction("Загрузить NEXUS/FASTA", self)

```



```

        self.action_load_model = QAction("Загрузить
модель", self)
        self.action_validate = QAction("Проверить
данные", self)
        self.action_build_tree = QAction("Построить
дерево", self)
        self.action_visualize_tree
=
QAction("Визуализировать дерево из файла", self)
        self.action_predict = QAction("Предсказать
bootstrap", self)
        self.action_export_tree
=
QAction("Сохранить дерево", self)
        self.action_about = QAction("О программе",
self)
        self.action_exit = QAction("Выход", self)
    def _build_menu(self):
        menu_file = self.menuBar().addMenu("Файл")

        menu_file.addAction(self.action_load_alignment)
        menu_file.addAction(self.action_load_tree)

        menu_file.addAction(self.action_load_model)
        menu_file.addSeparator()

        menu_file.addAction(self.action_export_tree)
        menu_file.addSeparator()
        menu_file.addAction(self.action_exit)
        menu_analysis
=
self.menuBar().addMenu("Phylo")

        menu_analysis.addAction(self.action_validate)

        menu_analysis.addAction(self.action_build_tree)

        menu_analysis.addAction(self.action_predict)

        menu_analysis.addAction(self.action_visualize_tree
)
        menu_help
=
self.menuBar().addMenu("Справка")
        menu_help.addAction(self.action_about)
    def _build_toolbar(self):
        toolbar = QToolBar("Основные действия")
        toolbar.setMovable(False)
        self.addToolBar(toolbar)
        for action in [
            self.action_load_alignment,
            self.action_load_tree,
            self.action_load_model,
            self.action_validate,
            self.action_build_tree,
            self.action_predict,
            self.action_visualize_tree,
            self.action_export_tree,
        ]:
            toolbar.addAction(action)
    def _build_header(self):
        frame = QFrame()
        frame.setFrameShape(QFrame.StyledPanel)
        layout = QVBoxLayout(frame)
        title = QLabel(APP_TITLE)
        font = QFont()
        font.setPointSize(15)
        font.setBold(True)
        title.setFont(font)
        subtitle = QLabel("Загрузка NEXUS/FASTA,
построение деревьев NJ/ML и ML-предсказание
поддержки для внутренних узлов.")
        subtitle.setStyleSheet("color: #555;")
        layout.addWidget(title)
        layout.addWidget(subtitle)
        return frame
    def _create_input_tab(self):
        page = QWidget()
        layout = QVBoxLayout(page)
        top_grid = QGridLayout()

```

```

        box_alignment
=
QGroupBox("Загрузка
выравнивания")
        alignment_layout
=
QVBoxLayout(box_alignment)
        self.alignment_path_edit = QLineEdit()

        self.alignment_path_edit.setPlaceholderText("Путь к
файлу .nex / .nexus / .fasta")
        self.btn_load_alignment
=
QPushButton("Загрузить NEXUS/FASTA")

        alignment_layout.addWidget(self.alignment_path_edit)

        alignment_layout.addWidget(self.btn_load_alignment
)
        box_tree
=
QGroupBox("Загрузка готового
дерева")
        tree_layout = QVBoxLayout(box_tree)
        self.tree_path_edit = QLineEdit()

        self.tree_path_edit.setPlaceholderText("Путь к
файлу дерева .nex / .nexus / .nwk / .newick")
        self.btn_load_tree = QPushButton("Загрузить
и визуализировать дерево")
        tree_layout.addWidget(self.tree_path_edit)
        tree_layout.addWidget(self.btn_load_tree)
        box_model
=
QGroupBox("Модель Random
Forest")
        model_layout = QVBoxLayout(box_model)
        self.combo_model_select = QComboBox()
        self.combo_model_select.addItem("Выберите
модель...", None)
        for item in self.preloaded_models:

            self.combo_model_select.addItem(item["label"],
item)
            self.model_path_edit = QLineEdit()
            self.model_path_edit.setReadOnly(True)

            self.model_path_edit.setPlaceholderText("Путь к
выбранной модели")
            self.btn_load_model
=
QPushButton("Загрузить выбранную модель")

            model_layout.addWidget(self.combo_model_select)

            model_layout.addWidget(self.model_path_edit)

            model_layout.addWidget(self.btn_load_model)
            box_validation = QGroupBox("Сводка входных
данных")
            validation_layout
=
QVBoxLayout(box_validation)
            self.validation_notes = QTextEdit()
            self.validation_notes.setReadOnly(True)
            self.btn_validate = QPushButton("Проверить
входные данные")

            validation_layout.addWidget(self.validation_notes)

            validation_layout.addWidget(self.btn_validate)
            top_grid.addWidget(box_alignment, 0, 0)
            top_grid.addWidget(box_tree, 0, 1)
            top_grid.addWidget(box_model, 0, 2)
            top_grid.addWidget(box_validation, 1, 0, 1,
3)

            layout.addLayout(top_grid)
            state_box = QGroupBox("Состояние данных")
            form = QFormLayout(state_box)
            self.lbl_tree_loaded = QLabel("-")
            self.lbl_alignment_loaded = QLabel("-")
            self.lbl_model_loaded = QLabel("-")
            self.lbl_validated = QLabel("-")
            self.lbl_source_format = QLabel("-")

```

```

        form.addRow("Дерево загружено:",
self.lbl_tree_loaded)
        form.addRow("Выравнивание загружено:",
self.lbl_alignment_loaded)
        form.addRow("Модель загружена:",
self.lbl_model_loaded)
        form.addRow("Проверка выполнена:",
self.lbl_validated)
        form.addRow("Формат входа:",
self.lbl_source_format)
        layout.addWidget(state_box)
        layout.addStretch(1)
        return page
    def _create_build_tree_tab(self):
        page = QWidget()
        layout = QVBoxLayout(page)
        config_box = QGroupBox("Параметры
построения дерева")
        form = QFormLayout(config_box)
        self.combo_tree_method = QComboBox()
        self.combo_tree_method.addItem("Neighbor
Joining", "Maximum Likelihood"])
        self.combo_distance = QComboBox()
        self.combo_distance.addItem(["p-distance
(pairwise deletion)"])
        form.addRow("Метод построения:",
self.combo_tree_method)
        form.addRow("Метрика расстояния:",
self.combo_distance)
        layout.addWidget(config_box)
        buttons = QHBoxLayout()
        self.btn_build_tree =
QPushButton("Построить дерево")
        self.btn_visualize_tree =
QPushButton("Визуализировать дерево из файла")
        self.btn_predict = QPushButton("Предсказать
bootstrap")
        self.btn_save_tree = QPushButton("Сохранить
дерево")
        buttons.addWidget(self.btn_build_tree)
        buttons.addWidget(self.btn_visualize_tree)
        buttons.addWidget(self.btn_predict)
        buttons.addWidget(self.btn_save_tree)
        buttons.addStretch(1)
        layout.addLayout(buttons)
        preview_box = QGroupBox("Сводка дерева")
        preview_layout = QVBoxLayout(preview_box)
        self.tree_build_preview = QTextEdit()
        self.tree_build_preview.setReadOnly(True)
        preview_layout.addWidget(self.tree_build_preview)
        layout.addWidget(preview_box, stretch=1)
        state_box = QGroupBox("Состояние
построения")
        form2 = QFormLayout(state_box)
        self.lbl_tree_built = QLabel("-")
        self.lbl_tree_source = QLabel("Не
определён")
        form2.addRow("Дерево построено:",
self.lbl_tree_built)
        form2.addRow("Источник дерева:",
self.lbl_tree_source)
        layout.addWidget(state_box)
        return page
    def _create_results_tab(self):
        page = QWidget()
        layout = QVBoxLayout(page)
        self.results_table = QTableWidgetItem(0, 8)
        self.results_table.setHorizontalHeaderLabels([
            "ID узла",
            "Длина ветви",
            "Глубина",
            "Листьев",
            "Поддержка узла",
            "Predicted RF",
            "Источник",
            "Листья поддерева",
        ])
        self.results_table.horizontalHeader().setStretchLa
stSection(True)
        layout.addWidget(self.results_table,
stretch=1)
        details_box = QGroupBox("Подробности по
узлам")
        details_layout = QVBoxLayout(details_box)
        self.result_details = QTextEdit()
        self.result_details.setReadOnly(True)
        details_layout.addWidget(self.result_details)
        layout.addWidget(details_box)
        return page
    def _create_visual_tab(self):
        page = QWidget()
        layout = QVBoxLayout(page)
        controls = QHBoxLayout()
        self.chk_show_branch_lengths =
QCheckBox("Показывать длины ветвей")
        self.chk_show_branch_lengths.setChecked(False)
        self.chk_show_predictions =
QCheckBox("Подсветка по predicted bootstrap")
        self.chk_show_predictions.setChecked(True)
        self.btn_zoom_in = QPushButton("+")
        self.btn_zoom_out = QPushButton("-")
        self.btn_fit_tree = QPushButton("Fit")
        self.leaf_spacing_slider =
QSlider(Qt.Horizontal)
        self.leaf_spacing_slider.setMinimum(6)
        self.leaf_spacing_slider.setMaximum(24)
        self.leaf_spacing_slider.setValue(10)
        self.leaf_spacing_slider.setTracking(False)
        controls.addWidget(self.chk_show_branch_lengths)
        controls.addWidget(self.chk_show_predictions)
        controls.addSpacing(16)
        controls.addWidget(QLabel("Масштаб:"))
        controls.addWidget(self.btn_zoom_out)
        controls.addWidget(self.btn_zoom_in)
        controls.addWidget(self.btn_fit_tree)
        controls.addSpacing(16)
        controls.addWidget(QLabel("Интервал
листьев:"))
        controls.addWidget(self.leaf_spacing_slider,
stretch=1)
        layout.addLayout(controls)
        self.tree_canvas = PhyloTreeCanvas()
        self.tree_scroll = QScrollArea()
        self.tree_scroll.setWidget(self.tree_canvas)
        self.tree_scroll.setWidgetResizable(False)
        self.tree_scroll.setAlignment(Qt.AlignLeft
| Qt.AlignTop)
        layout.addWidget(self.tree_scroll,
stretch=1)
        return page
    def _create_export_tab(self):
        page = QWidget()
        layout = QVBoxLayout(page)
        self.export_log = QPlainTextEdit()
        self.export_log.setReadOnly(True)
        self.export_log.setPlaceholderText("История
сохранения дерева и промежуточных файлов.")
        layout.addWidget(self.export_log)
        return page
    def _connect_signals(self):

```

```

self.action_load_alignment.triggered.connect(self.
load_alignment)

self.action_load_tree.triggered.connect(self.load_
tree)

self.action_load_model.triggered.connect(self.load_
_model)

self.action_validate.triggered.connect(self.valida
te_input)

self.action_build_tree.triggered.connect(self.buil
d_tree)

self.action_visualize_tree.triggered.connect(self.
load_tree)

self.action_predict.triggered.connect(self.predict_
_bootstrap)

self.action_export_tree.triggered.connect(self.exp
ort_tree)

self.action_about.triggered.connect(self.show_abou
t)

self.action_exit.triggered.connect(self.close)

self.btn_load_alignment.clicked.connect(self.load_
alignment)

self.btn_load_tree.clicked.connect(self.load_tree)

self.btn_load_model.clicked.connect(self.load_mode
l)

self.combo_model_select.currentIndexChanged.conne
ct(self._sync_selected_model_path)

self.btn_validate.clicked.connect(self.validate_in
put)

self.btn_build_tree.clicked.connect(self.build_tre
e)

self.btn_visualize_tree.clicked.connect(self.load_
tree)

self.btn_predict.clicked.connect(self.predict_boot
strap)

self.btn_save_tree.clicked.connect(self.export_tre
e)

self.nav_list.currentRowChanged.connect(self.tabs.
setCurrentIndex)

self.results_table.itemSelectionChanged.connect(se
lf.show_selected_node_details)

self.chk_show_branch_lengths.toggled.connect(self.
_update_tree_view)

self.chk_show_predictions.toggled.connect(self._u
pdate_tree_view)

self.btn_zoom_in.clicked.connect(self._zoom_in_tre
e)

self.btn_zoom_out.clicked.connect(self._zoom_out_t
ree)

self.btn_fit_tree.clicked.connect(self._fit_tree)

```

```

self.leaf_spacing_slider.sliderReleased.connect(se
lf._apply_leaf_spacing)
    def _show_busy(self, title: str, text: str) ->
QProgressDialog:
    dlg = QProgressDialog(text, None, 0, 0,
self)
        dlg.setWindowTitle(title)
        dlg.setWindowModality(Qt.ApplicationModal)
        dlg.setMinimumDuration(0)
        dlg.setCancelButton(None)
        dlg.setAutoClose(False)
        dlg.setAutoReset(False)
        dlg.show()

QApplication.setOverrideCursor(Qt.WaitCursor)
QApplication.processEvents()
    return dlg
    def _hide_busy(self, dlg: QProgressDialog |
None):
        QApplication.restoreOverrideCursor()
        if dlg is not None:
            dlg.close()
            dlg.deleteLater()
        QApplication.processEvents()
    def _set_build_controls_enabled(self, enabled:
bool):
        self.btn_build_tree.setEnabled(enabled)
        self.action_build_tree.setEnabled(enabled)
        self.combo_tree_method.setEnabled(enabled)
        self.combo_distance.setEnabled(enabled)
        def _start_background_tree_build(self, method:
str):
            self._build_tree_busy =
self._show_busy("Построение дерева", "ML-дерево
строится в фоне. Окно остаётся отзывчивым,
пожалуйста подождите...")
            self.status_bar.showMessage("Строится ML-
дерево...", 0)
            self._set_build_controls_enabled(False)
            self._build_tree_thread = QThread(self)
            self._build_tree_worker =
TreeBuildWorker(self.state.alignment, method)

self._build_tree_worker.moveToThread(self._build_t
ree_thread)

self._build_tree_thread.started.connect(self._bui
ld_tree_worker.run)

self._build_tree_worker.finished.connect(self._on_
background_tree_build)

self._build_tree_worker.failed.connect(self._on_ba
ckground_tree_build_failed)

self._build_tree_worker.finished.connect(self._bui
ld_tree_thread.quit)

self._build_tree_worker.failed.connect(self._build_
tree_thread.quit)

self._build_tree_thread.finished.connect(self._bui
ld_tree_worker.deleteLater)

self._build_tree_thread.finished.connect(self._bui
ld_tree_thread.deleteLater)
    self._build_tree_thread.start()
    def _on_background_tree_build(self, artifacts):
        self._hide_busy(self._build_tree_busy)
        self._build_tree_busy = None
        self._set_build_controls_enabled(True)
        self._build_tree_thread = None
        self._build_tree_worker = None

self._apply_built_tree_artifacts(artifacts)

```

```

def _on_background_tree_build_failed(self,
error_text: str):
    self._hide_busy(self._build_tree_busy)
    self._build_tree_busy = None
    self._set_build_controls_enabled(True)
    self._build_tree_thread = None
    self._build_tree_worker = None
    self._error(f"Не удалось построить дерево:
{error_text}")
    def _apply_built_tree_artifacts(self,
artifacts: AnalysisArtifacts):
        self.state.tree = artifacts.tree
        self.state.tree_built = True
        self.state.tree_loaded = True
        self.state.node_rows = artifacts.node_rows
        self.state.tree_method =
artifacts.tree_info.get("build_method",
self.combo_tree_method.currentText())
        self.state.tree_support_kind =
artifacts.tree_info.get("support_kind", "observed")
        self.state.tree_support_label =
artifacts.tree_info.get("support_label",
"Bootstrap")
        self.lbl_tree_source.setText("Построено по
выравниванию")
        self._apply_tree_artifacts(artifacts)
        self.tabs.setCurrentIndex(3)
        self.nav_list.setCurrentRow(3)
        self._append_log(f"Построено           дерево
методом: {self.state.tree_method}.")
        self.status_bar.showMessage(f"Дерево
построено: {self.state.tree_method}", 4000)
        self._refresh_state_view()
        def load_alignment(self):
            path, _ = QFileDialog.getOpenFileName(self,
"Выбор файла выравнивания", "", "Alignment files
(*.nex *.nexus *.nexorg *.fasta *.fa *.fas
*.txt);;All files (*.*)")
            if not path:
                return
            try:
                self.service.inspect_alignment_file(path)
                artifacts =
self.service.load_alignment(path)
            except Exception as exc:
                self._error(f"Не удалось загрузить
выравнивание: {exc}")
                return
            self.state.alignment_file = path
            self.state.alignment_loaded = True
            self.state.alignment = artifacts.alignment
            self.state.source_format =
f"alignment:{Path(path).suffix.lower()}"
            self.alignment_path_edit.setText(path)
            self._fill_alignment_summary(artifacts)
            self._append_log(f"Загружено выравнивание:
{path}")
            self.status_bar.showMessage("Выравнивание
загружено", 4000)
            self._refresh_state_view()
            def load_tree(self):
                path, _ = QFileDialog.getOpenFileName(self,
"Выбор файла дерева", "", "Tree files (*.nex *.nexus
*.nwk *.newick *.tree *.txt);;All files (*.*)")
                if not path:
                    return
                try:
                    self.service.inspect_tree_file(path)
                    artifacts =
self.service.load_tree(path)
                except Exception as exc:
                    self._error(f"Не удалось загрузить
дерево: {exc}")
                    return
                self.state.tree_file = path

                self.state.tree_loaded = True
                self.state.tree_built = True
                self.state.tree = artifacts.tree
                self.state.node_rows = artifacts.node_rows
                self.state.tree_method =
artifacts.tree_info.get("build_method",
"Loaded from file")
                self.state.tree_support_kind =
artifacts.tree_info.get("support_kind", "observed")
                self.state.tree_support_label =
artifacts.tree_info.get("support_label",
"Bootstrap")
                self.tree_path_edit.setText(path)
                self.lbl_tree_source.setText("Загружено из
файла и визуализировано")
                self._apply_tree_artifacts(artifacts)
                self.tabs.setCurrentIndex(3)
                self.nav_list.setCurrentRow(3)
                self._append_log(f"Загружено           и
визуализировано дерево: {path}")
                self.status_bar.showMessage("Файл         дерева
загружен и визуализирован", 4000)
                self._refresh_state_view()
                def _sync_selected_model_path(self):
                    data =
self.combo_model_select.currentData()
                    if isinstance(data, dict):
                        self.model_path_edit.setText(data.get("path", ""))
                    else:
                        self.model_path_edit.clear()
                def load_model(self):
                    data =
self.combo_model_select.currentData()
                    if not isinstance(data, dict):
                        self._error("Сначала выберите модель в
выпадающем списке.")
                        return
                    model_path = data["path"]
                    metadata_path = data.get("metadata")
                    busy = self._show_busy("Загрузка модели",
"Загружается модель Random Forest...")
                    try:
                        metadata, loaded_now =
self.service.load_model(model_path, metadata_path)
                    except Exception as exc:
                        self._hide_busy(busy)
                        self._error(f"Не удалось загрузить
модель: {exc}")
                        return
                    self._hide_busy(busy)
                    self.state.model_file = model_path
                    self.state.model_loaded = True
                    self.state.model_metadata = metadata
                    self.model_path_edit.setText(model_path)
                    if loaded_now:
                        self._append_log(f"Загружена модель:
{model_path}")
                        self.status_bar.showMessage("Модель
загружена", 4000)
                    else:
                        self._append_log(f"Модель уже была в
памяти: {model_path}")
                        self.status_bar.showMessage("Модель уже
загружена, повторная загрузка не требуется", 5000)
                    self._refresh_state_view()
                    def validate_input(self):
                        lines = []
                        self.state.input_validated = False
                        self.state.taxa_consistent = True
                        if self.state.alignment_file:
                            try:
                                meta =
self.service.inspect_alignment_file(self.state.ali
gnment_file)

```

```

        lines.append(f"Выравнивание: файл
корректный ({meta['format']}).")
        lines.append("    Найдены блоки: " +
", ".join(meta.get("present_blocks", [])))
        if self.state.alignment is not
None:
            aln = self.state.alignment
            lines.append(f"    Таксонов:
{len(aln)},    длина    выравнивания:
{aln.get_alignment_length()}.")
            except Exception as exc:
                lines.append(f"Выравнивание: ошибка
проверки – {exc}")
            else:
                lines.append("Выравнивание    не
загружено.")
                if self.state.tree_file:
                    try:
                        meta =
self.service.inspect_tree_file(self.state.tree_fil
e)
                        lines.append(f"Дерево:    файл
корректный ({meta['format']}).")
                        lines.append("    Найдены блоки: " +
", ".join(meta.get("present_blocks", [])))
                        if self.state.tree is not None:
                            lines.append(f"    Терминальных
узлов: {len(self.state.tree.get_terminals())}.")
                            except Exception as exc:
                                lines.append(f"Дерево:    ошибка
проверки – {exc}")
                            else:
                                lines.append("Дерево не загружено.")
                                if self.state.model_loaded:
                                    lines.append("Модель Random Forest
загружена.")
                                else:
                                    lines.append("Модель Random Forest не
загружена.")
                                if self.state.alignment is not None and
self.state.tree is not None:
                                    aln_taxa = {record.id for record in
self.state.alignment}
                                    tree_taxa = {clade.name for clade in
self.state.tree.get_terminals() if clade.name}
                                    common = aln_taxa & tree_taxa
                                    only_in_aln = aln_taxa - tree_taxa
                                    only_in_tree = tree_taxa - aln_taxa
                                    if not common:
                                        lines.append("\nКРИТИЧЕСКОЕ
НЕСОВПАДЕНИЕ: Множества таксонов полностью не
пересекаются!")
                                        lines.append(f"    В выравнивании,
но нет в дереве: {'', '.join(sorted(only_in_aln)) or
'-'})")
                                        lines.append(f"    В дереве, но нет
в выравнивании: {'', '.join(sorted(only_in_tree)) or
'-'})")
                                        self.state.taxa_consistent = False
                                        elif only_in_aln or only_in_tree:
                                            lines.append("\n⚠    ЧАСТИЧНОЕ
НЕСОВПАДЕНИЕ таксонов:")
                                            if only_in_aln:
                                                lines.append(f"    Отсутствуют в
дереве: {'', '.join(sorted(only_in_aln))}")
                                            if only_in_tree:
                                                lines.append(f"    Отсутствуют в
выравнивании: {'', '.join(sorted(only_in_tree))}")
                                            lines.append(f"    Совпадает:
{len(common)}/{len(aln_taxa)} таксонов.")
                                            self.state.taxa_consistent = False
                                            else:
                                                lines.append(f"\nВсё    таксоны
полностью    согласованы
({len(common)}/{len(aln_taxa)}).")

```

```

self.validation_notes.setPlainText("\n".join(lines
))
        self.state.input_validated
=
self.state.taxa_consistent and any(" in line for
line in lines)
        self._refresh_state_view()
        self._append_log("Проверка входных данных
выполнена.")
        def build_tree(self):
            if self.state.alignment is None:
                self._error("Для построения дерева
сначала загрузите NEXUS/FASTA файл с
выравниванием.")
                return
            method
=
self.combo_tree_method.currentText()
            if method == "Maximum Likelihood":
                self._start_background_tree_build(method)
                return
            try:
                artifacts
=
self.service.build_tree_from_alignment(self.state.
alignment, method=method)
                except Exception as exc:
                    self._error(f"Не удалось построить
дерево: {exc}")
                    return
                self._apply_built_tree_artifacts(artifacts)
                def predict_bootstrap(self):
                    if self.state.tree is None or
self.state.alignment is None:
                        self._error("Для предсказания нужно
загрузить выравнивание и дерево.")
                        return
                    if not self.state.model_loaded:
                        self._error("Сначала выберите и
загрузите модель Random Forest.")
                        return
                    aln_taxa = {record.id for record in
self.state.alignment}
                    tree_taxa = {clade.name for clade in
self.state.tree.get_terminals() if clade.name}
                    if aln_taxa != tree_taxa:
                        missing_in_tree = aln_taxa - tree_taxa
                        extra_in_tree = tree_taxa - aln_taxa
                        msg = "Предсказание заблокировано:
выравнивание и дерево не парные.\n\n"
                        msg += "\nЗагрузите дерево, построенное
именно на этом выравнивании, или сначала постройте
его заново."
                        self._error(msg)
                        return
                    busy
=
self._show_busy("Предсказание
bootstrap", "Модель считает predicted bootstrap для
внутренних узлов...")
                    try:
                        predictions
=
self.service.predict_bootstrap(self.state.tree,
self.state.alignment)
                        except Exception as exc:
                            self._hide_busy(busy)
                            self._error(f"Не удалось выполнить
предсказание: {exc}")
                            return
                            self._hide_busy(busy)
                            pred_map
=
{row["node_id"]:
row["predicted_bootstrap"] for row in predictions}
                            pred_rows = {row["node_id"]: row for row in
predictions}
                            for row in self.state.node_rows:
                                pred_info
=
pred_rows.get(row["node_id"], {})

```

```

        row["predicted_bootstrap"] =
pred_info.get("predicted_bootstrap")
        row["feature_influences"] =
pred_info.get("feature_influences", [])
        row["feature_influence_text"] =
pred_info.get("feature_influence_text", "")
        if row["predicted_bootstrap"] is not
None:
            row["support_source"] =
"predicted_rf"
            self.state.analysis_completed = True

self._populate_node_rows(self.state.node_rows)
self._update_tree_view()
self.tabs.setCurrentIndex(2)
self.nav_list.setCurrentRow(2)
self._append_log("Выполнено ML-предсказание
bootstrap для внутренних узлов и рассчитаны влияющие
параметры.")
self.status_bar.showMessage("Predicted
bootstrap рассчитан", 4000)
self._refresh_state_view()
def export_tree(self):
    if self.state.tree is None:
        self._error("Нет дерева для
сохранения.")
        return
    path, _ = QFileDialog.getSaveFileName(self,
"Сохранить дерево", "tree_built.nwk", "Newick
(*.nwk *.newick);;All files (*.*)")
    if not path:
        return
    try:
        from phylo.nj_builder import
save_tree_newick
        save_tree_newick(self.state.tree, path)
    except Exception as exc:
        self._error(f"Не удалось сохранить
дерево: {exc}")
    return

self.export_log.appendPlainText(f"[{self._now()}]
Сохранено дерево: {path}")
self._append_log(f"Дерево сохранено в
{path}")
self.status_bar.showMessage("Дерево
сохранено", 4000)
def show_selected_node_details(self):
    row = self.results_table.currentRow()
    if row < 0 or row >=
len(self.state.node_rows):
        return
    info = self.state.node_rows[row]
    support = "-" if info.get("bootstrap") is
None else f"{info['bootstrap']:.3f}"
    predicted = "-" if
info.get("predicted_bootstrap") is None else
f"{info['predicted_bootstrap']:.2f}"
    influence =
info.get("feature_influence_text") or "Нет данных о
влияющих параметрах."
    self.result_details.setPlainText(
        f"Узел: {info['node_id']}\n"
        f"Длина ветви:
{info['branch_length']:.6f}\n"
        f"Глубина: {info['depth']:.6f}\n"
        f"Количество листьев:
{info['leaf_count']}\n"
        f"{self.state.tree_support_label}:
{support}\n"
        f"Predicted bootstrap (RF):
{predicted}\n"
        f"Источник: {info.get('support_source',
'-')}\n"
        f"Листья поддерева:
{info['descendants']}\n\n")

        f"{influence}"
    )
    def _fill_alignment_summary(self, artifacts:
AnalysisArtifacts):
        info = artifacts.alignment_info
        self.validation_notes.setPlainText(
            f"Alignment загружен успешно.\n"
            f"Таксонов: {info.get('taxa_count',
0)}\n"
            f"Длина выравнивания:
{info.get('length', 0)}\n"
            f"Доля gap/missing:
{info.get('gap_fraction', 0.0):.4f}\n"
            f"Перед анализом можно запустить
проверку структуры файла."
        )
    def _apply_tree_artifacts(self, artifacts:
AnalysisArtifacts):
        tree_info = artifacts.tree_info
        support_min = tree_info.get('support_min')
        support_max = tree_info.get('support_max')
        support_range = '-' if support_min is None
or support_max is None else f"{support_min:.2f} ..
{support_max:.2f}"
        self.tree_build_preview.setPlainText(
            f"Метод: {tree_info.get('build_method',
self.state.tree_method)}\n"
            f"Терминальных узлов:
{tree_info.get('taxa_count', 0)}\n"
            f"Внутренних узлов:
{tree_info.get('internal_nodes', 0)}\n"
            f"Суммарная длина ветвей:
{tree_info.get('total_branch_length', 0.0):.6f}\n"
            f"Rooted: {'Да' if
tree_info.get('is_rooted') else 'Нет'}\n"
            f"Поддержка узлов:
{tree_info.get('support_label',
self.state.tree_support_label)}\n"
            f"Диапазон support: {support_range}"
        )
    )
    self._populate_node_rows(artifacts.node_rows)
    self._update_tree_view()
    def _populate_node_rows(self, rows:
list[dict]):
        self.results_table.setRowCount(len(rows))
        for row_idx, info in enumerate(rows):
            values = [
                info['node_id'],
                f"{info['branch_length']:.6f}",
                f"{info['depth']:.6f}",
                str(info['leaf_count']),
                '-' if info.get('bootstrap') is
None else f"{info['bootstrap']:.3f}",
                '-' if
info.get('predicted_bootstrap') is None else
f"{info['predicted_bootstrap']:.2f}",
                info.get('support_source', '-'),
                info['descendants'],
            ]
            for col_idx, value in
enumerate(values):
                item = QTableWidgetItem(value)
                tooltip =
info.get('feature_influence_text')
                if tooltip:
                    item.setToolTip(tooltip)
                if col_idx == 5 and
info.get('predicted_bootstrap') is not None:
                    pred =
float(info['predicted_bootstrap'])
                    if pred < 50:
                        item.setBackground(Qt.red)
                    elif pred < 75:
                        item.setBackground(Qt.yellow)

```

```

        else:
            item.setBackground(Qt.green)

self.results_table.setItem(row_idx, col_idx, item)
    if rows:
        self.results_table.selectRow(0)
        self.show_selected_node_details()
    def _update_tree_view(self):
        prediction_map = {row['node_id']:
row['predicted_bootstrap'] for row in
self.state.node_rows if
row.get('predicted_bootstrap') is not None}
        support_map = {row['node_id']:
row['bootstrap'] for row in self.state.node_rows if
row.get('bootstrap') is not None}
        show_support = self.state.tree_support_kind
== "ml_ultrafast_bootstrap"
        node_id_map =
build_clade_node_id_map(self.state.tree) if
self.state.tree is not None else {}
        node_tooltip_map = {row['node_id']:
row.get('feature_influence_text', '') for row in
self.state.node_rows if
row.get('feature_influence_text')}
        self.tree_canvas.set_tree(
            self.state.tree,

self.chk_show_branch_lengths.isChecked(),
        prediction_map,
        self.chk_show_predictions.isChecked(),
        node_id_map,
        support_map=support_map,
        show_support_values=show_support,

support_label=self.state.tree_support_label,
        node_tooltip_map=node_tooltip_map,
    )
    def _zoom_in_tree(self):
        self.tree_canvas.zoom_in()
    def _zoom_out_tree(self):
        self.tree_canvas.zoom_out()
    def _fit_tree(self):
        self.tree_canvas.fit_view()
    def _apply_leaf_spacing(self):

self.tree_canvas.set_leaf_spacing(self.leaf_spacin
g_slider.value() / 10.0)
    def _refresh_state_view(self):
        self.lbl_tree_loaded.setText("Да" if
self.state.tree_loaded else "Нет")
        self.lbl_alignment_loaded.setText("Да" if
self.state.alignment_loaded else "Нет")
        self.lbl_model_loaded.setText("Да" if
self.state.model_loaded else "Нет")
        self.lbl_validated.setText("Да" if
self.state.input_validated else "Нет")
        self.lbl_tree_built.setText("Да" if
self.state.tree_built else "Нет")

self.lbl_source_format.setText(self.state.source_f
ormat or "-")
    def _append_log(self, text: str):
        timestamped = f"[{self._now()}] {text}"
        if hasattr(self, 'export_log'):

self.export_log.appendPlainText(timestamped)
    def _error(self, text: str):
        self.state.last_error = text
        QMessageBox.critical(self, "Ошибка", text)
        self.status_bar.showMessage(text, 6000)
    def _now(self):
        return datetime.now().strftime("%H:%M:%S")
    def show_about(self):
        QMessageBox.about(self, "О программе",
"Рабочий прототип для диплома:\n- загрузка

```

```

NEXUS/FASTA;\n- построение NJ-дерева;\n-
предсказание bootstrap моделью Random Forest;\n-
визуализация дерева с подсветкой по достоверности.")
def run():
    app = QApplication(sys.argv)
    app.setApplicationName(APP_TITLE)
    window = MainWindow()
    window.show()
    sys.exit(app.exec_())
from __future__ import annotations
from dataclasses import dataclass
from typing import Dict
from Bio.Phylo.BaseTree import Clade, Tree
from matplotlib.backends.backend_qt5agg import
FigureCanvasQTagg as FigureCanvas
from matplotlib.figure import Figure
from PyQt5.QtCore import QPointF, Qt
from PyQt5.QtGui import QMouseEvent, QWheelEvent
from PyQt5.QtWidgets import QToolTip
@dataclass
class LayoutNode:
    x: float
    y: float
class PhyloTreeCanvas(FigureCanvas):
    def __init__(self, parent=None):
        self.figure = Figure(figsize=(16, 10),
tight_layout=False)
        super().__init__(self.figure)
        self.setParent(parent)
        self._tree: Tree | None = None
        self._show_branch_lengths = False
        self._prediction_map: dict[str, float] = {}
        self._show_predictions = True
        self._node_id_map: Dict[Clade, str] = {}
        self._support_map: dict[str, float] = {}
        self._show_support_values = False
        self._support_label = "Bootstrap"
        self._node_tooltip_map: dict[str, str] = {}
        self._node_screen_points: list[tuple[float,
float, str]] = []
        self._node_data_points: list[tuple[float,
float, str]] = []
        self._zoom = 1.0
        self._leaf_spacing = 1.0
        self._dragging = False
        self._last_mouse_pos = QPointF()
        self._axes = None
        self._base_xlim: tuple[float, float] | None
= None
        self._base_ylim: tuple[float, float] | None
= None
        self._layout_cache: dict[Clade, LayoutNode]
= {}
        self._terminals_cache: list[Clade] = []
        self._layout_key: tuple | None = None
        self._suspend_tooltips = False
        self.setFocusPolicy(Qt.StrongFocus)
        self.setMouseTracking(True)
    def set_tree(
        self,
        tree: Tree | None,
        show_branch_lengths: bool = False,
        prediction_map: dict[str, float] | None =
None,
        show_predictions: bool = True,
        node_id_map: Dict[Clade, str] | None = None,
        support_map: dict[str, float] | None = None,
        show_support_values: bool = False,
        support_label: str = "Bootstrap",
        node_tooltip_map: dict[str, str] | None =
None,
    ):
        old_tree = self._tree
        self._tree = tree
        self._show_branch_lengths =
show_branch_lengths

```

```

        self._prediction_map = prediction_map or {}
        self._show_predictions = show_predictions
        self._node_id_map = node_id_map or {}
        self._support_map = support_map or {}
        self._show_support_values =
show_support_values
        self._support_label = support_label
        self._node_tooltip_map = node_tooltip_map
or {}
        if old_tree is not tree:
            self._layout_key = None
            self._layout_cache = {}
            self._terminals_cache = []
            self._zoom = 1.0
            self.redraw()
        def set_leaf_spacing(self, value: float):
            value = max(0.6, min(3.0, float(value)))
            if abs(value - self._leaf_spacing) < 1e-9:
                return
            self._leaf_spacing = value
            self._layout_key = None
            self.redraw()
        def set_zoom(self, value: float):
            self._zoom = max(0.4, min(4.0,
float(value)))
            self._apply_zoom()
        def zoom_in(self):
            self.set_zoom(self._zoom * 1.15)
        def zoom_out(self):
            self.set_zoom(self._zoom / 1.15)
        def fit_view(self):
            self._zoom = 1.0
            self._apply_zoom(reset=True)
        def wheelEvent(self, event: QWheelEvent):
            if event.modifiers() & Qt.ControlModifier:
                if event.angleDelta().y() > 0:
                    self.zoom_in()
                else:
                    self.zoom_out()
            event.accept()
            return
            super().wheelEvent(event)
        def mousePressEvent(self, event: QMouseEvent):
            if event.button() == Qt.LeftButton:
                self._dragging = True
                self._last_mouse_pos = event.pos()
                self.setCursor(Qt.ClosedHandCursor)
                event.accept()
                return
            super().mousePressEvent(event)
        def mouseMoveEvent(self, event: QMouseEvent):
            if self._dragging:
                scroll =
self.parentWidget().parentWidget()
                if
self.parentWidget() is not None and
self.parentWidget().parentWidget() is not None else
None
                if scroll is not None and
hasattr(scroll, 'horizontalScrollBar'):
                    dx = event.pos().x() -
self._last_mouse_pos.x()
                    dy = event.pos().y() -
self._last_mouse_pos.y()
                scroll.horizontalScrollBar().setValue(scroll.horiz
ontalScrollBar().value() - int(dx))
                scroll.verticalScrollBar().setValue(scroll.vertica
lScrollBar().value() - int(dy))
                self._last_mouse_pos = event.pos()
                event.accept()
                return
            else:
                tooltip =
self._find_node_tooltip(event.pos().x(),
event.pos().y())

```

```

            if tooltip:
                QToolTip.showText(event.globalPos(), tooltip, self)
            else:
                QToolTip.hideText()
                super().mouseMoveEvent(event)
            def mouseReleaseEvent(self, event:
QMouseEvent):
                if event.button() == Qt.LeftButton and
self._dragging:
                    self._dragging = False
                    self.setCursor(Qt.ArrowCursor)
                    event.accept()
                    return
                super().mouseReleaseEvent(event)
            def _find_node_tooltip(self, x: float, y: float)
-> str | None:
                if not self._node_screen_points or
self._suspend_tooltips:
                    return None
                threshold = 12.0
                best_text = None
                best_dist = None
                for px, py, node_id in
self._node_screen_points:
                    d2 = (px - x) ** 2 + (py - y) ** 2
                    if d2 <= threshold ** 2 and (best_dist
is None or d2 < best_dist):
                        best_dist = d2
                        best_text =
self._node_tooltip_map.get(node_id)
                        return best_text
                def _layout_signature(self) -> tuple:
                    tree_id = id(self._tree)
                    return (tree_id, round(self._leaf_spacing,
4))
                def _ensure_layout_cache(self):
                    if self._tree is None:
                        self._layout_cache = {}
                        self._terminals_cache = []
                        return
                    key = self._layout_signature()
                    if key == self._layout_key and
self._layout_cache:
                        return
                    tree = self._tree
                    tree.ladderize(reverse=True)
                    depths = tree.depths()
                    if not max(depths.values(), default=0):
                        depths =
tree.depths(unit_branch_lengths=True)
                    terminals = tree.get_terminals()
                    spacing = self._leaf_spacing
                    y_positions: dict[Clade, float] = {term:
float((len(terminals) - i) * spacing) for i, term
in enumerate(terminals)}
                    def assign_internal_y(clade: Clade) ->
float:
                        if clade in y_positions:
                            return y_positions[clade]
                        child_ys = [assign_internal_y(ch) for
ch in clade.clades]
                        y_positions[clade] = sum(child_ys) /
len(child_ys)
                        return y_positions[clade]
                    assign_internal_y(tree.root)
                    self._layout_cache = {
                        clade:
LayoutNode(float(depths.get(clade, 0.0)),
float(y_positions[clade])),
                        for clade
in
tree.find_clades(order="preorder")}
                    self._terminals_cache = terminals
                    self._layout_key = key
                def _apply_zoom(self, reset: bool = False):

```



```

        if self._axes is None or self._base_xlim is
None or self._base_ylim is None:
            return
        ax = self._axes
        if reset:
            ax.set_xlim(*self._base_xlim)
            ax.set_ylim(*self._base_ylim)
            self.draw_idle()
            return
        base_xlim = self._base_xlim
        base_ylim = self._base_ylim
        full_w = base_xlim[1] - base_xlim[0]
        full_h = base_ylim[1] - base_ylim[0]
        zoom = max(0.4, min(4.0, self._zoom))
        new_w = full_w / zoom
        new_h = full_h / zoom
        cur_xlim = ax.get_xlim()
        cur_ylim = ax.get_ylim()
        center_x = (cur_xlim[0] + cur_xlim[1]) / 2
        if cur_xlim else (base_xlim[0] + base_xlim[1]) / 2
        center_y = (cur_ylim[0] + cur_ylim[1]) / 2
        if cur_ylim else (base_ylim[0] + base_ylim[1]) / 2
        x0 = max(base_xlim[0], center_x - new_w /
2)
        x1 = min(base_xlim[1], center_x + new_w /
2)
        y0 = max(base_ylim[0], center_y - new_h /
2)
        y1 = min(base_ylim[1], center_y + new_h /
2)
        if (x1 - x0) < new_w:
            if x0 <= base_xlim[0]:
                x1 = min(base_xlim[1], x0 + new_w)
            else:
                x0 = max(base_xlim[0], x1 - new_w)
        if (y1 - y0) < new_h:
            if y0 <= base_ylim[0]:
                y1 = min(base_ylim[1], y0 + new_h)
            else:
                y0 = max(base_ylim[0], y1 - new_h)
        ax.set_xlim(x0, x1)
        ax.set_ylim(y0, y1)
        self.draw_idle()
    def redraw(self):
        self.figure.clear()
        ax = self.figure.add_subplot(111)
        self._axes = ax
        self._node_screen_points = []
        self._node_data_points = []
        if self._tree is None:
            ax.text(0.5, 0.5, "Дерево ещё не
загружено",
                    ha="center", va="center",
                    transform=ax.transAxes)
            ax.set_axis_off()
            self._base_xlim = None
            self._base_ylim = None
            self.draw_idle()
            return
        self._ensure_layout_cache()
        tree = self._tree
        layout = self._layout_cache
        terminals = self._terminals_cache
        spacing = self._leaf_spacing
        max_x = max((n.x for n in layout.values()),
default=1.0)
        min_y = min((n.y for n in layout.values()),
default=0.0)
        max_y = max((n.y for n in layout.values()),
default=1.0)
        taxa_count = max(1, len(terminals))
        max_label_len = max((len(t.name or "?") for
t in terminals), default=10)
        fig_height = max(8.0, min(70.0, taxa_count
* 0.30 * spacing + 2.0))
        fig_width = max(14.0, min(70.0, 9.0 + max_x
* 10.0 + max_label_len * 0.14))

```

```

        self.figure.set_size_inches(fig_width,
fig_height, forward=True)
        px_w = int(fig_width * self.figure.dpi)
        px_h = int(fig_height * self.figure.dpi)
        self.setMinimumSize(px_w, px_h)
        self.resize(px_w, px_h)
        label_gap = max(0.02, max_x * 0.01)
        x_label_line_end = max_x + label_gap
        x_label_text = x_label_line_end +
max(0.003, max_x * 0.003)
        x_pred_offset = max(0.008, max_x * 0.008)
        show_node_text = len(terminals) <= 250
        show_leaf_text = len(terminals) <= 600
        self._suspend_tooltips = False
        def pred_color(pred: float | None) -> str:
            if pred is None:
                return "#444"
            if pred < 50:
                return "#c0392b"
            if pred < 75:
                return "#d68910"
            return "#1e8449"
        for parent in
tree.find_clades(order="preorder"):
            p = layout[parent]
            if parent.clades:
                ys = [layout[ch].y for ch in
parent.clades]
                ax.plot([p.x, p.x], [min(ys),
max(ys)], color="#222", lw=1.4,
solid_capstyle="round", zorder=2)
                node_id = self._node_id_map.get(parent)
                pred =
self._prediction_map.get(node_id) if node_id else
None
                node_color = pred_color(pred) if
self._show_predictions else "#222"
                if parent is not tree.root:
                    ax.scatter([p.x], [p.y], s=18,
color=node_color, zorder=4)
                    if node_id:
                        self._node_data_points.append((float(p.x),
float(p.y), node_id))
                        if show_node_text and
self._show_predictions and pred is not None:
                            ax.text(p.x + x_pred_offset,
p.y + 0.14 * spacing, f"RF {pred:.0f}", fontsize=8,
color=node_color, va="bottom", ha="left", zorder=5)
                        if show_node_text and
self._show_support_values and node_id is not None:
                            support_val =
self._support_map.get(node_id)
                            if support_val is not None:
                                ax.text(p.x +
x_pred_offset, p.y - 0.16 * spacing, f"PP
{support_val:.0f}", fontsize=8, color="#2c3e50",
va="top", ha="left", zorder=5)
                                for child in parent.clades:
                                    c = layout[child]
                                    child_id =
self._node_id_map.get(child)
                                    child_pred =
self._prediction_map.get(child_id) if child_id else
None
                                    color = pred_color(child_pred) if
self._show_predictions else "#222"
                                    ax.plot([p.x, c.x], [c.y, c.y],
color=color, lw=1.6, solid_capstyle="round",
zorder=2)
                                    if self._show_branch_lengths and
show_node_text:
                                        branch_length =
child.branch_length if child.branch_length is not
None else 0.0
                                        if abs(branch_length) > 1e-12:

```

```

        mid_x = (p.x + c.x) / 2
        ax.text(mid_x, c.y + 0.17 *
spacing, f"{max(float(branch_length), 0.0):.3f}",
fontSize=7, color="#555", ha="center", va="bottom",
zorder=5)
        if show_leaf_text:
            for term in terminals:
                pos = layout[term]
                ax.plot([pos.x, x_label_line_end],
[pos.y, pos.y], color="#666", lw=0.9, alpha=0.9,
zorder=1)
                ax.text(x_label_text, pos.y,
term.name or "?", fontsize=9, ha="left",
va="center", color="#111", zorder=6)
            else:
                for term in terminals:
                    pos = layout[term]
                    ax.plot([pos.x, x_label_line_end],
[pos.y, pos.y], color="#666", lw=0.9, alpha=0.9,
zorder=1)
                    title = "Филогенетическое дерево"
                    if self._show_support_values:
                        title += f" - {self._support_label}"
                    if not show_leaf_text:
                        title += " (упрощённый режим)"
                    ax.set_title(title, fontsize=13)
                    self._base_xlim = (-max(0.02, max_x * 0.03),
x_label_text + max(0.04, max_label_len * 0.018))

```

```

        self._base_ylim = (min_y - 1 * spacing,
max_y + 1 * spacing)
        ax.set_xlim(*self._base_xlim)
        ax.set_ylim(*self._base_ylim)
        ax.set_yticks([])
        ax.set_xticks([])
        for spine in ax.spines.values():
            spine.set_visible(False)
        ax.margins(x=0, y=0)
        self.figure.subplots_adjust(left=0.02,
right=0.985, top=0.95, bottom=0.03)
        self._apply_zoom(reset=self._zoom == 1.0)
        self.draw_idle()
        def draw_idle(self, *args, **kwargs):
            super().draw_idle(*args, **kwargs)
            self._refresh_node_screen_points()
        def _refresh_node_screen_points(self):
            if self._axes is None or not
self._node_data_points:
                return
            transformed = []
            for x, y, node_id in self._node_data_points:
                disp
                =
self._axes.transData.transform((x, y))
                transformed.append((float(disp[0]),
float(self.height() - disp[1]), node_id))
            self._node_screen_points = transformed

```

ЛИСТ С ЗАЯВЛЕНИЕМ О САМОСТОЯТЕЛЬНОСТИ РАБОТЫ

Бакалаврская работа выполнена мной совершенно самостоятельно. Все использованные в работе материалы и концепции из опубликованной научной литературы и других источников имеют ссылки на них.

Объем работы 156 листов.

Библиография 24 наименования.

Объем приложений 86 листов.

Отпечатано в __ экземпляре.

Один экземпляр сдан на кафедру.

«__» _____ 2026 г.

_____/ Л.С. Малафеев /
(подпись)